Sirius

TUTORIAL

Table of contents

Overview 5

Preparation 7

Install the sample Domain Model 7

Launch a new runtime configuration 8

Select the Sirius perspective 8

Import a sample model 9

Basic diagram 11

Create a Viewpoint Specification Project 11

Define a Viewpoint 12

Define a Diagram 13

Add a Node to the Layer 14

Set a Style to the Node 16

Add a second Node 20

Add a Relation Based Edge 21

Edit the Style of the Relation 22

Edition tools 24

Add a Palette for edition tools 24

Create a Section 24

Add a Node Creation 24

Edge Creation Tool 29

Precondition 31

Reconnect Edge Tool 32

Delete Element Tool 33

Direct Edit Label Tool 35

Advanced diagram 37

Style Customization 37

Layers 40

Filters 42

Validation Rules 44

Quick Fixes 46

Other representations 49

Object-Centered Diagram 49

Container 51

Sub Nodes 53

Border Node 55

Double-click 56

Table 58

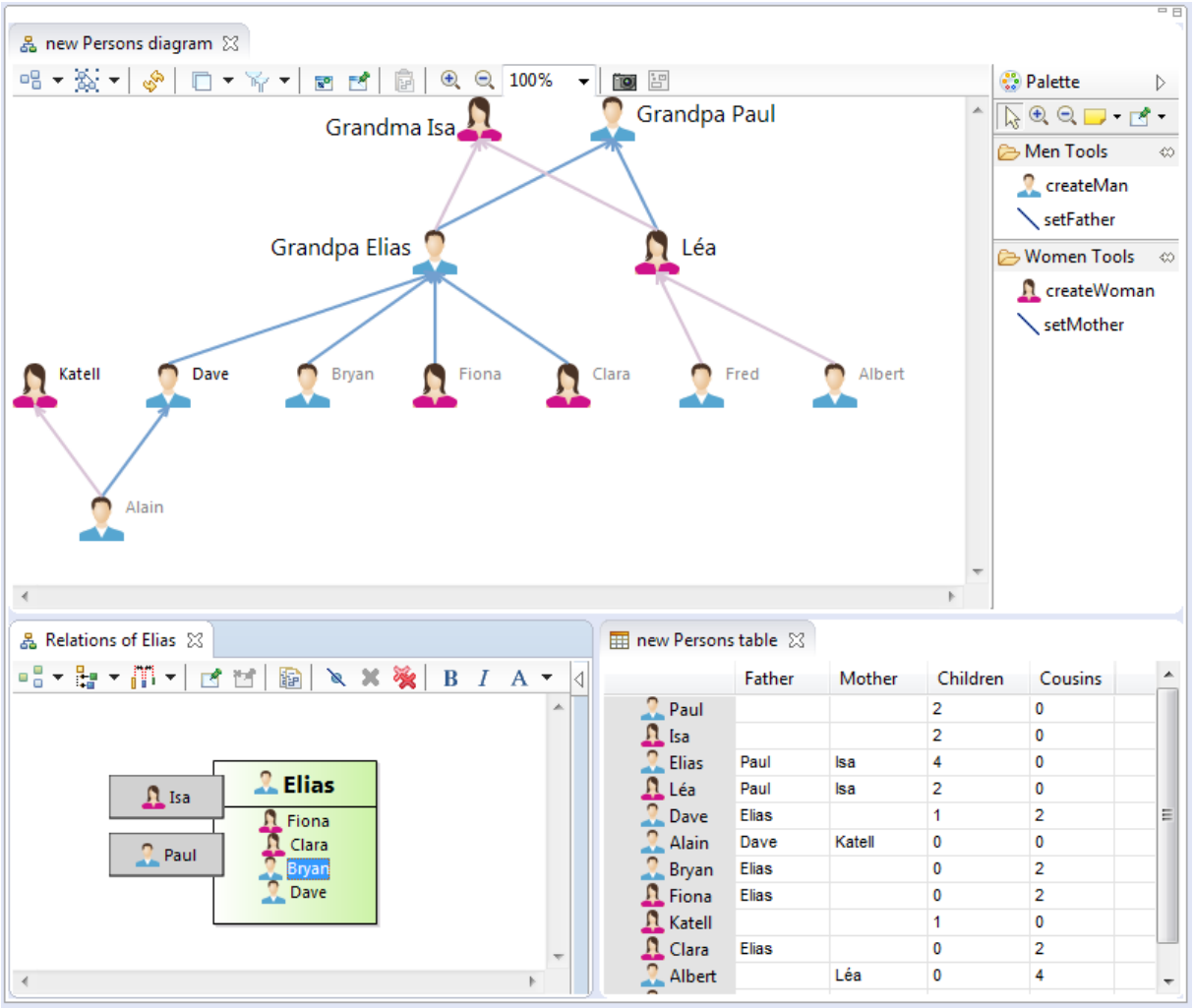
Java Services 63

Overview

This tutorial explains how to create a modeling workbench with Sirius.

These instructions are based on a simple Domain Model which describes basic concepts about families.

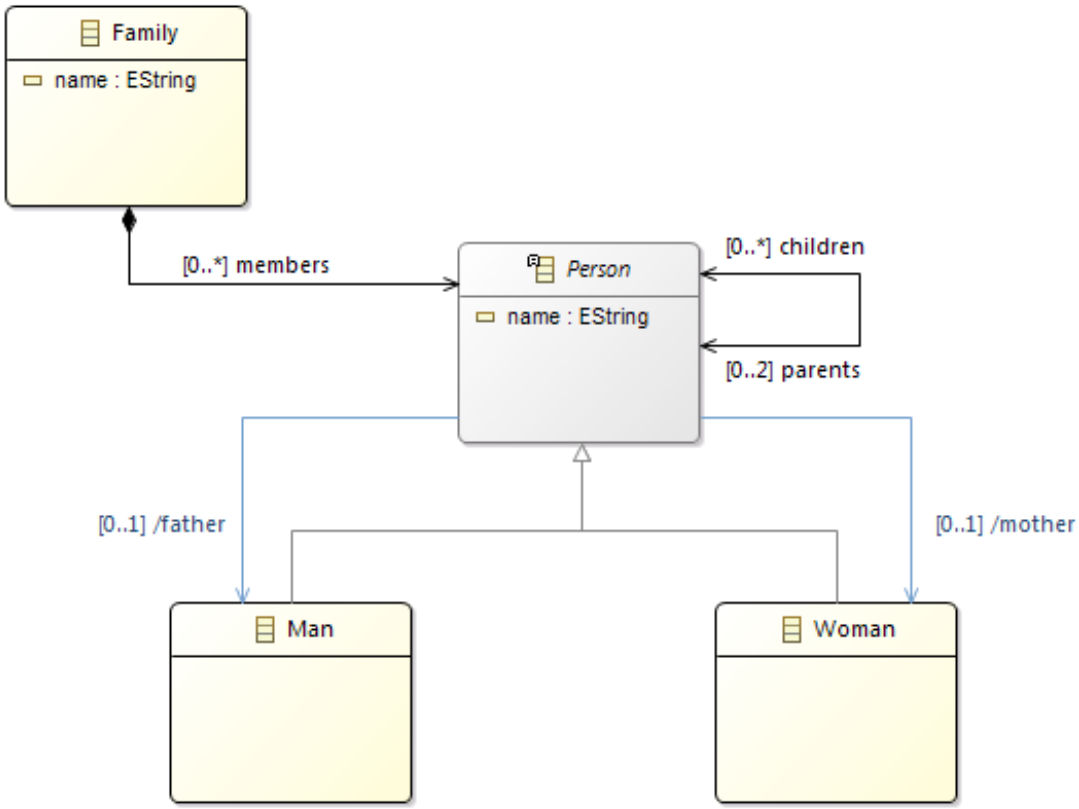
The modeling workbench created with this tutorial will provide diagrams and tables to allow the users to visualize and edit the members of a family and their parental relationships.



A modeling workbench allows users to create or visualize models of a given Domain Model defined with EMF (Ecore model).

This tutorial relies on a Domain Model (DSL) defining the basic concepts of families:

- A family is composed of several persons.
- Persons are men or women.
- A person has:
    - a name,
    - children,
    - two parents
    - a father and a mother.



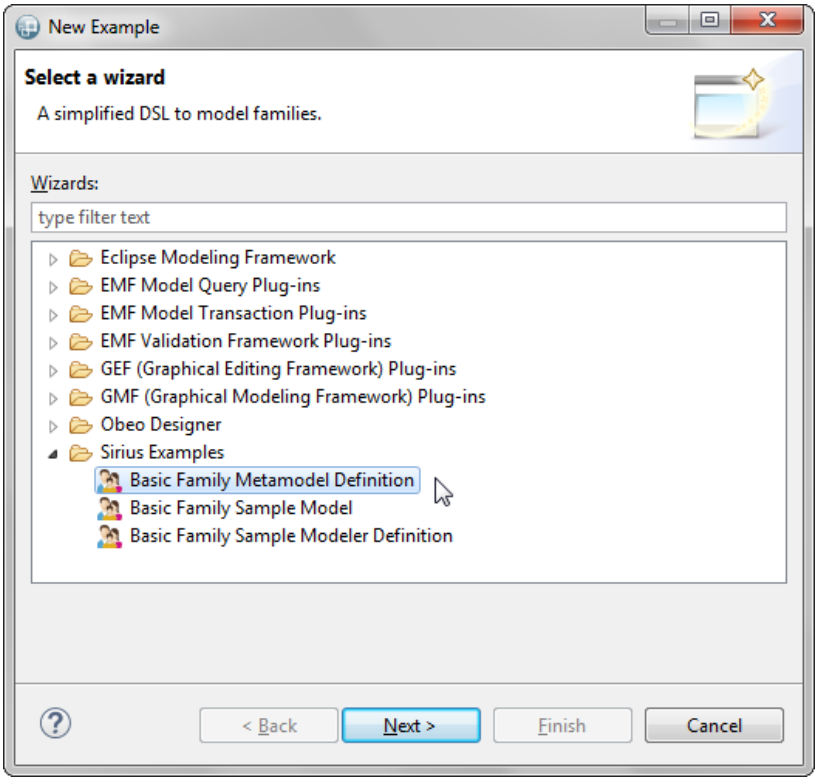
Preparation

Launch Obeo Designer Community 8.1.

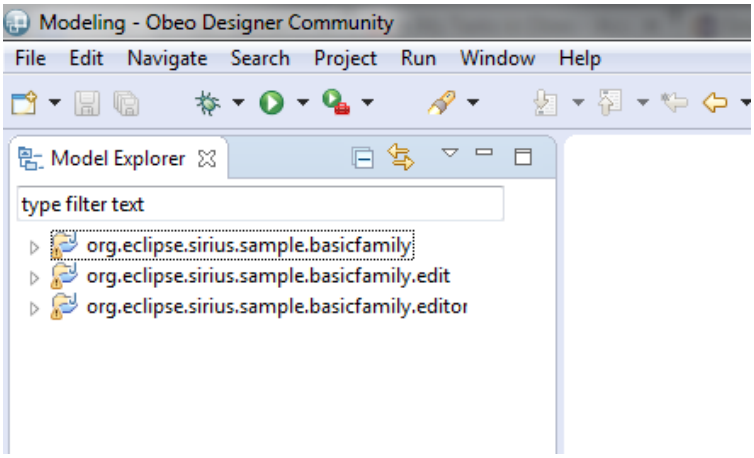
Install the sample Domain Model

The domain model is implemented with several EMF projects that you need to import into your workspace.

These projects can be installed from the provided examples (menu ***File > New > Example...*** : select ***Basic Family Metamodel Definition***).

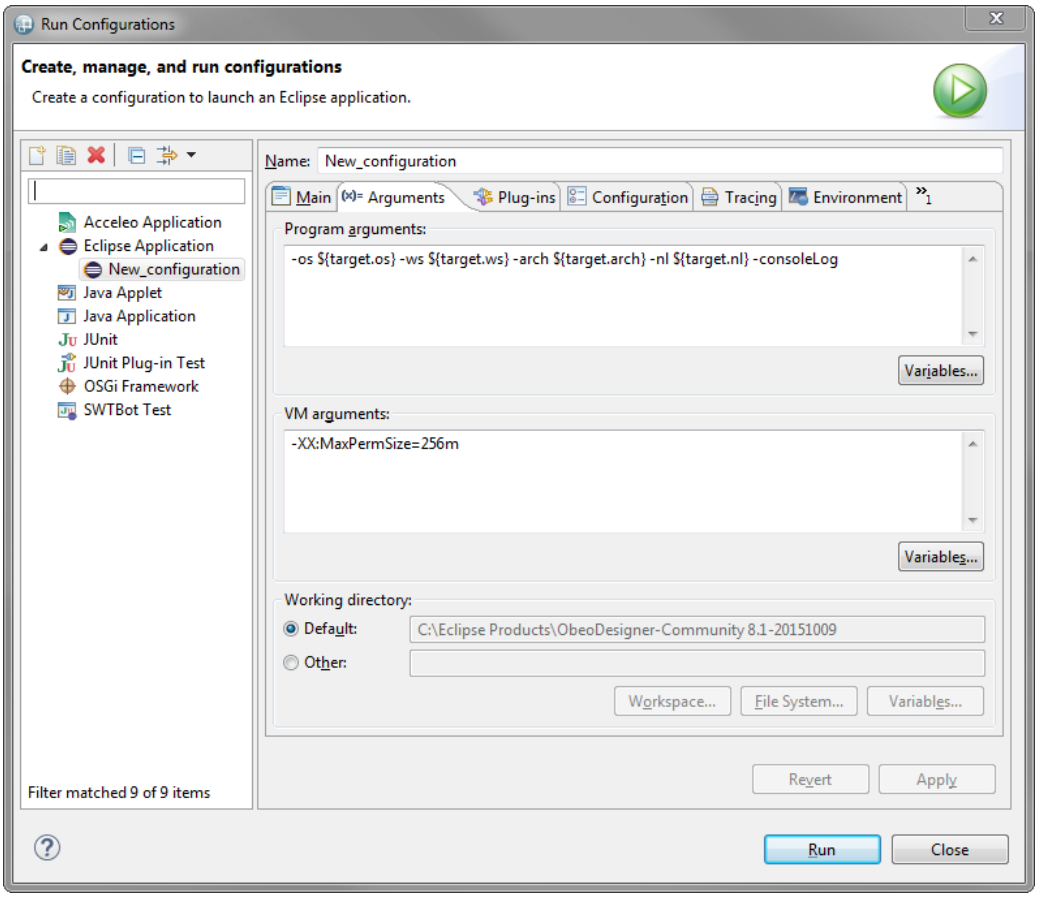


This should create three projects in your workspace:



Launch a new runtime configuration

To launch a new Eclipse application, click on ***Run / Run Configurations*** and double click on ***Eclipse Application*** to get a *New_configuration*.



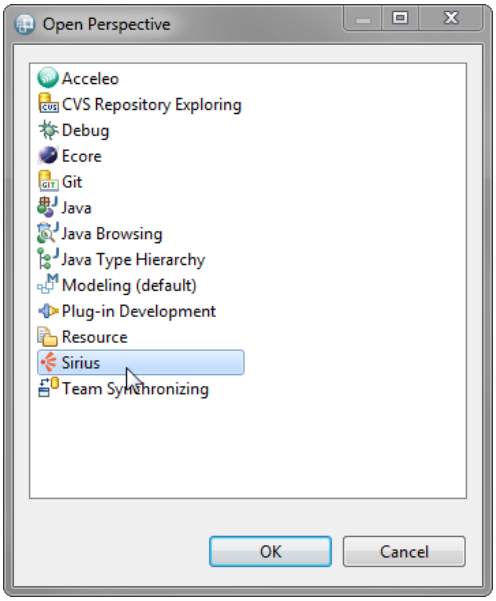
Note: If you are <u>not</u> running with Java 8, you should add this option in your VM arguments :

```
-XX:MaxPermSize=256m
```

Select the Sirius perspective

In the new Eclipse application, open the Sirius perspective (button **Open Perspective**).

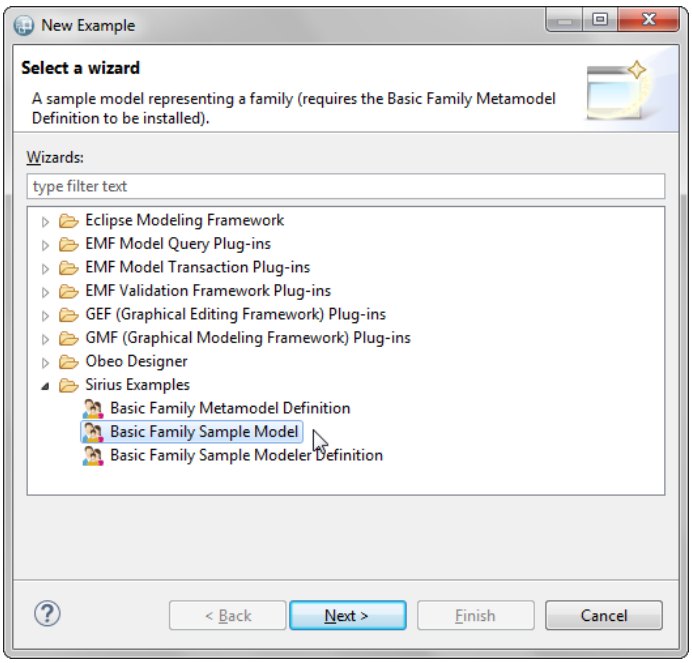
Then select Sirius.



This perspective provides specific Sirius menus and a new kind of project (*Modeling Project*). A Modeling Project contains models and the corresponding graphical representations created with Sirius.

Import a sample model

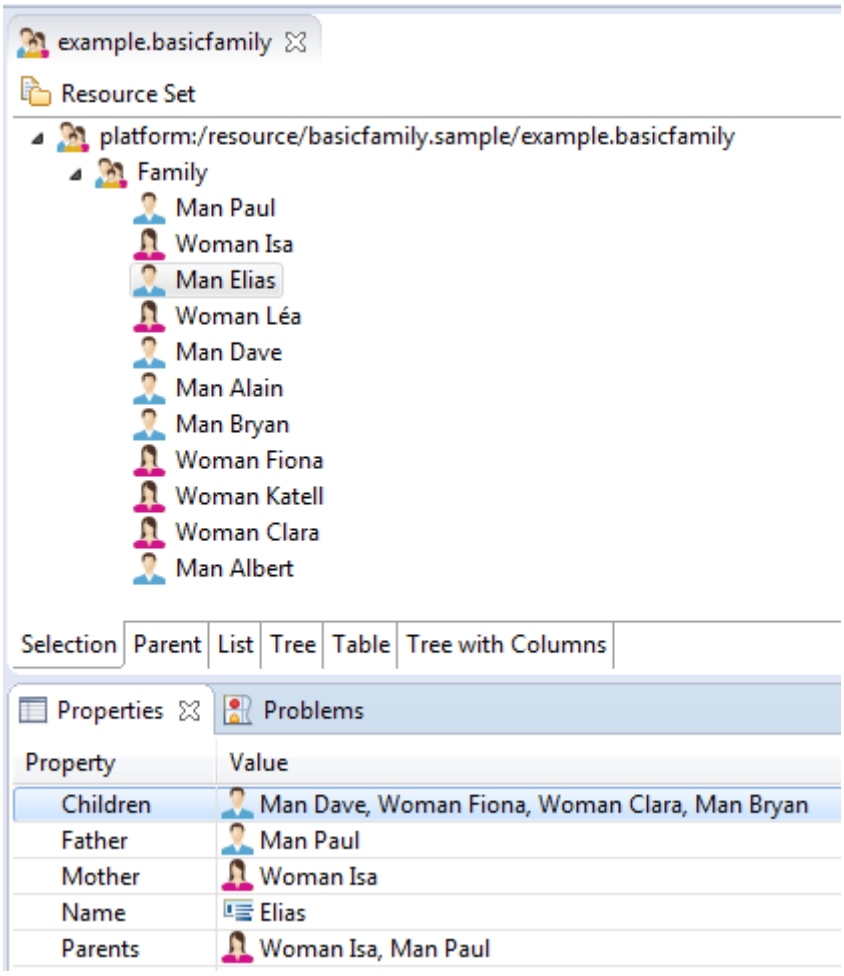
A sample model is available from the provided examples (menu ***File > New > Example...*** : select ***Basic Family Sample Model***).



The wizard then installs a *Modeling Project* containing a file named *example.basicfamily*.

Double-clicking on this sample model opens a tree editor generated by EMF.

This editor allows you to see the properties and relationships of the model elements.



Basic diagram

Let's start creating a first basic diagram with Sirius.

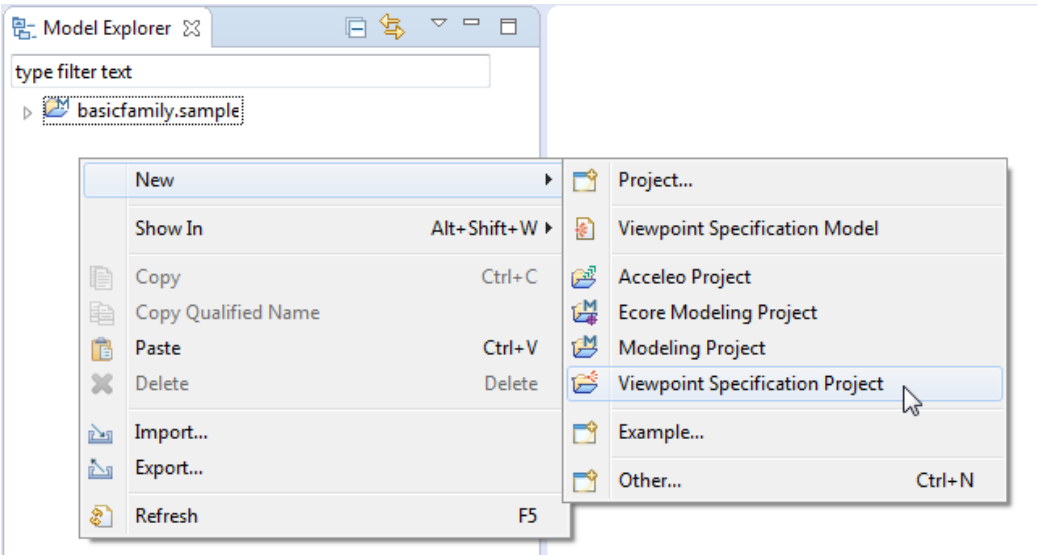
In this tutorial, we will create a diagram editor that allows users to visualize and edit a family with its members and their relationships (father and mother).

The sample model imported previously will be used to test this editor.

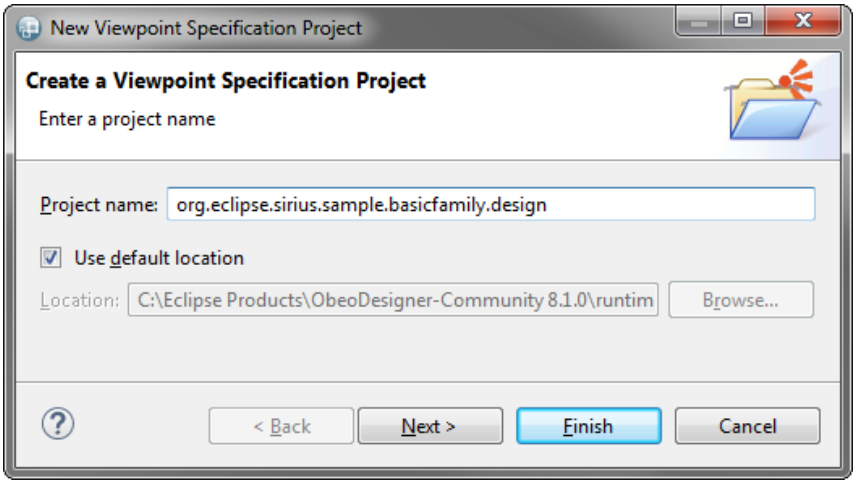
Create a Viewpoint Specification Project

A *Viewpoint Specification Project* will contain the definition of your modeling workbench. It can be created in a development workspace or in a runtime one.

In order to facilitate the test of the designer, we recommend creating this project in the same Eclipse runtime launched previously.

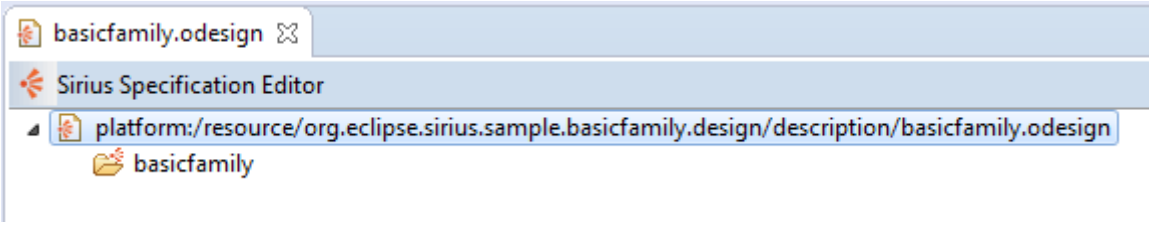


Give a name to this project.



The *Viewpoint Specification Project* creation wizard creates a new project containing a *.odesign* file that describes a modeling workbench which can be interpreted by the Sirius runtime.

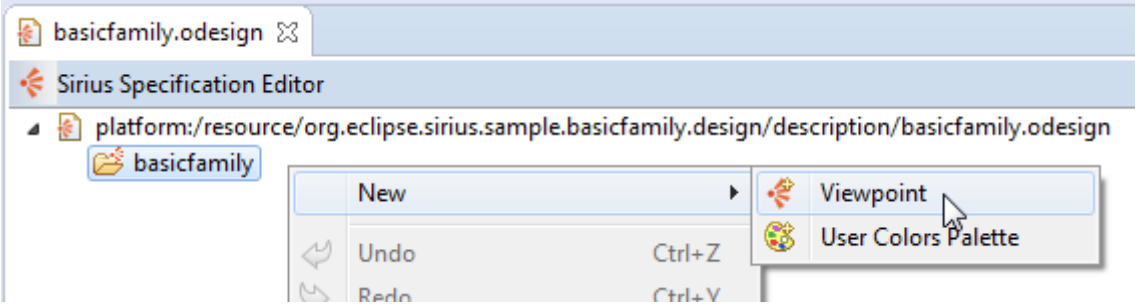
Sirius automatically opens this file with a specific editor.



Define a Viewpoint

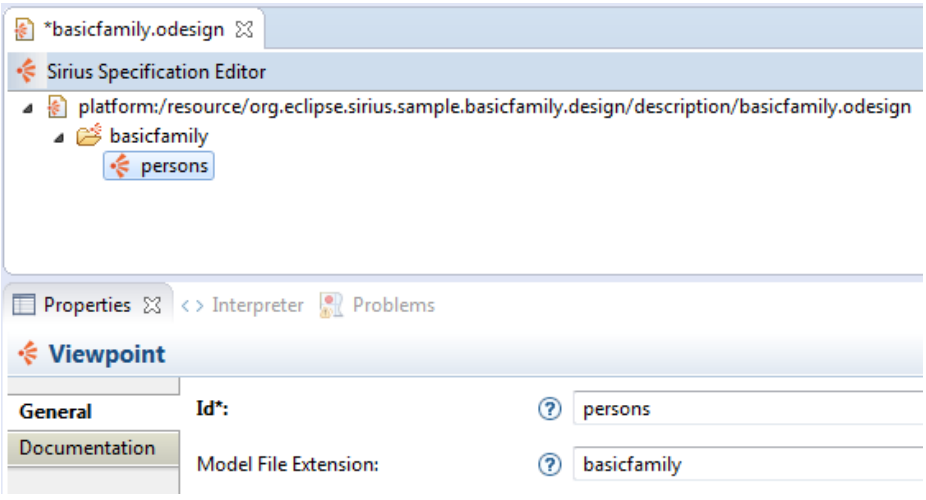
A *.odesign* file contains *viewpoints*. Each *viewpoint* defines a set of *representations* (diagrams, tables or trees).

That's why you need to define at least one viewpoint in the *.odesign* file (menu ***New... / Viewpoint***).



Note: Don't forget to specify the *Id* of this viewpoint.

The *Model File Extension* property defines the kind of models associated to the designer. We encourage you to set the extension associated to your domain model.

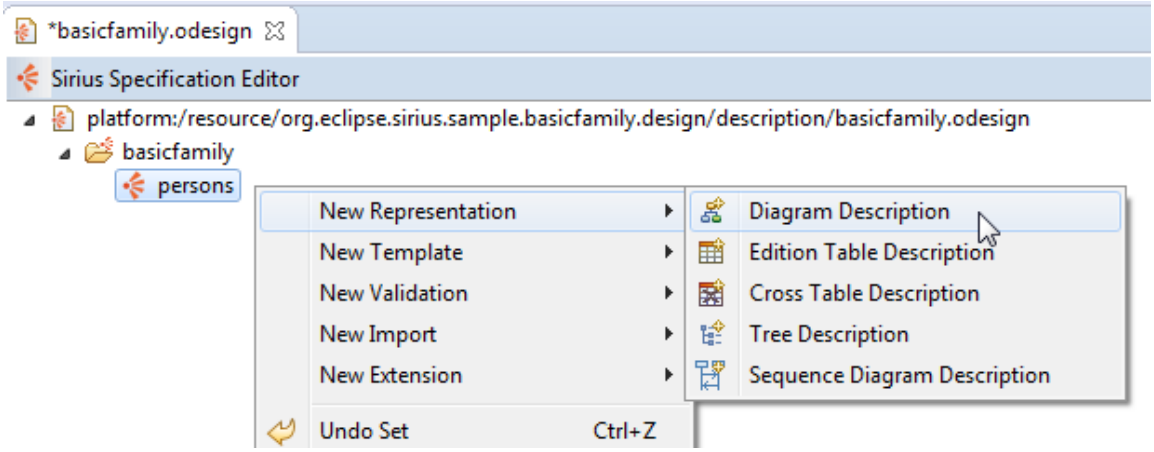


Properties of a Viewpoint

- **Id**: The identifier of this element. Must be unique. Changing this identifier will break existing user models which reference the old identifier.
- **Label**: The label used to display this to the end-user.
- **Model File Extension**: This field allows to associate this viewpoint to one or more semantic resource file extension(s), for several file extensions, they must be space separeted. For example to associate this viewpoint to uml and ecore metamodels, put «uml ecore».

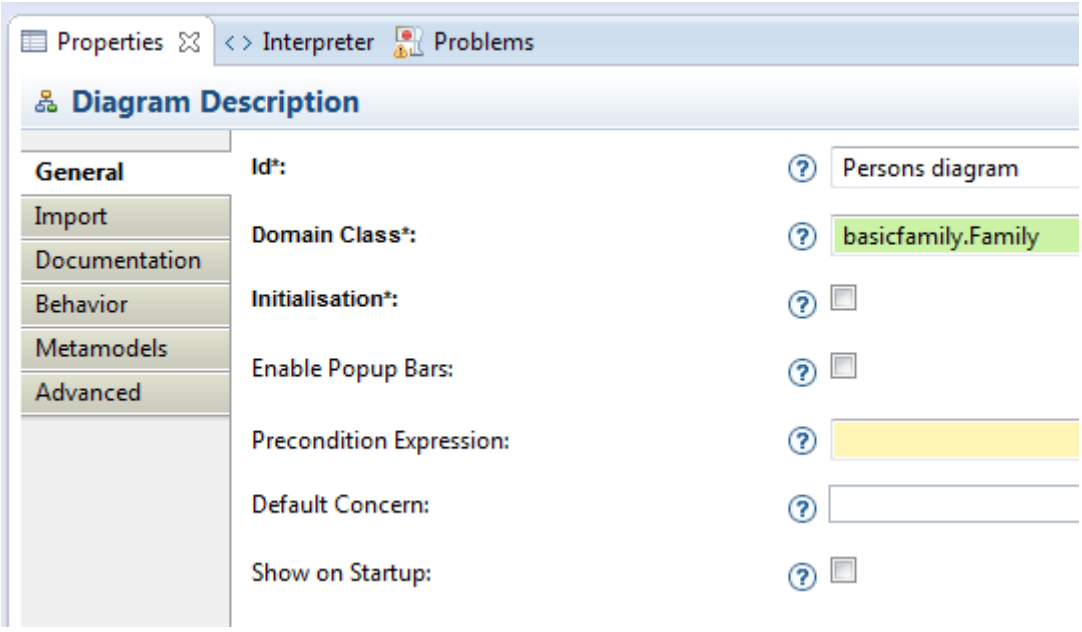
Define a Diagram

Add a first Diagram to your Viewpoint (menu **New Representation... / Diagram Description**).



A diagram must be associated to a Domain Class: the diagram will graphically represent instances of this Domain Class.

At this step, create *Persons Diagram* which will display the persons of a family.

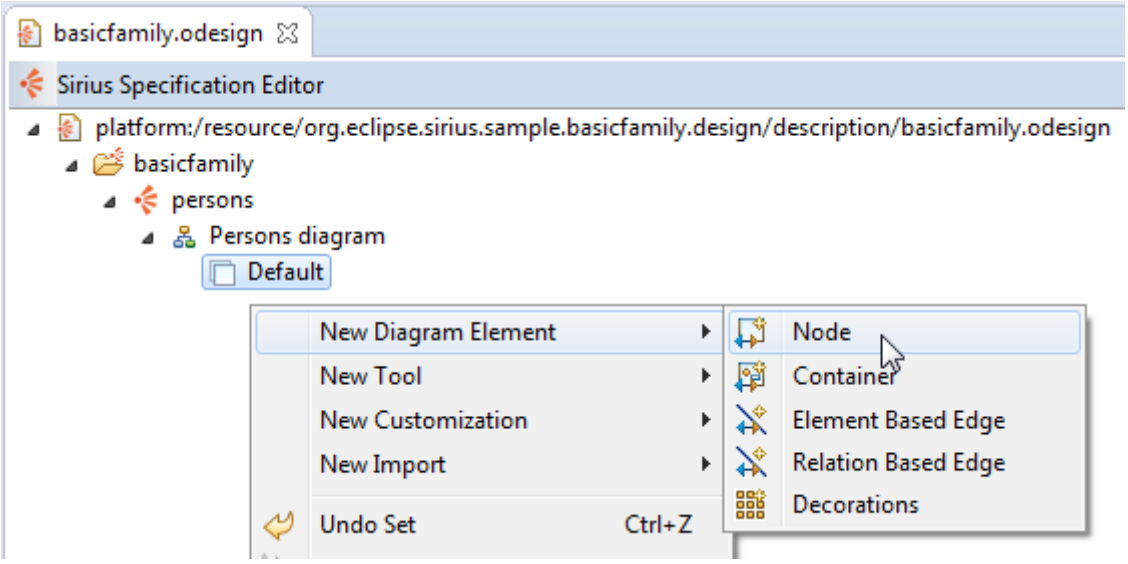


Properties of a Diagram

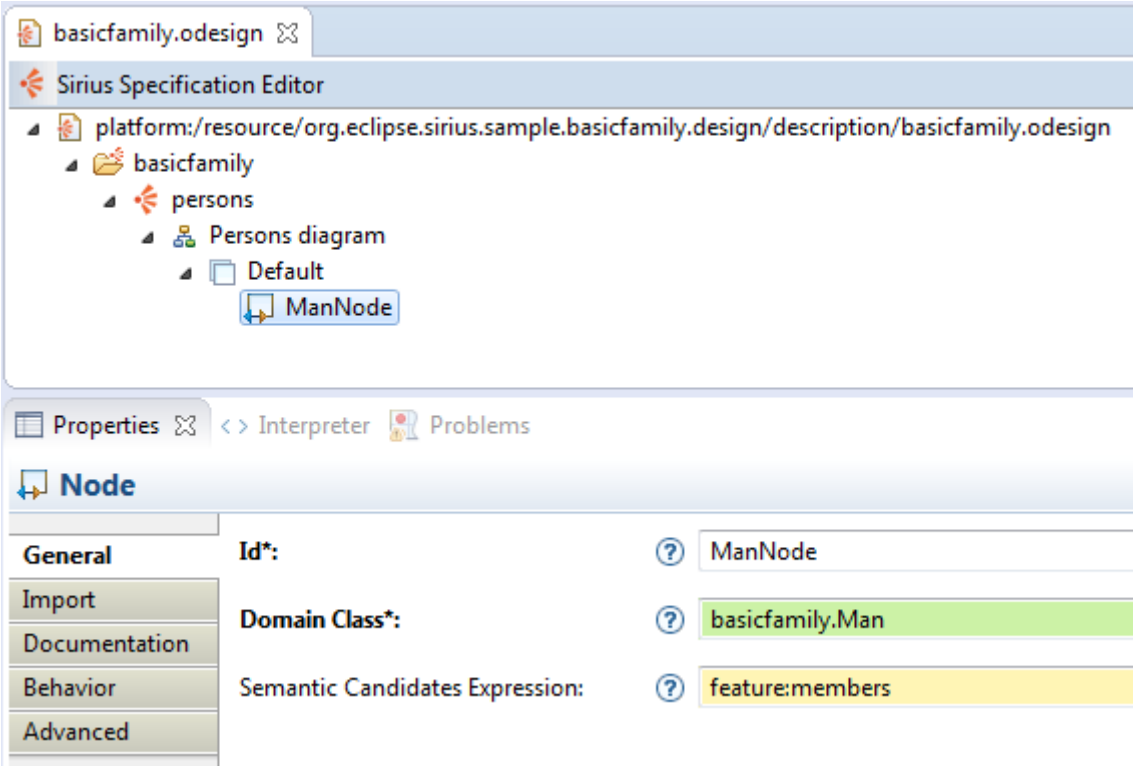
- **Id**: The identifier of this element. Must be unique. Changing this identifier will break existing user models which reference the old identifier.
- **Label**: The label used to display this to the end-user.
- **Domain Class**: The type of the root diagram element.
- **Initialisation**: If set to true, the representation will be automatically created when creating a new session.
- **Enable Popup Bars**: If true, creation tools which are visible in the palette will also be available as pop-up toolbars in the diagram itself.
- **Precondition Expression**: The precondition is an expression preventing the creation of a diagram. If the precondition is set and the expression returns false on the root diagram element, then the diagram won't be created.
- **Show on Startup**: If true, existing representations of this type will be automatically opened when a session is opened.

Add a Node to the Layer

A *Diagram* shows *Nodes* which are elements of the model. To create a new *Node* to the diagram, right-click on the *Layer* and select the menu ***New Diagram Element... / Node***.



All created nodes must have an *Id* and must be associated to a *Domain class*.

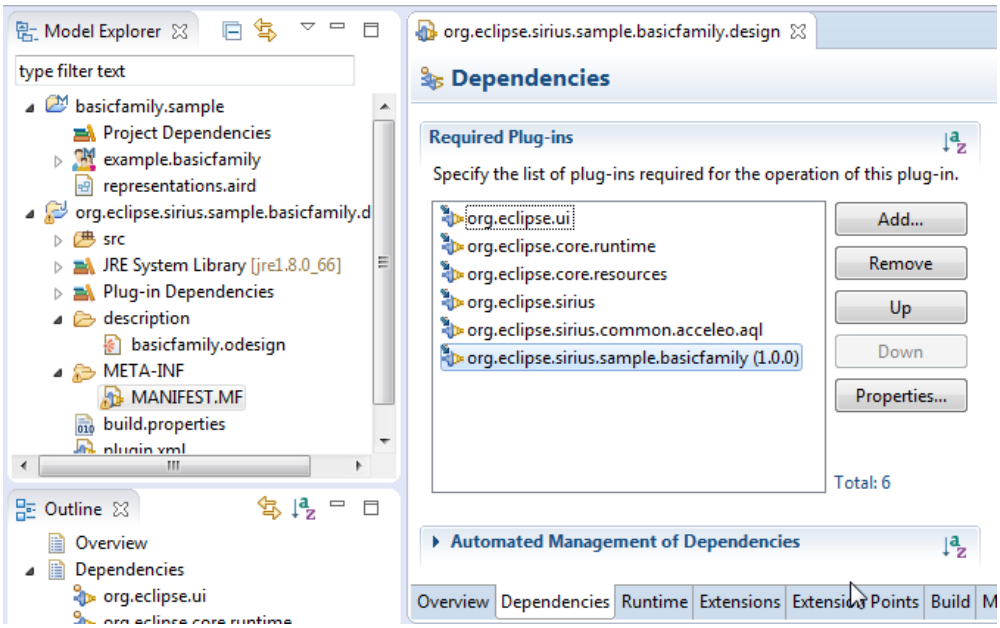


Properties of a Node

- **Id**: The identifier of this element. Must be unique. Changing this identifier will break existing user models which reference the old identifier.
- **Label**: The label used to display this to the end-user.
- **Domain Class**: The type of the root diagram element.
- **Semantic Candidates Expression**: Restrict the list of elements to consider before creating the graphical elements. If you set this attribute then only the elements returned by the expression evaluation will be considered.

The *Semantic Candidates Expression* can be an AQL (Acceleo Query Language) expression prefixed by *aql:* . This expression is evaluated on the object associated to the diagram (here: an instance of Family).

Note: To improve the performance of your modeling tool, we encourage you to declare your metamodel in your project. Just open the *MANIFEST.MF* file and add the plug-in that defines your metamodel in the list of the *Required Plug-ins* (in the *Dependencies* tab). It is **required** if your AQL expressions directly refer types of the metamodel (ex: `basicfamily::Man`).



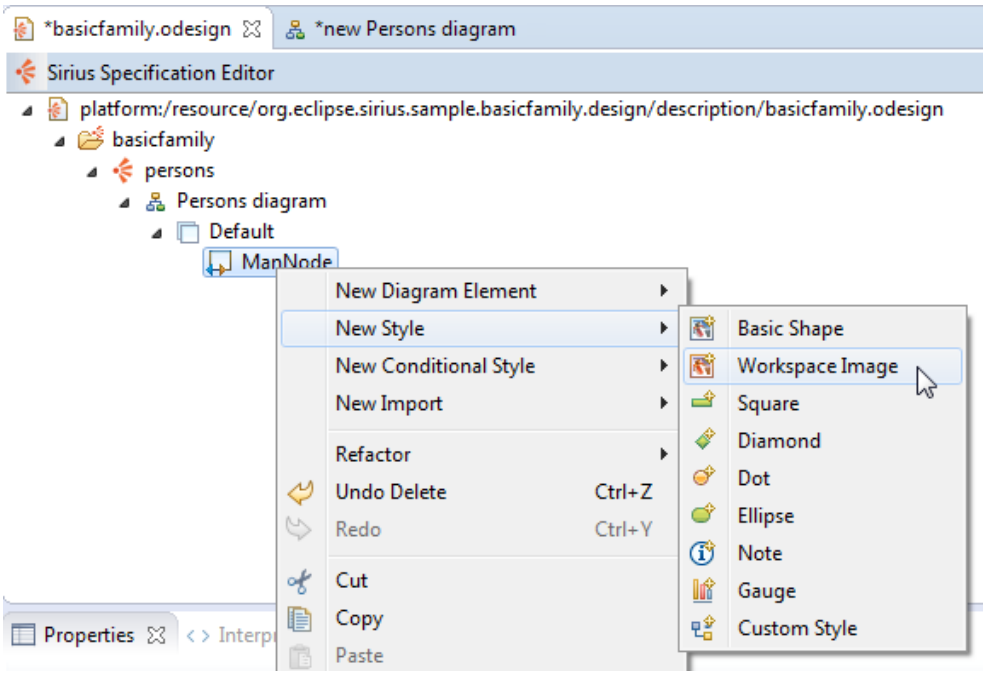
You can also use three interpreters dedicated to very simple expressions, by using specific tags (*feature:* , *service:* and *var:*) followed by the name of the feature, service or variable.

Auto-completion is available on an empty *Semantic Candidates Expression* property by placing the cursor in the empty field and hitting *Ctrl+Space*.

Set a Style to the Node

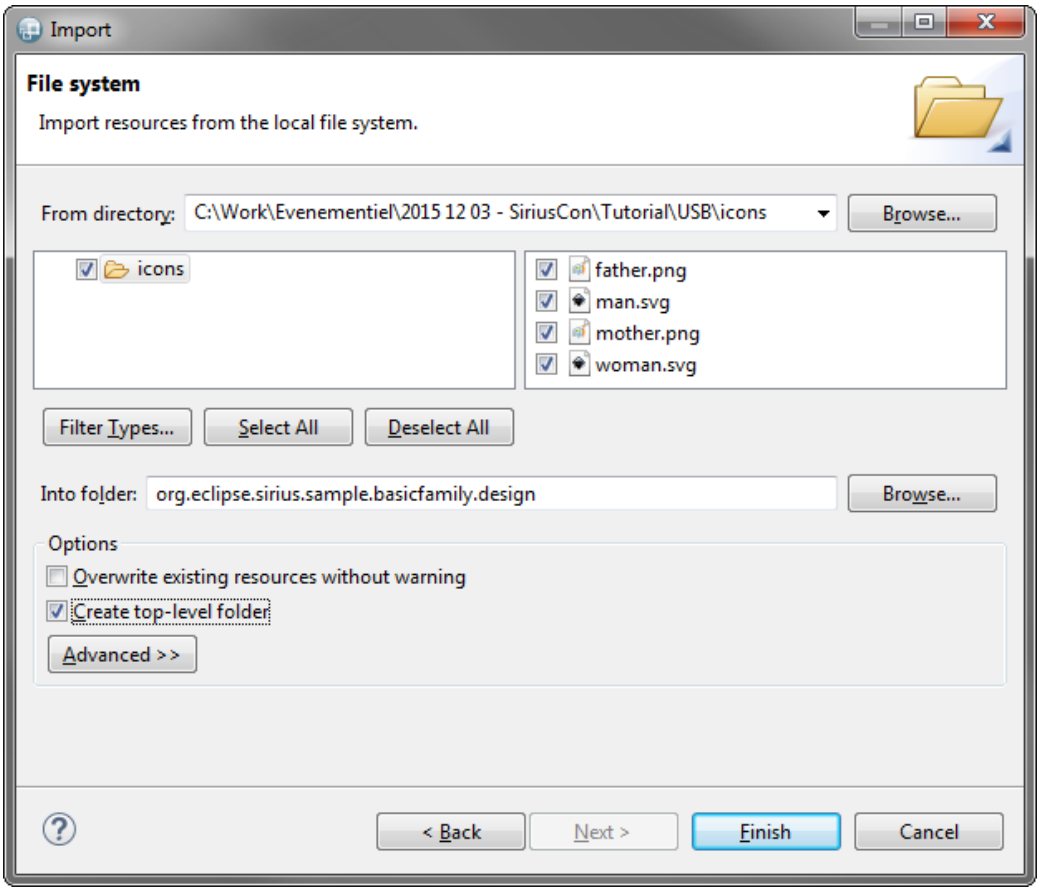
To define the way the node is graphically represented on the diagram, it must declare a *Style* (menu ***New Style***).

Here, we have chosen a *Workspace Image*.



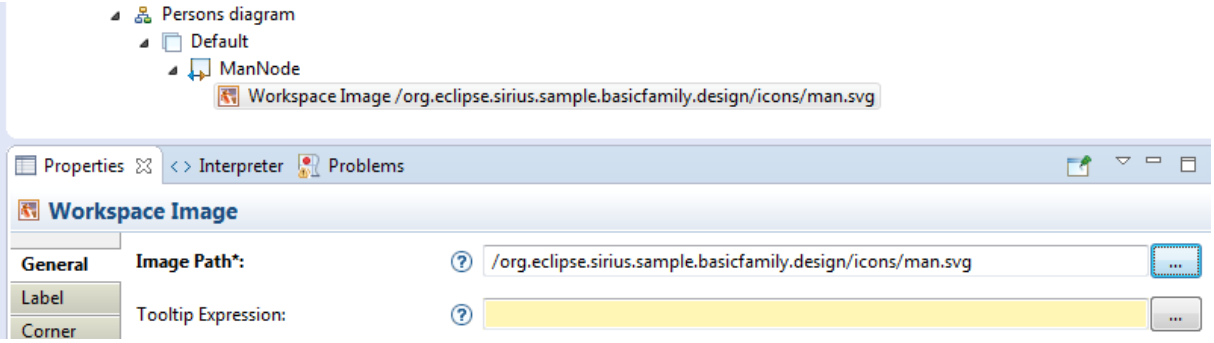
Specify the *Workspace Path* to define the image (the images can be placed in a specific folder of your project).

First, import the icons folder provided with the tutorial instructions.

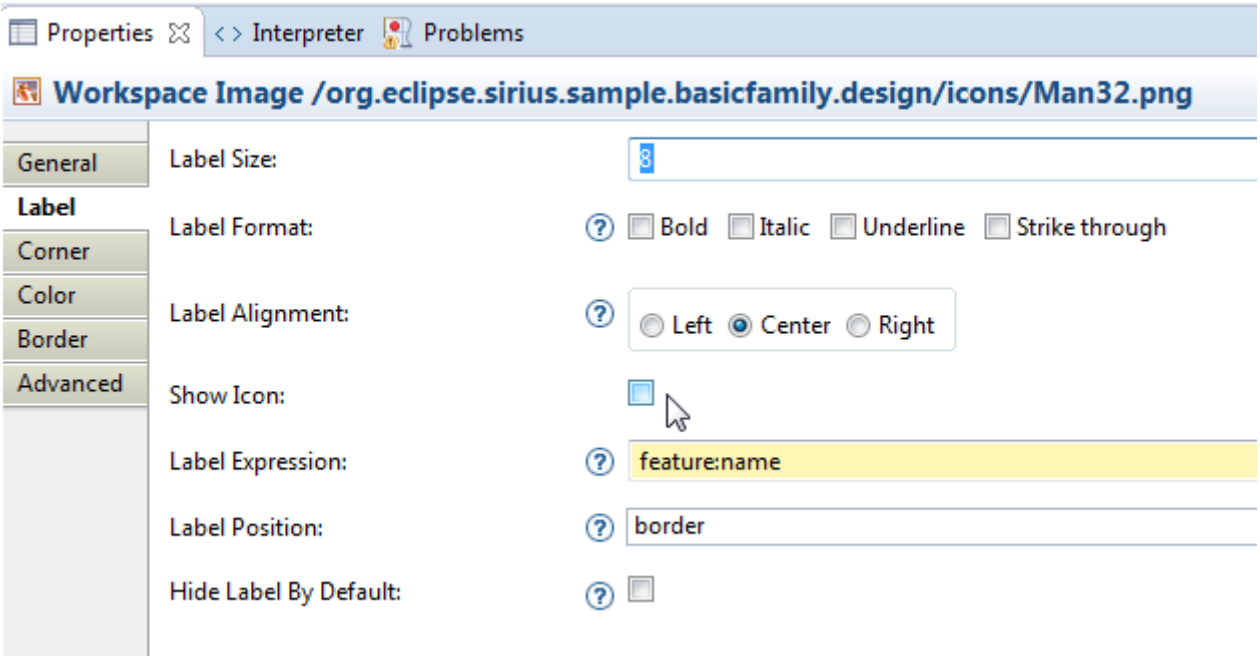


Then, set the properties of the Workspace Image:

- **Workspace Path**: Path for the image in the form of */myProjectID/path/to/image.png*. If the image is not found in the workspace the tooling will look for it in the plugins.
- **Tooltip Expression**: An optional expression which should return the text of the element's tool-tip.



In the *Label* tab, deselect the *Show icon* option to avoid displaying the 16x16 icon with the SVG image, and select the "border" option for the *Label Position*.



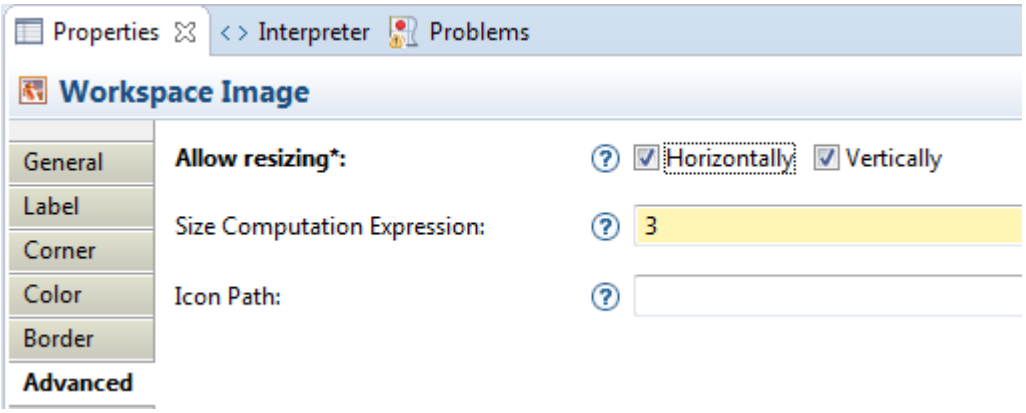
The *Style* also defines the label of the *Node*. By default the label is calculated with the expression `feature:name`

By using AQL syntax you can enter your own expression. For example, you could display the name to upper case by specifying the Label Expression to `aql:self.name.toUpper()`

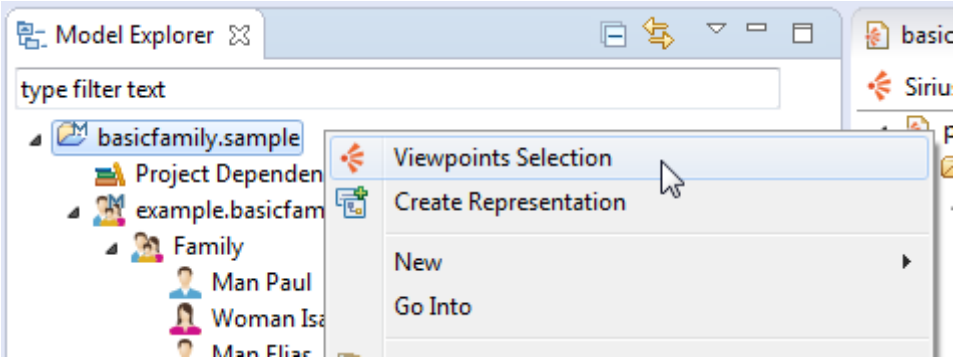
Properties of a Node Style

- **Label Format**: The font formatting style to use for the label
- **Label Position**: Pick either ‹node› position which mean ‹inside the node› or ‹border› to put the label around the node.
- **Hide Label By Default**: Define the default visibility status of this label (available only if label position is «border»).
- **Label Alignment**: The alignment of the label.
- **Label Expression**:
    - Expected return type: a string.
    - Available variables:
        - **diagram**: viewpoint.DDiagram | the current DSemanticDiagram.
        - **view**: viewpoint.DDiagramElement | the current view for which the label is calculated.

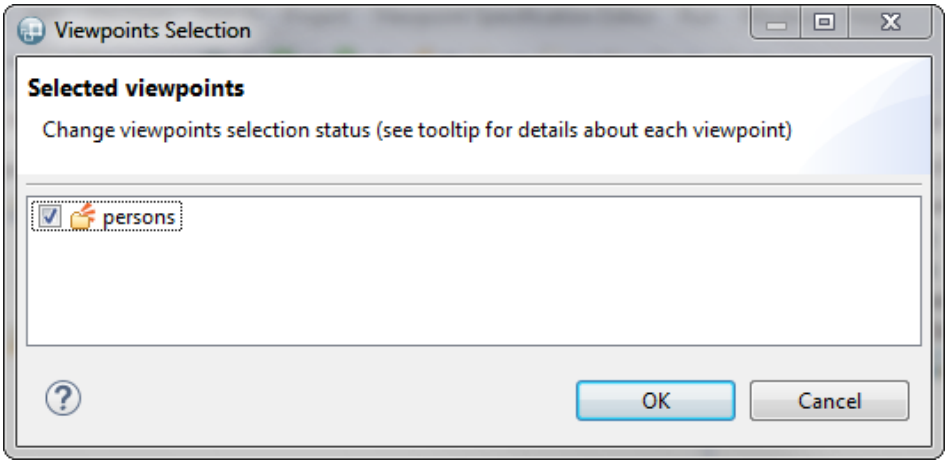
As the provided SVG image is big, you have to specify a reasonable size with the *Size Computation Expression* property (set it to "3").



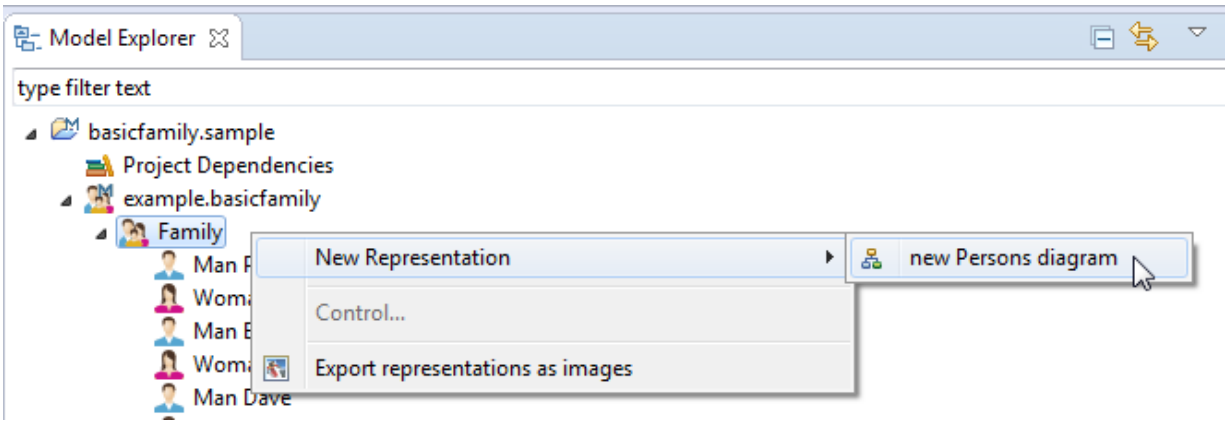
To test this first diagram, right-click in the *Model Explorer* view on the Modeling project and select the menu ***Viewpoint Selection***.



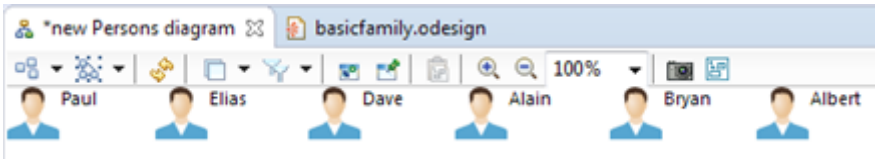
You should see the new viewpoint *persons*. You must activate this viewpoint to be able to create the representations which are defined by this viewpoint.



Then right-click on the sample model and select the menu ***New Representation / new Persons diagram***.

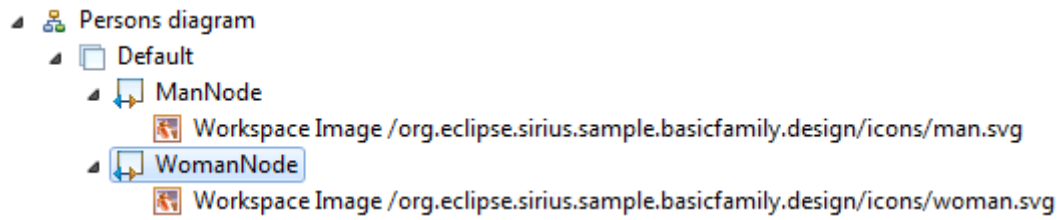


When the diagram opens it displays all the instances of *Man* contained in the sample model.

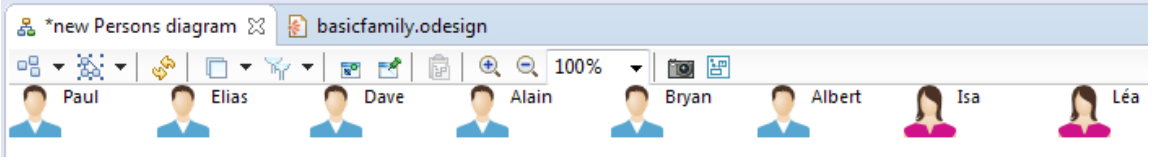


Add a second Node

Proceed the same way to create nodes for instances of *Woman*. Choose *woman.svg* image for this node.

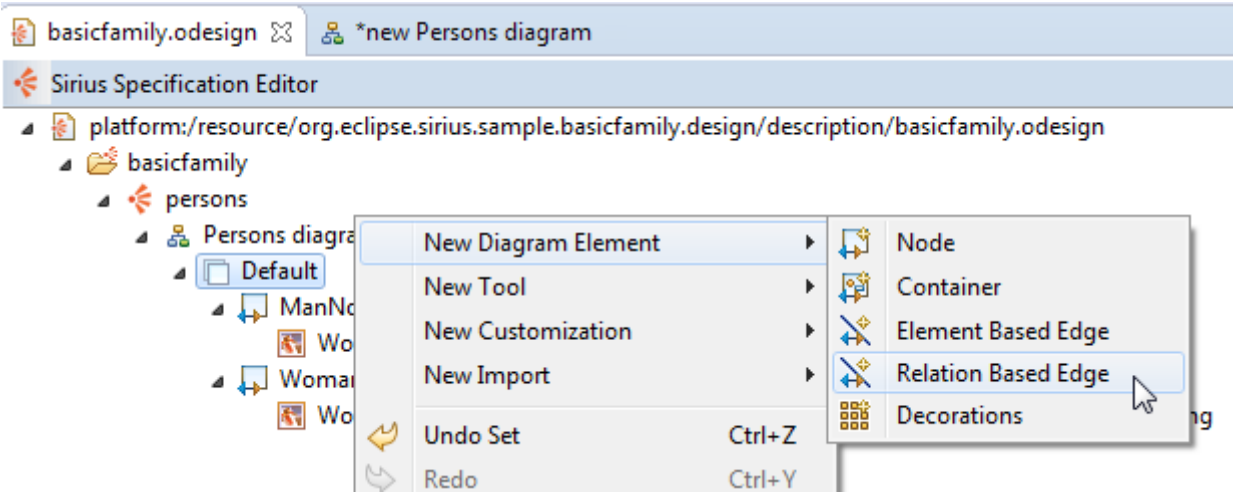


Saving the *.odesign* file immediately updates the current diagram to reflect the changes.



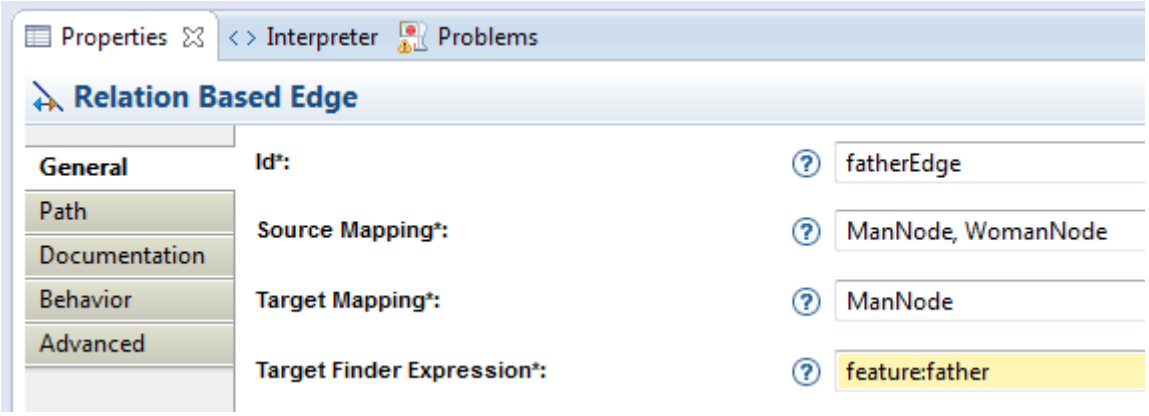
Add a Relation Based Edge

To display relations between *Nodes*, create a *Relation Based Edge*.



Enter the *Source* and *Target Mappings* of the relation: Sirius will select each instance of the concerned nodes which are visible on the current diagram.

Then, Sirius will evaluate the *Target Finder Expression* on each *Source Mapping* to detect the corresponding *Target Mapping*.



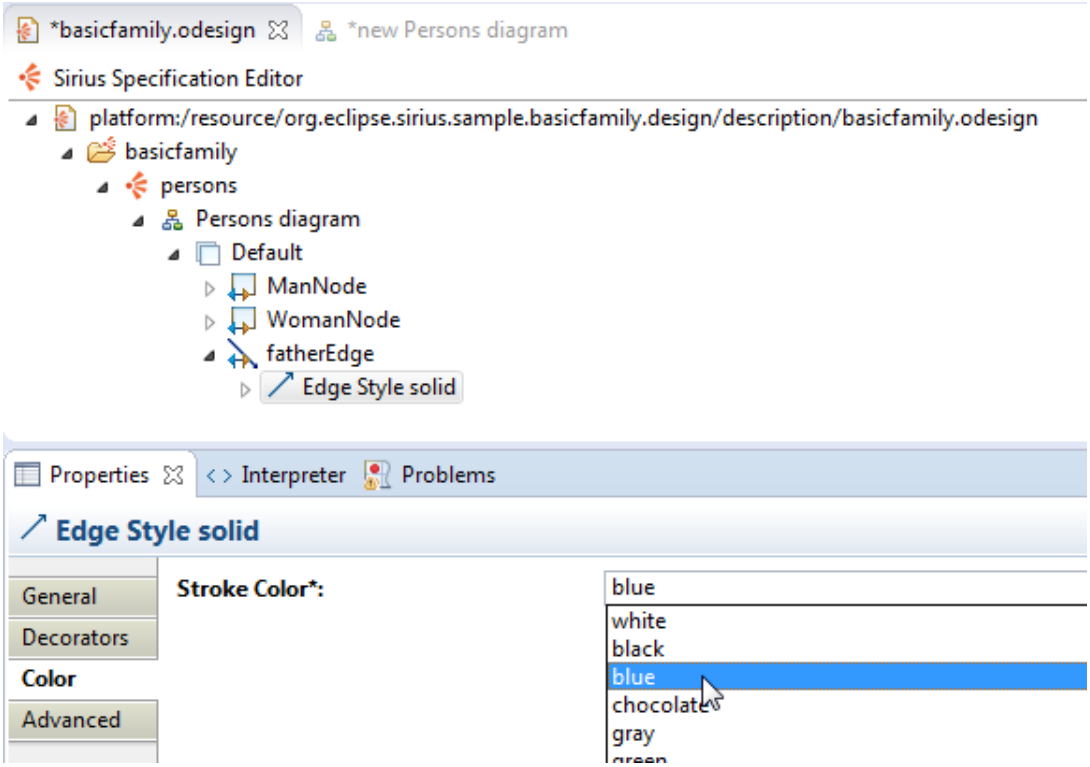
Properties of a Relation Based Edge

- **Id**: The identifier of this element. Must be unique. Changing this identifier will break existing user models which reference the old identifier.
- **Source Mapping**: Mapping from which the edge should start.
- **Target Mapping**: Mapping from which the edge should end.

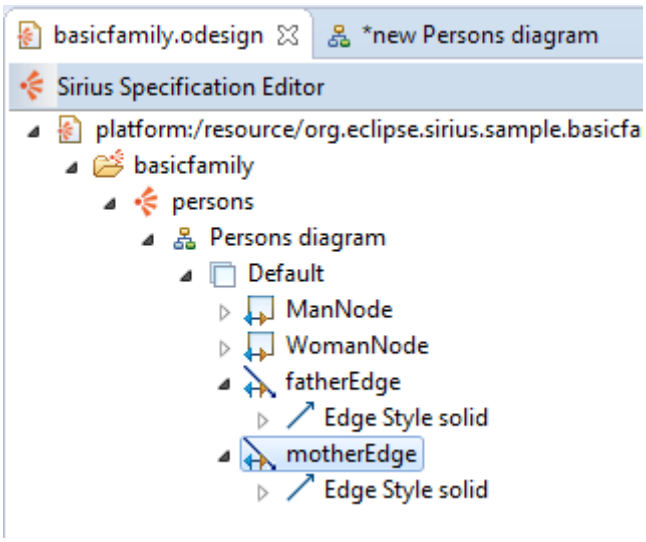
- **Target Finder Expression**: Expression which should return, starting from the target element (which may be the domain class if the edge is domain based, otherwise it's the target of the source node) the target semantic element.

Edit the Style of the Relation

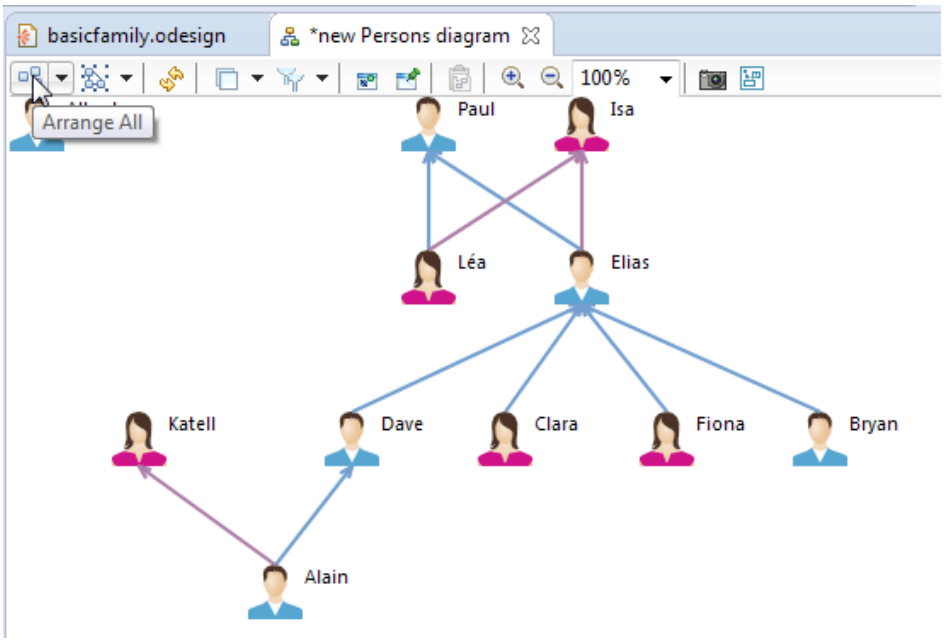
Edges are initialized with a default style which defines its graphical appearance (color, line style, etc.). You can edit this style to display *father* relations in blue.



Then create a second *Edge* to display *mother* relations in light purple.

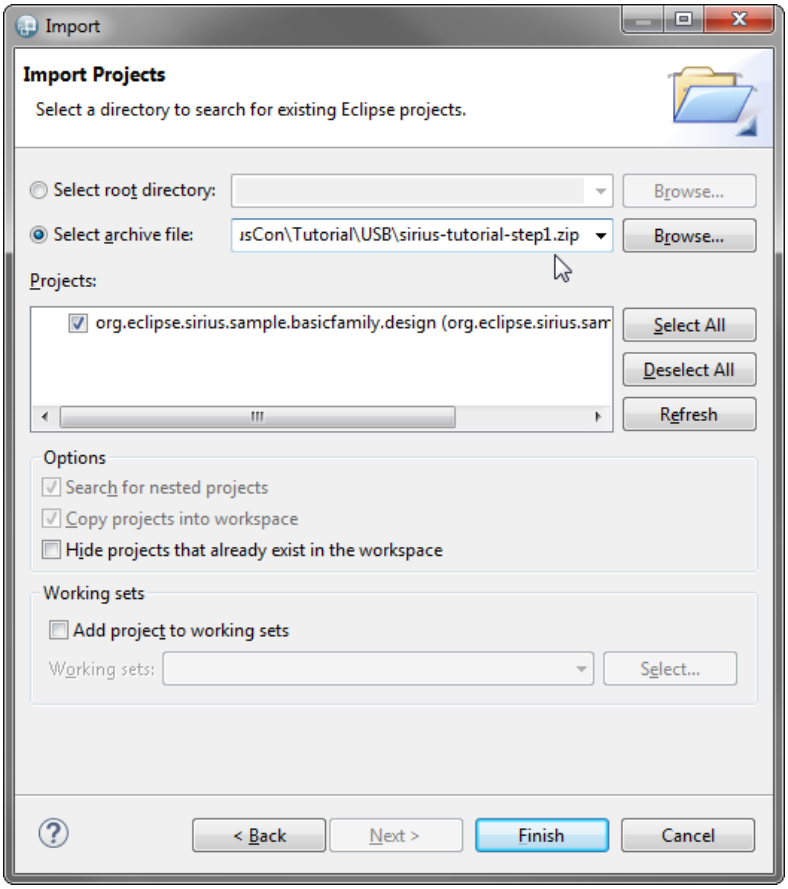


Save the *.odesign* file, refresh the diagram and click on the *Arrange All* button to see the changes.



The solution of is this step is available in **sirius-tutorial-step1.zip**

To install this solution, first delete your project, then import the solution (
Menu **File > Import > Existing Project into Workspace**) and select the archive file.



Edition tools

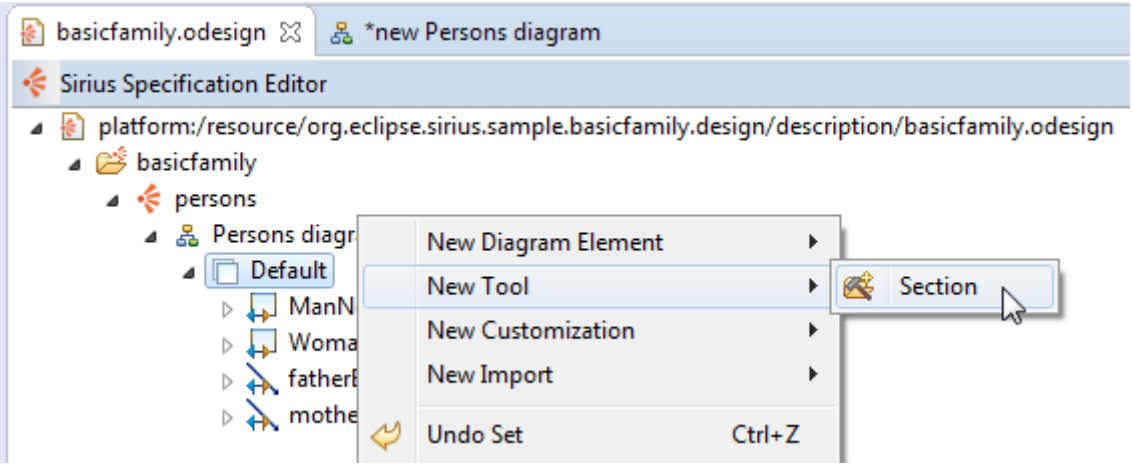
Add a Palette for edition tools

In the previous steps, we have created a modeling workbench which can display an existing model.

Let's complete this designer with a palette containing tools to allow users to create new model elements.

Create a Section

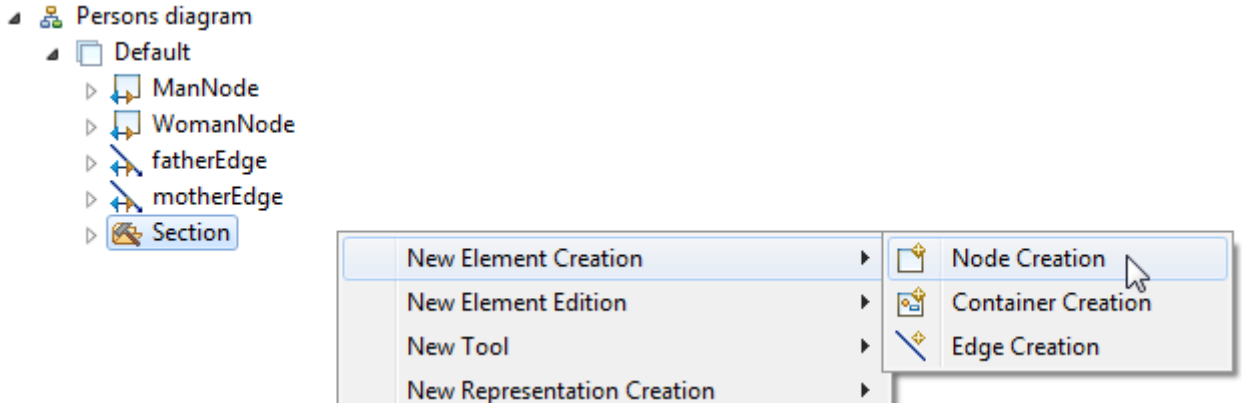
Add a new Section to the Layer (menu **New tool... / Section**).



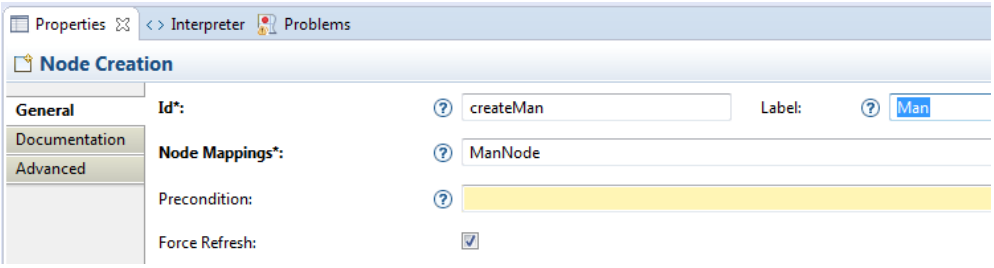
Add a Node Creation

The palette is composed of tools which will allow the user to create new objects.

To create new instances of Domain Classes, add a *Node Creation* element to the *Section* (menu ***New Element Creation... / Node Creation Description***).



To define which kind of nodes will be created by the tool, the *Node Creation Description* element must be associated to an existing node (e.g. *ManNode*).

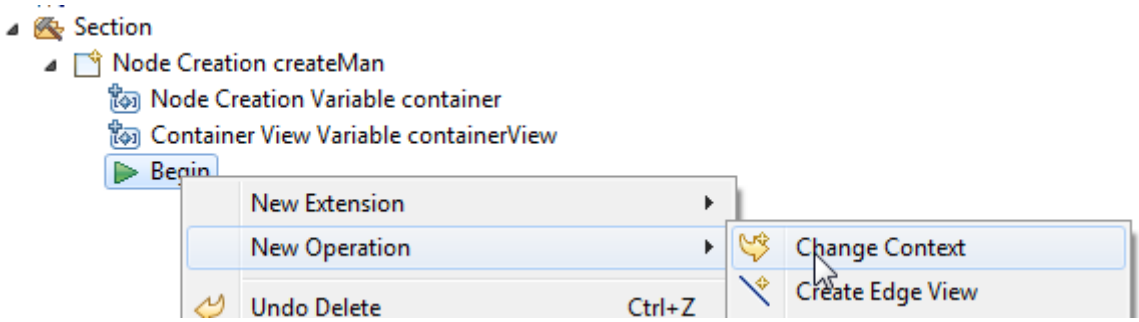


Properties of a Node Creation

- **Id**: The identifier of this element. Must be unique. Changing this identifier will break existing user models which reference the old identifier.
- **Label**: The label used to display this to the end-user.
- **Node Mappings**: Node mappings you may need to create once the tool has been executed.
- Precondition:
    - Expected return type: a boolean.
    - Available variables:
        - container: ecore.EObject | the container.

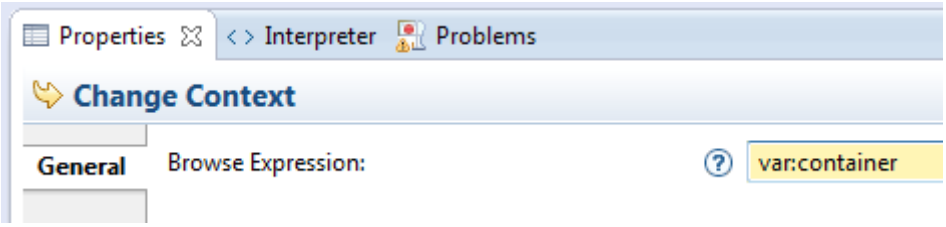
When the user will use the tool, actions will be executed. You have to define them.

The first step consists in adding a *Change Context* element to define the context of the actions.



The *Change Context* element contains an expression which allows Sirius to find the model element which will be the context.

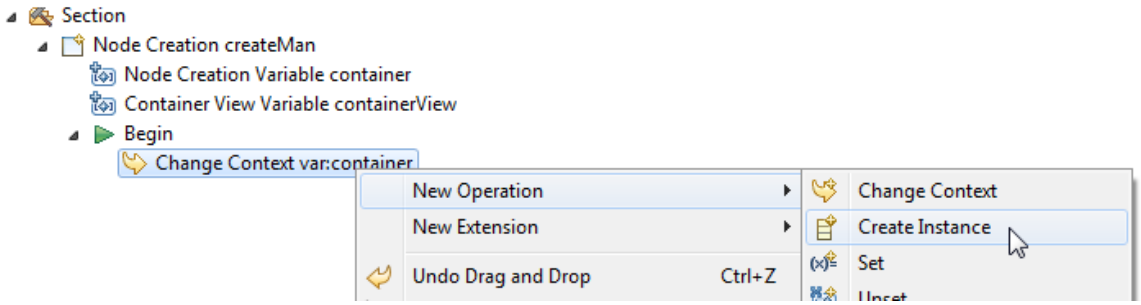
To write the expression, you can use the variable container which references the object holding the current diagram (e.g. a Family instance).



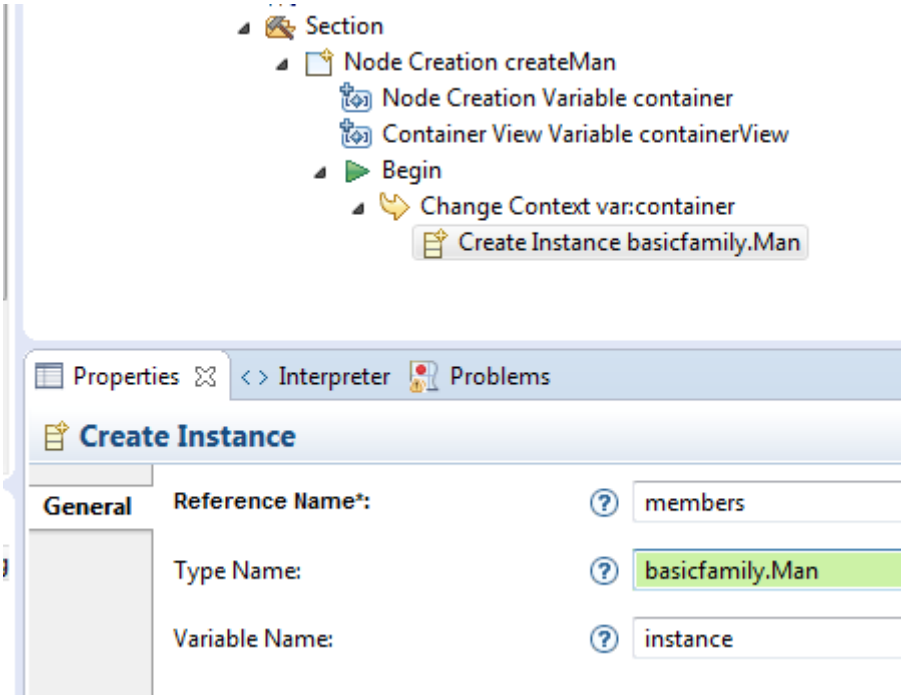
Properties of a Change Context

- **Browse Expression** : Expression to navigate in the model to a new element.

Then, add a *Create Instance* which will create a new instance of *Man*.



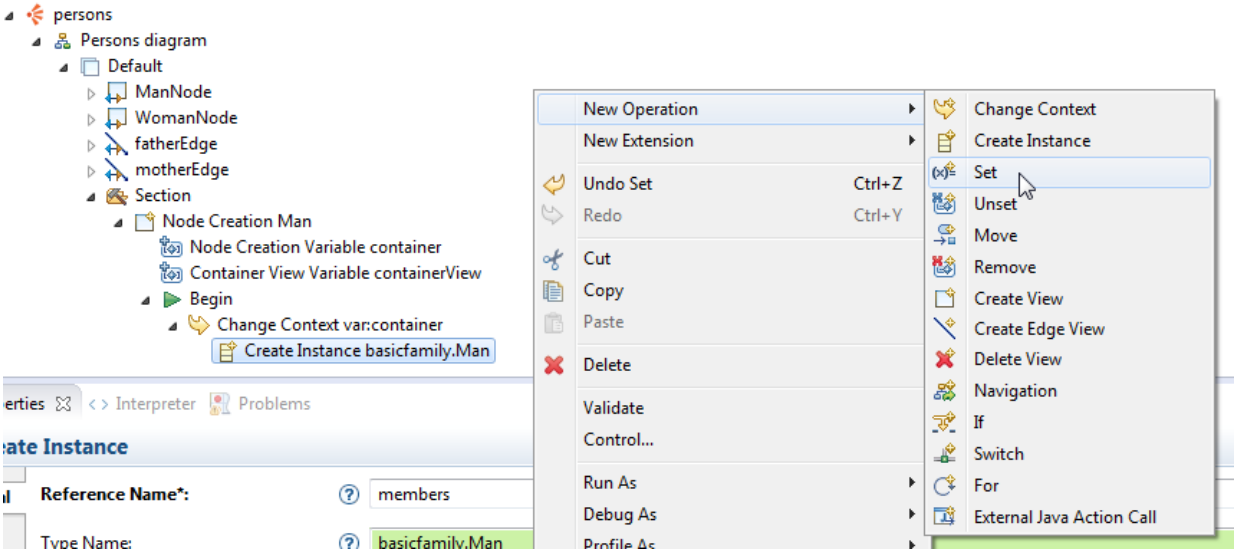
Specify the type of the new instance and how it will be attached to the context.



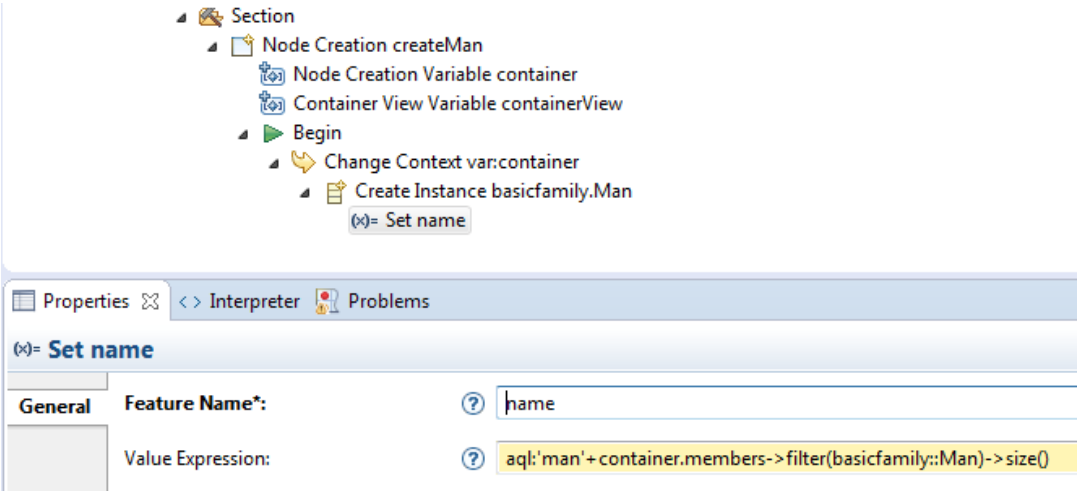
Properties of a Create Instance

- **Reference Name** : Reference name in which the new instance will be stored. This reference should be part of the container element.
- **Type Name**: Type name of the instance to create.
- **Variable Name** : Once the instance is created, a new variable with the given name will exist binding to the newly created object.

To specify the name of the created element, you can also add a *Set Value* operation.



For example, the following AQL expression will compute a default name depending on the number of men already created. `aql:'man'+container.members->filter(basicfamily::Man)->size()`



Properties of a Set Value

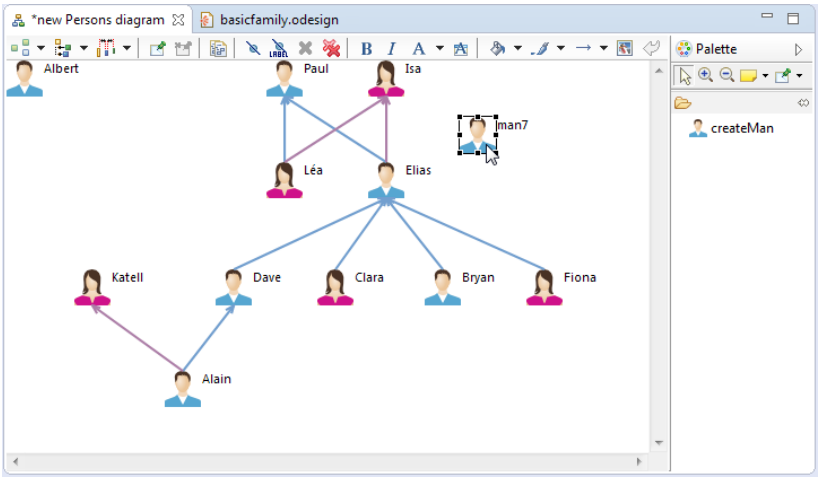
- **Feature Name**: Name of the feature to set.
- **Value Expression**: Expression returning the value to set on the current element.

When you go back to the diagram which displays your sample model, you can notice that the palette contains a section and a *createMan* tool

You can click on this tool and move the cursor to the diagram.

The cursor will automatically change to indicate that you can click on the diagram to create a new object.

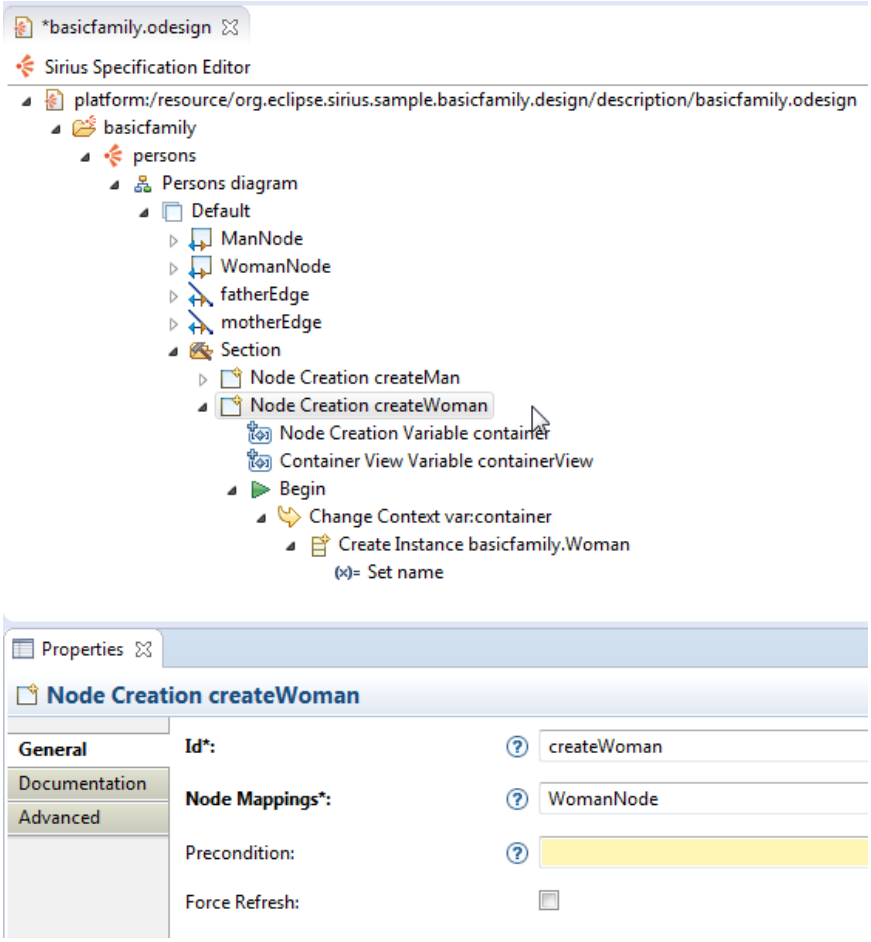
Once you have clicked on the diagram, a new *Man* is created and displayed on the diagram.



Its properties can be edited using the Properties view.

Copy and paste the tool *CreateMan*, then replace Man by *Woman*:

- in the **Id** of the tool
- in the **Node Mappings** of the tool (use the wizard to select *WomanNode*)
- in the **Create Instance** operation (under the *Begin* section)
- in the **Value Expression** of the *Set name* (under the *Create instance*)

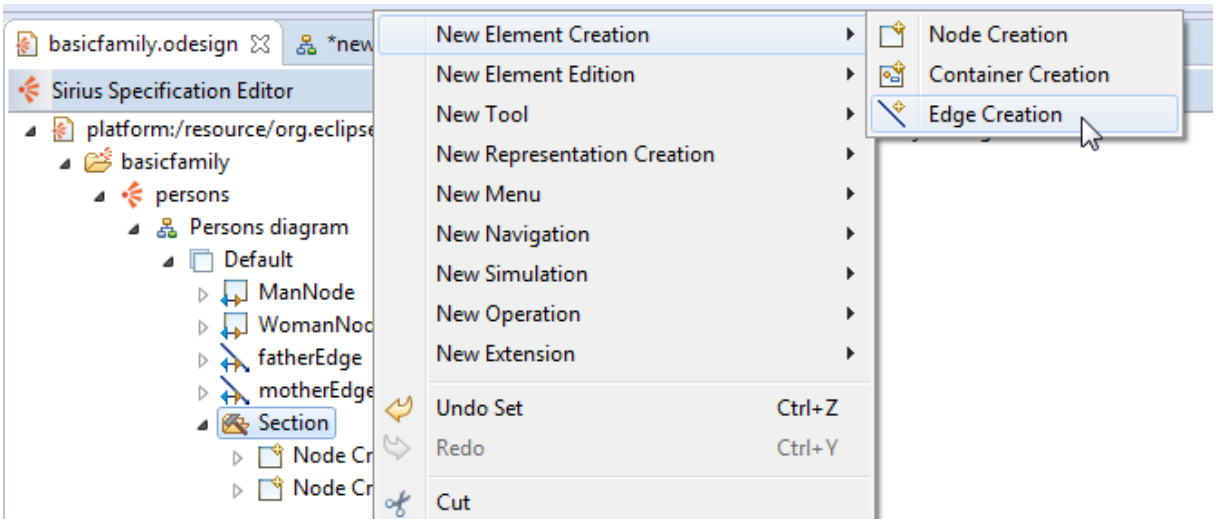


Edge Creation Tool

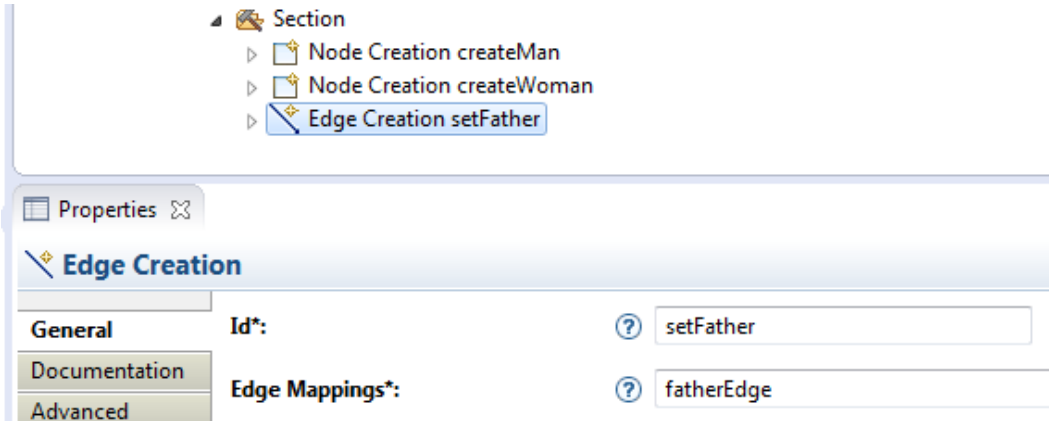
An *Edge Creation* tool allows the user to create relationships directly from the diagram, by using the palette.

Let's use this tool to allow the user to set the *father* and the *mother* relations of a *Person*.

Right click on the *Section* and select the menu **New Element Creation > Edge Creation**



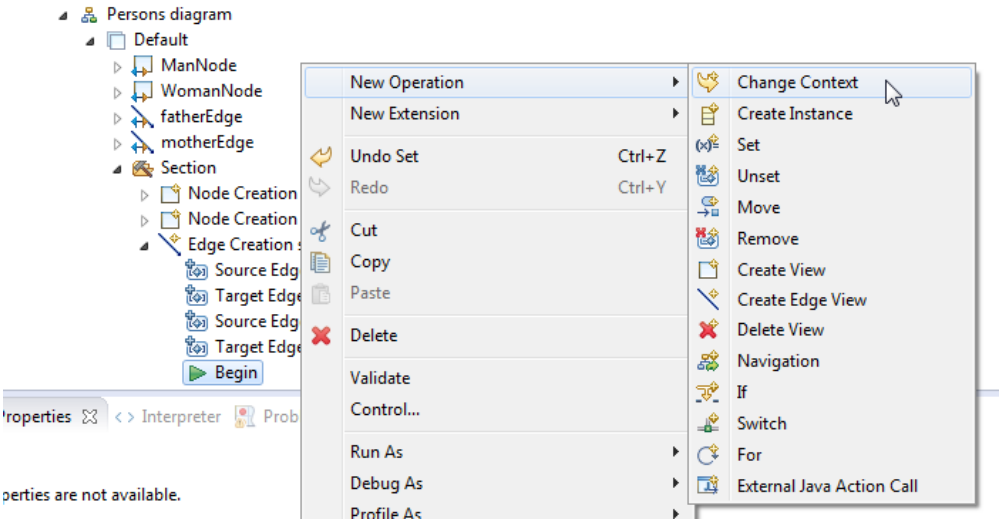
Give an Id to this tool (`setFather`) and associate it to *fatherEdge*, the *Edge Mapping* which defines the *father* graphical relation.



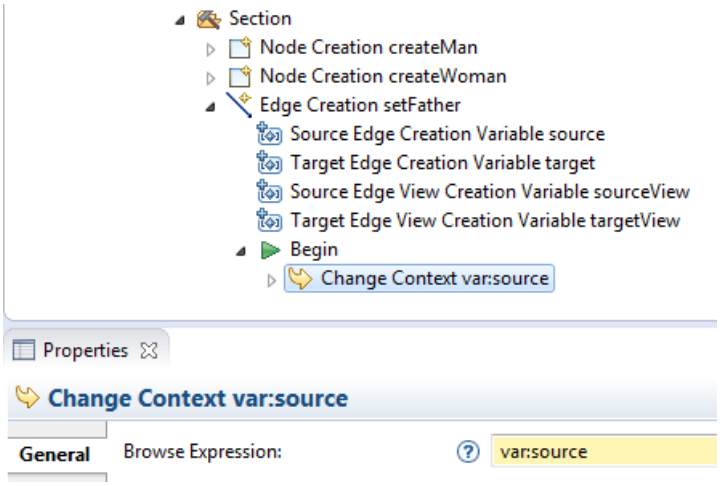
Then define the operations that will be performed by this tool each time the user will click on it. These operations can use four variables that are automatically set by Sirius:

- **source**: the first object on which the user has clicked (the source of the relation he is going to create)
- **target**: the second object on which the user has clicked (the target of the relation he is going to create)
- **sourceView**: the graphical object representing the source
- **targetView**: the graphical object representing the target

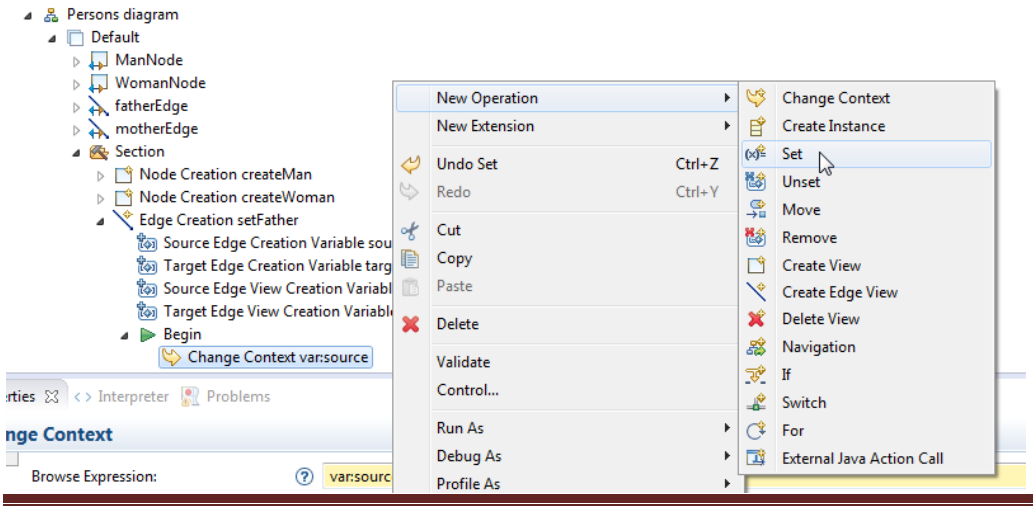
Under the *Begin* object, create a *Change Context*.



Set its *Browse Expression* to `var:source` (the first person on which the user has clicked) in order to define the execution context of the next operations.

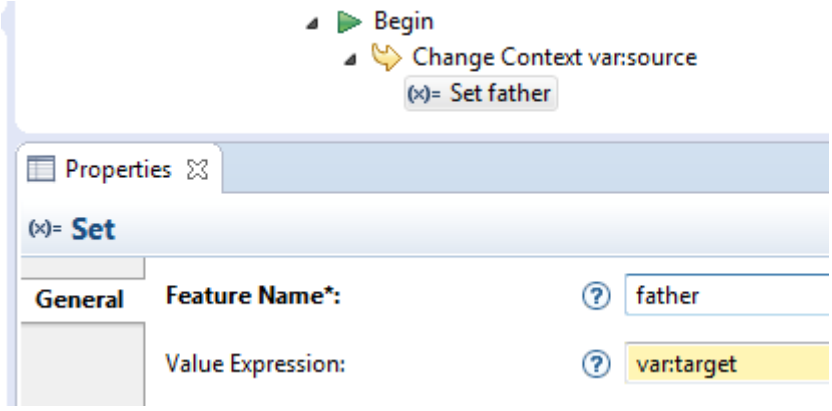


Under the *Change Context* create a *Set*.

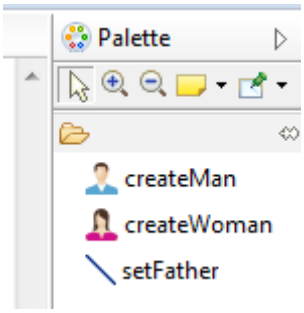


It will set the *father* of the first Person clicked (source) to the second Person clicked (target):

- Feature Name: `father`
- Value Expression: `var:target`

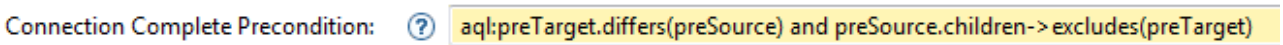


Save the *odesign* file and you will see a new tool *setFather* in the palette.



Precondition

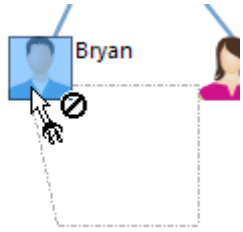
To prevent the user to create a *father* relationship from a person to itself, or to one of its children, you can add the precondition expression `aql:preTarget.differs(preSource) and preSource.children->excludes(preTarget)` (written in AQL).



This expression uses two new variables:

- **preSource**: the object on which the user has clicked first
- **preTarget**: the object under the cursor

With this precondition, the tool will prevent the creation of forbidden links by showing a specific icon.



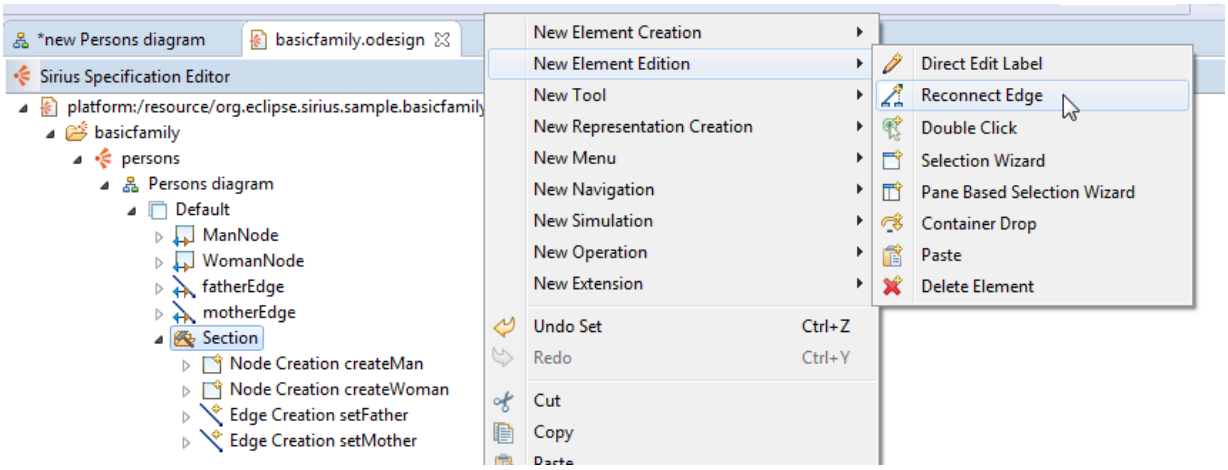
Then copy, paste and update this tool to create the *setMother* tool.

Reconnect Edge Tool

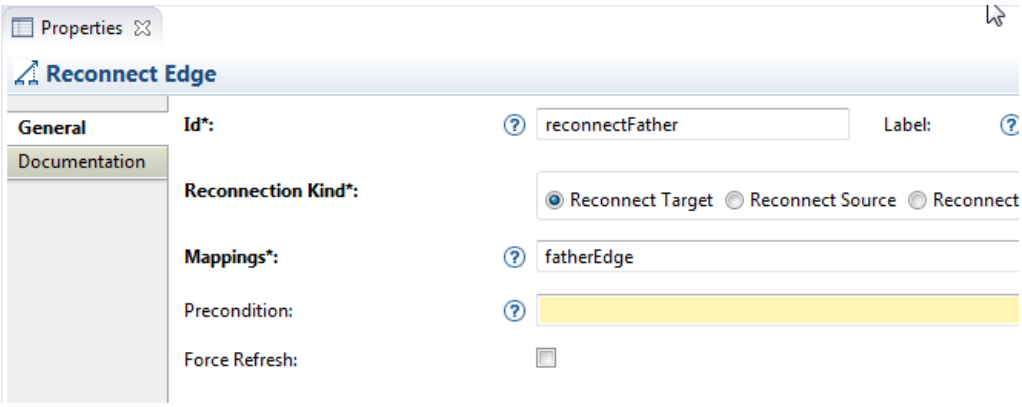
A *Reconnect Edge* tool allows the user to change the end of a relationship by moving it directly from the diagram.

Let's create a *Reconnect Edge* tool to change the *father* and the *mother* of a Person.

Right click on the *Section* and select the menu ***New Element Edition > Reconnect Edge***



Associate the *fatherEdge* to this reconnect tool.

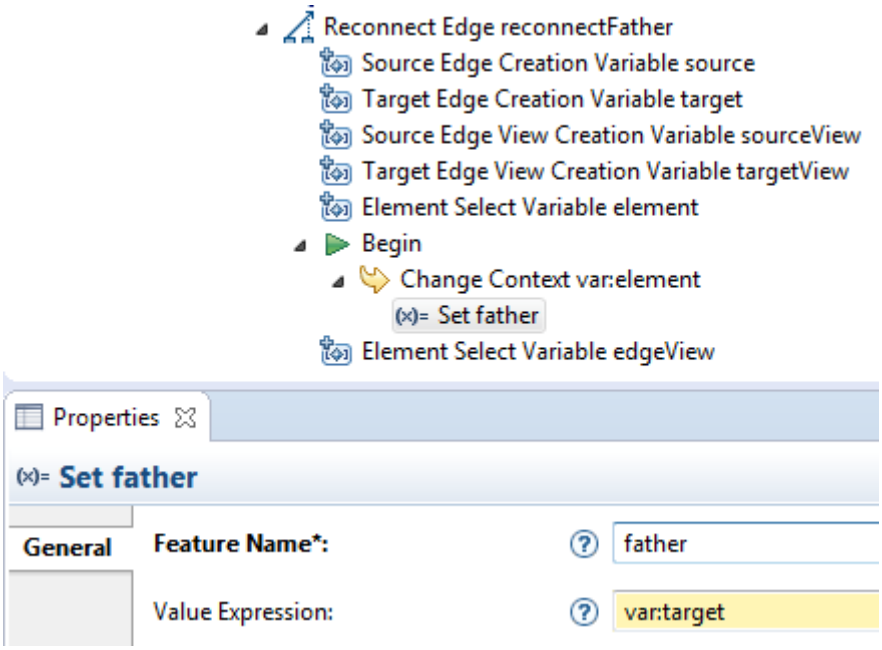


This tool comes with six variables:

- **source**: the object currently attached to the moved end
- **target**: the object going to be attached to the moved end
- **sourceView**: the graphical object representing source
- **targetView**: the graphical object representing target
- **element**: the object attached to the unchanged end
- **elementView**: the graphical object representing element

Then create a *Change Context* and set its expression to `var:element` (the person who will change his father)

Finally, create a *Set* to assign the new selected parent (`var:target`) as *father* of this person.



Copy, paste and update this tool to create a reconnect tool for *mother* relationship.

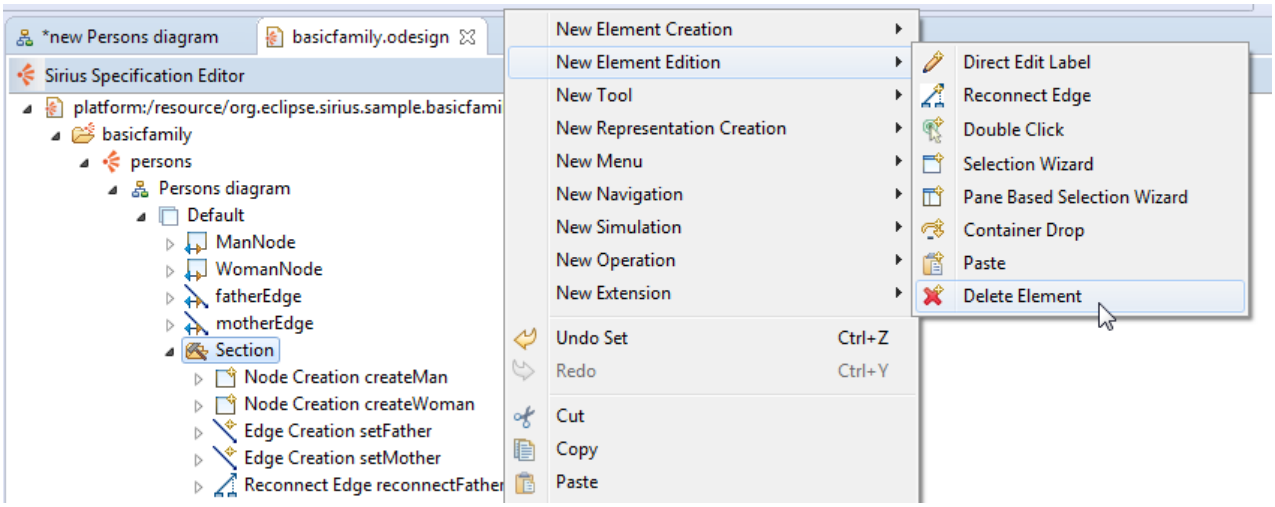
Delete Element Tool

A *Delete Element* tool specifies which actions have to be performed when the user hits the *delete* key on a diagram element.

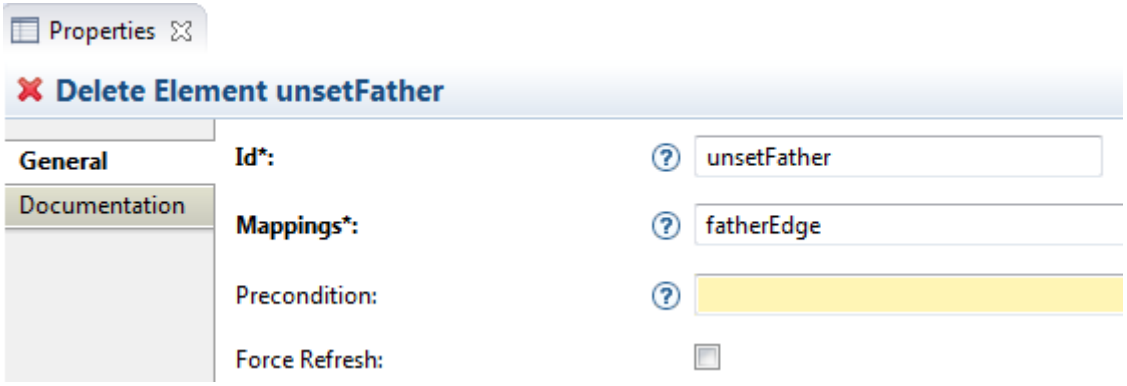
It is necessary on elements which deletion can't be interpreted by Sirius (for example *edges*) or if you need to perform specific actions during the deletion.

Let's create a *Delete Element* tool to specify what to do when the user deletes a parental relation.

Right click on the *Section* and select the menu **New Element Edition > Delete Element**

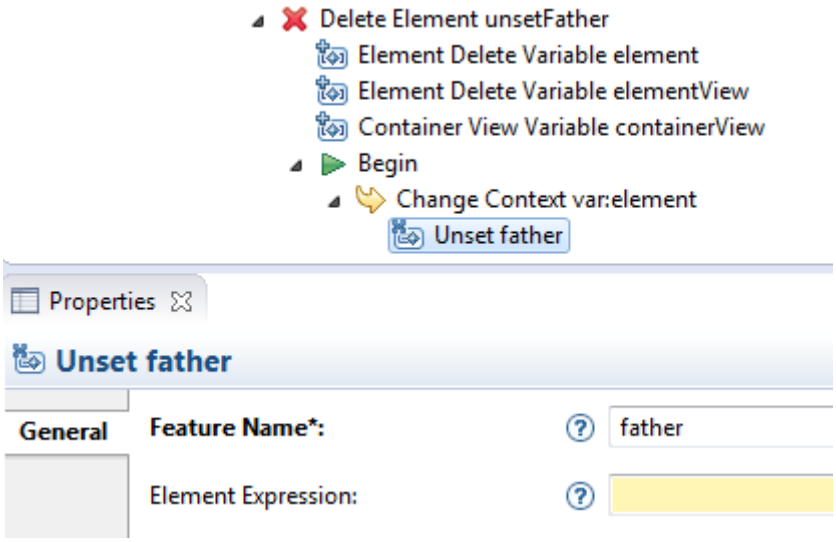


Associate the *fatherEdge* to this Delete tool.



When deleting an edge, the variable **element** refers to the source of the relationship.

So, create a *Change Context* and set its expression to `var:element` (the person which is the children). Then create a *Unset* on the feature *father* to remove this relation.



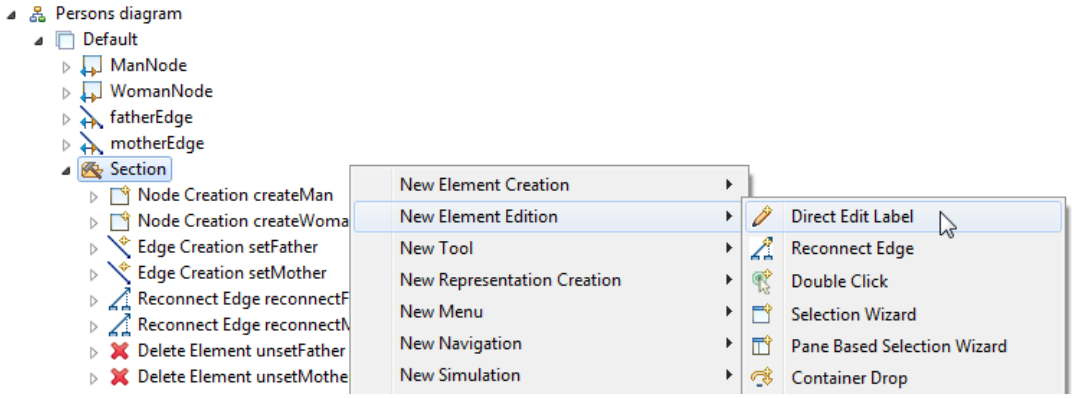
Copy, paste and update this tool to create a *delete tool* for *mother* relationship.

Direct Edit Label Tool

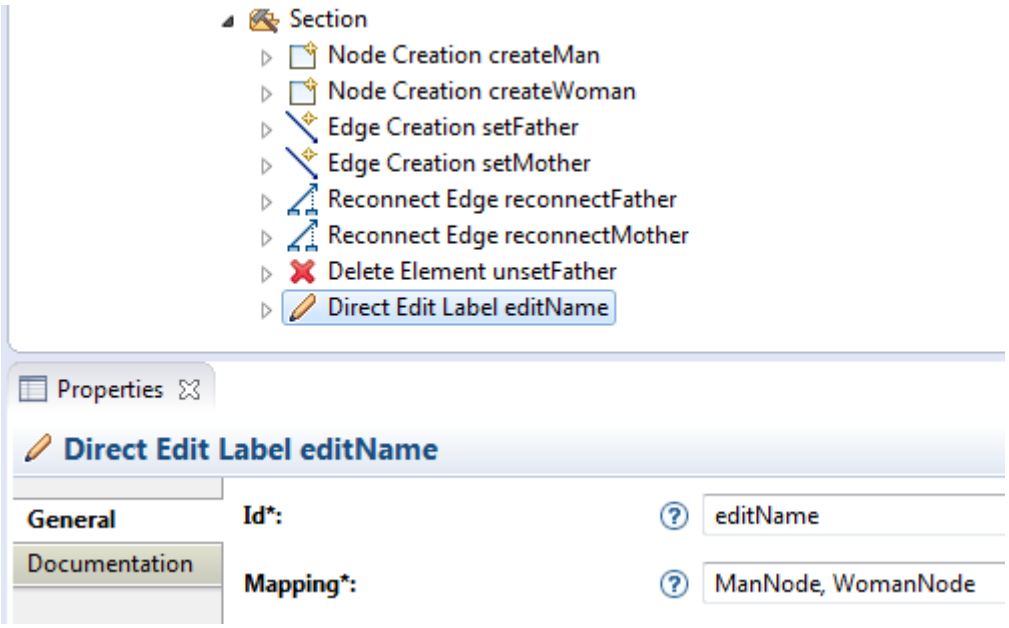
A *Direct Edit Label* specifies how to interpret the modification of graphical object's label in order to modify the model.

Let's create a *Direct Edit Label* to change the name of a Person directly from the diagram.

Right click on the *Section* and select the menu **New Element Edition > Direct Edit Label**

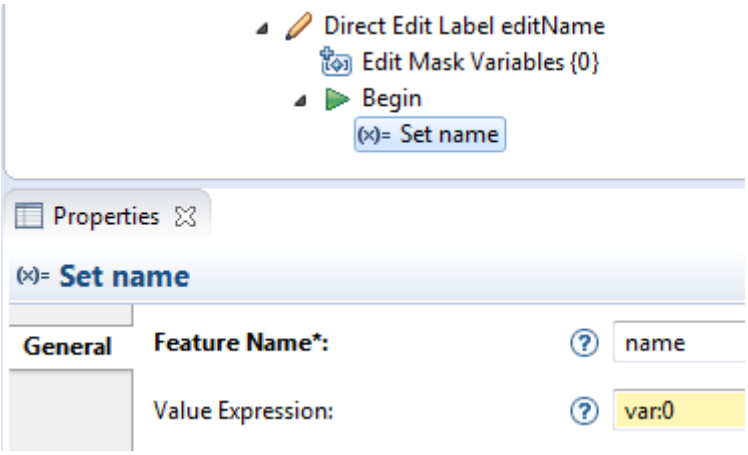


Associate this tool to the objects which have a name: *ManNode* and *WomanNode*



This tool comes with a mask that creates variables depending on the label's value. By default, the mask is set to `{0}`, which means that the variable named *0* will contain the full value of the label.

Create a *Set* to indicate the feature of the edited object that will be modified (`name`) and the value to assign (`var:0`).



The solution of is this step is available in **sirius-tutorial-step2.zip**

Advanced diagram

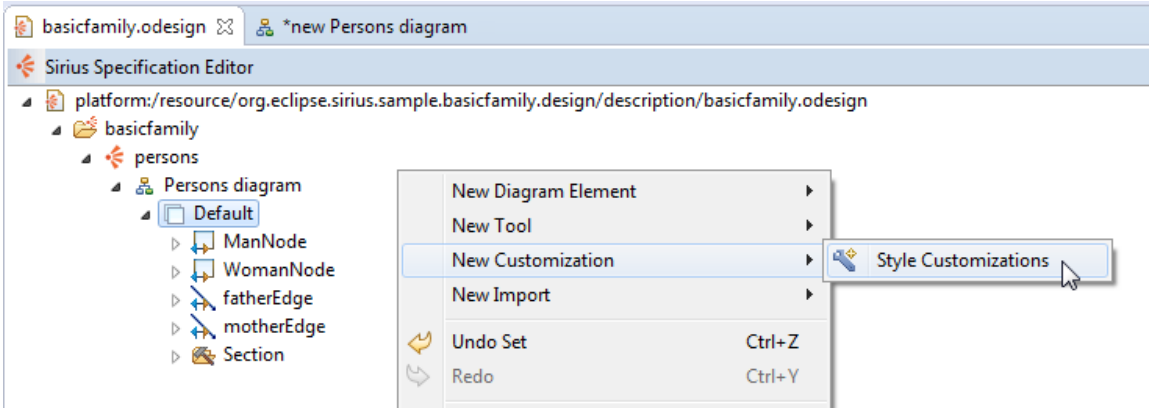
Style Customization

Style Customizations change graphical properties of diagram elements (color, label, size, ...) depending on conditions.

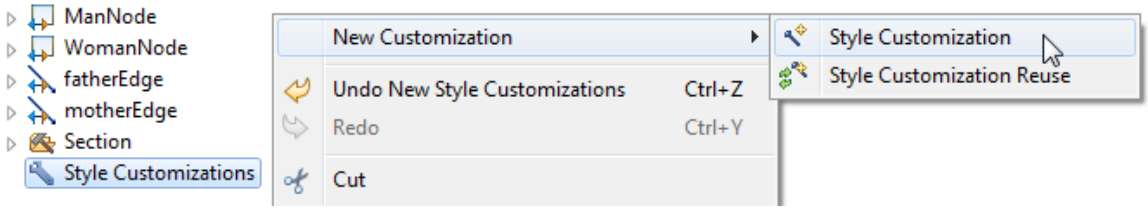
For example, let's change the label of a Person depending on the number of its children:

- Person without children => color label in gray
- Person having more than one children => set label size to 12
- Person having grandchildren => prefix label with *Grandpa* or *Grandma*

Right click on the *Default Layer* and select the menu **New Customization > Style Customizations**. It will group all the different customizations.



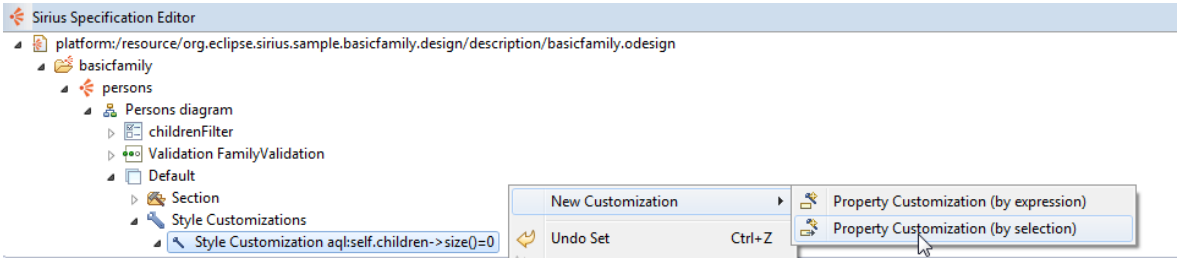
Right click on the *Style Customizations* and select the menu **New Customization > Style Customization** to create the first customization.



Enter the AQL expression associated to this customization: `aql:self.children->size()=0`

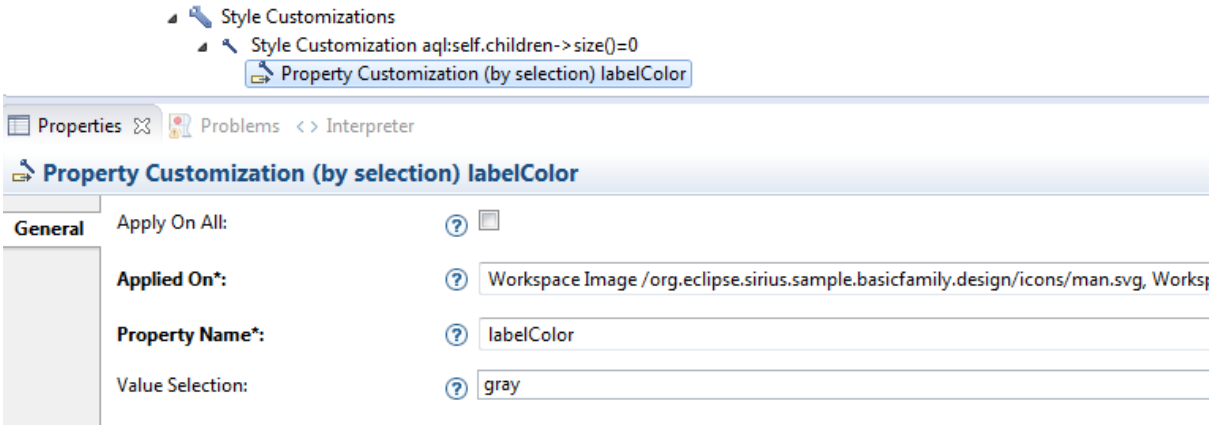
Then define the graphical properties that will change if the condition is met.

Right click on the *Style Customization* and select the menu **New Customization**. To change the label color, create a *Property Customization (by selection)*.



On this *Property Customization*, select the two Workspace Images for the *Applied On* field (they hold the label to change), enter *labelColor* in the Property Name field, then select the *gray* color.

Note that you can use the automatic-completion to select the applicable property.

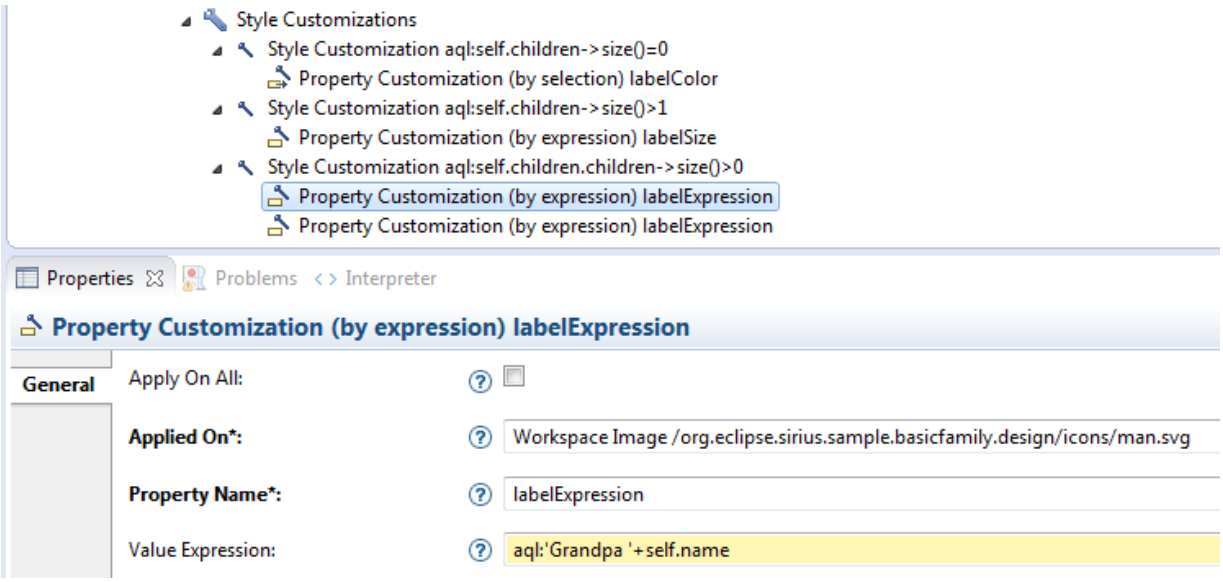


Create a second *Style Customizations* with the condition: `aql:self.children->size()>1` and add a *Style Customization (by expression)* to set the *labelSize* to `12`.

Finally, create a third *Style Customizations* with the condition:

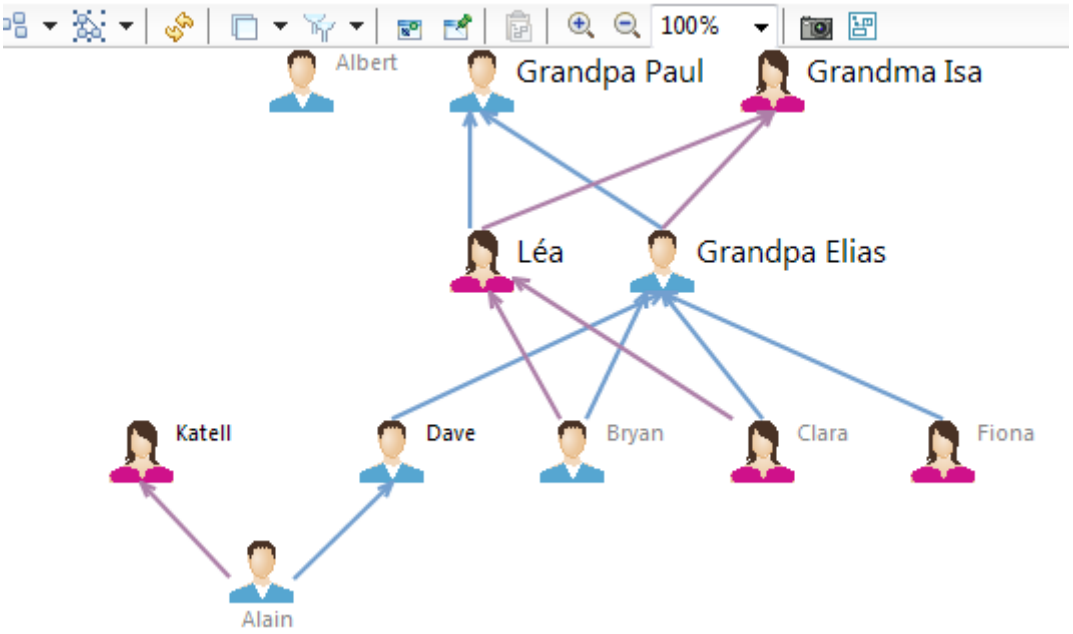
```
aql:self.children.children->size()>0
```

Add a *Style Customization (by expression)* for the *Workspace Image* corresponding to the Man, enter the property name *labelExpression* and the expression `aql:'Grandpa '+self.name` (to prefix the person name with *Grandpa*).



Do the same with the *Woman* image and the expression `aql:'Grandma '+self.name`.

Save the *odesign* file and see the result on the diagram.

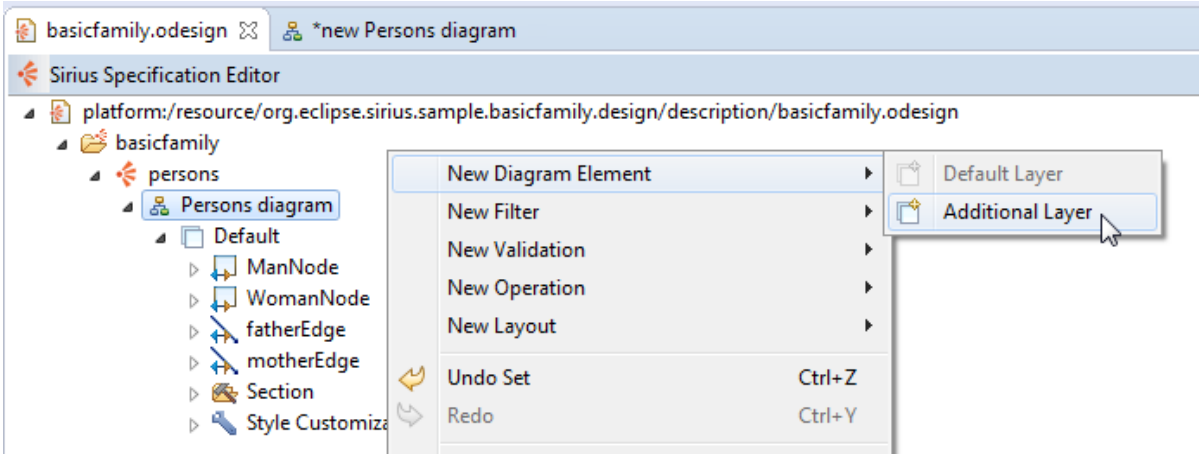


Layers

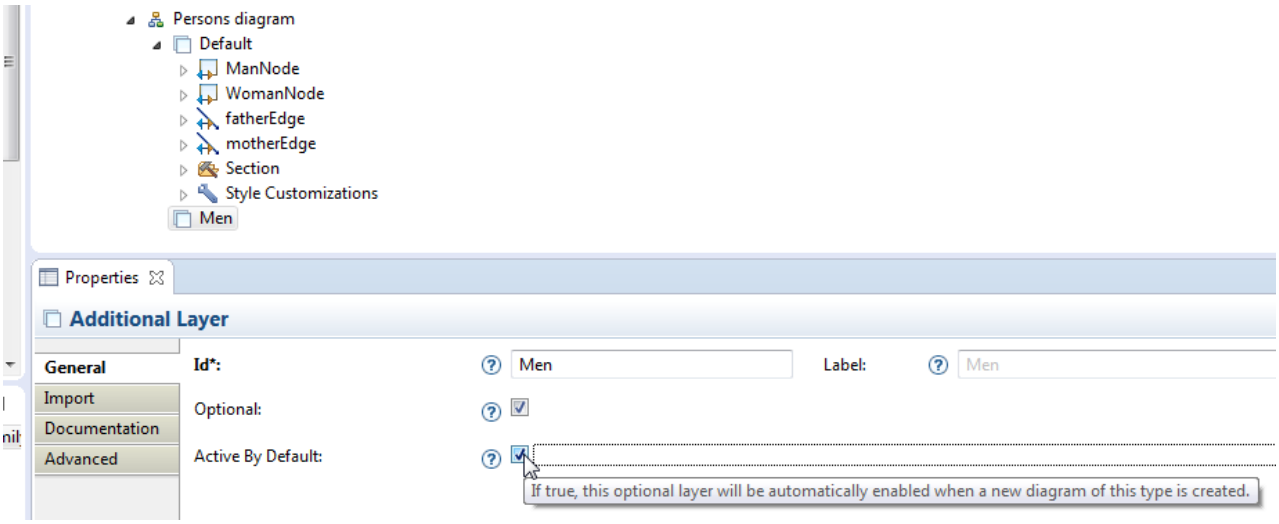
With Sirius it is possible to allocate specific graphical elements to layers that can be activated/deactivated on demand by the user.

Let's create two layers: one for the Men and one for the Women.

Right click on *Persons diagram* and select the menu **New Diagram Element > Additional Layer**.



Call this *Layer* `Men` and check the option *Active by default*.

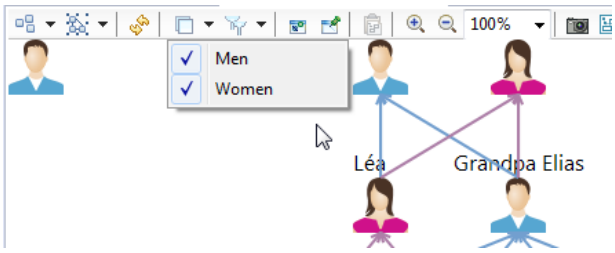


Create a *Section* in this layer then drag and drop all the elements related to the *Man* type into this layer.

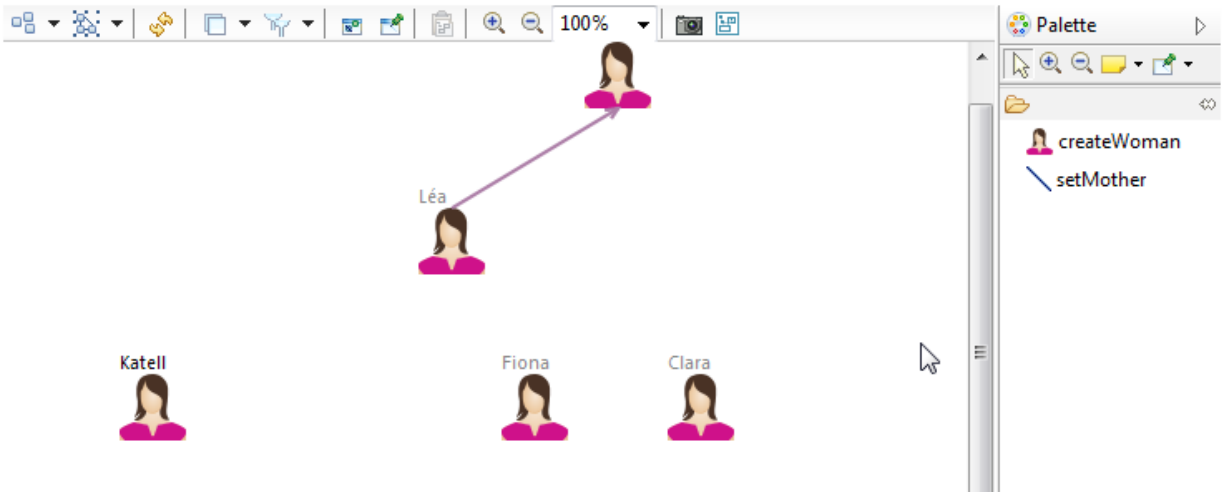
- Persons diagram
  - Default
    - WomanNode
    - motherEdge
    - Section Tools
      - Node Creation createWoman
      - Edge Creation setMother
      - Reconnect Edge reconnectMother
      - Delete Element unsetMother
      - Direct Edit Label editName
    - Style Customizations
  - Men
    - ManNode
    - fatherEdge
    - Section Men Tools
      - Node Creation createMan
      - Edge Creation setFather
      - Reconnect Edge reconnectFather
      - Delete Element unsetFather

Create a second *Layer* `Women` and drag and drop all the elements related to the *Woman* type.

Re-open the sample diagram. You can see the two optional layers Men and Women (selected by default).



If you de-activate the *Men* layer, only women remain on the diagram. Note that the tools related to men have been hidden too.

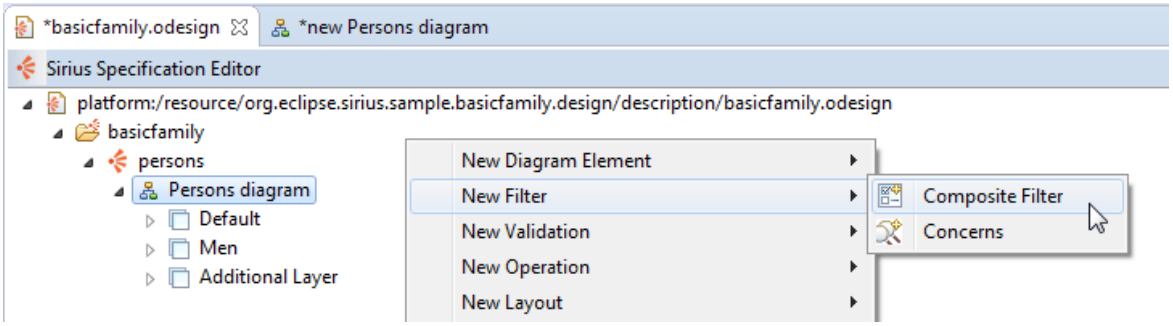


Filters

With *Filters*, Sirius provides an additional mechanism to automatically show or hide elements of a diagram: a condition determines which elements remain visible.

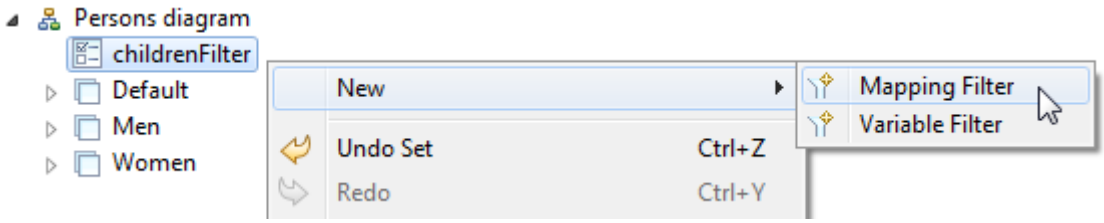
Let's create a *Filter* to hide persons without children.

Right click on *Persons diagram* and select the menu **New Filter > Composite Filter**.

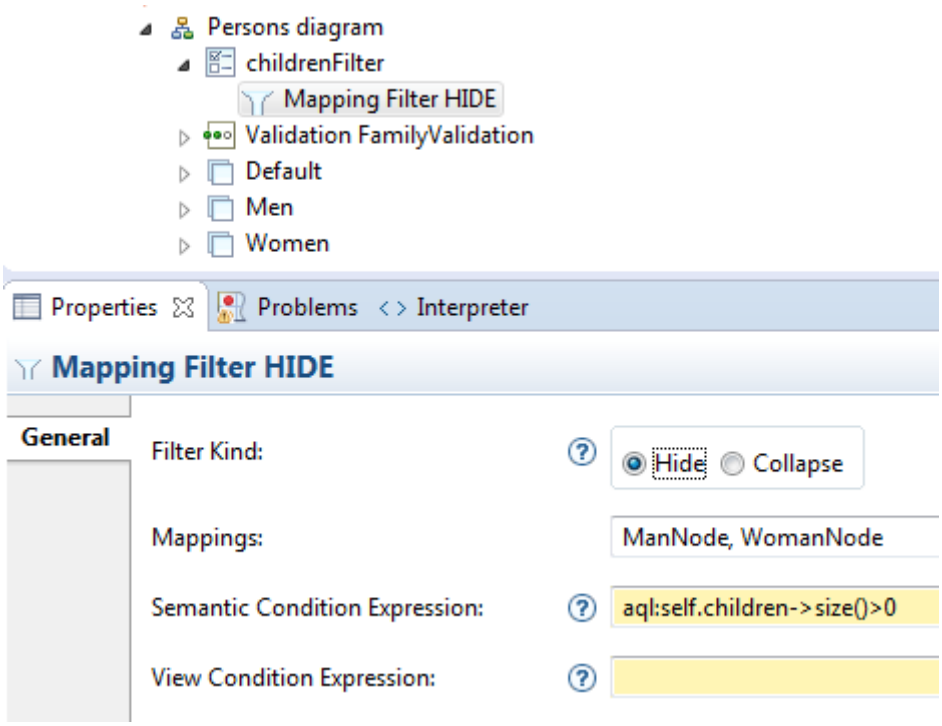


Set the *id* to `childrenFilter`.

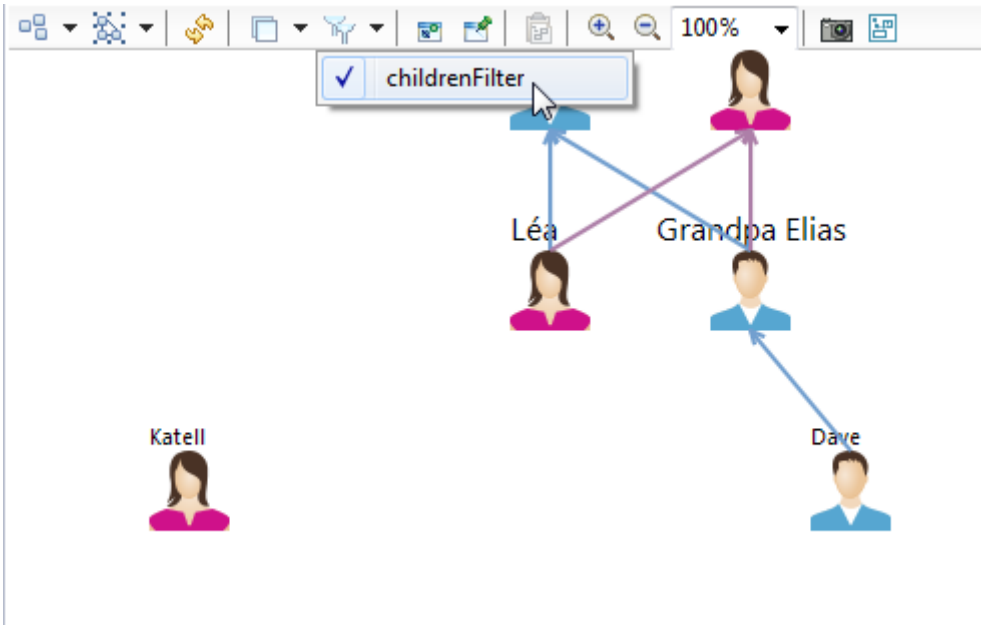
Then create a *Mapping Filter*.



Associate *ManNode* and *WomanNode* to this filter and define the *Semantic Condition Expression* : `aql:self.children->size()>0`



On the diagram, you can now select the filter *childrenFilter* and see that only persons with children remain visible.



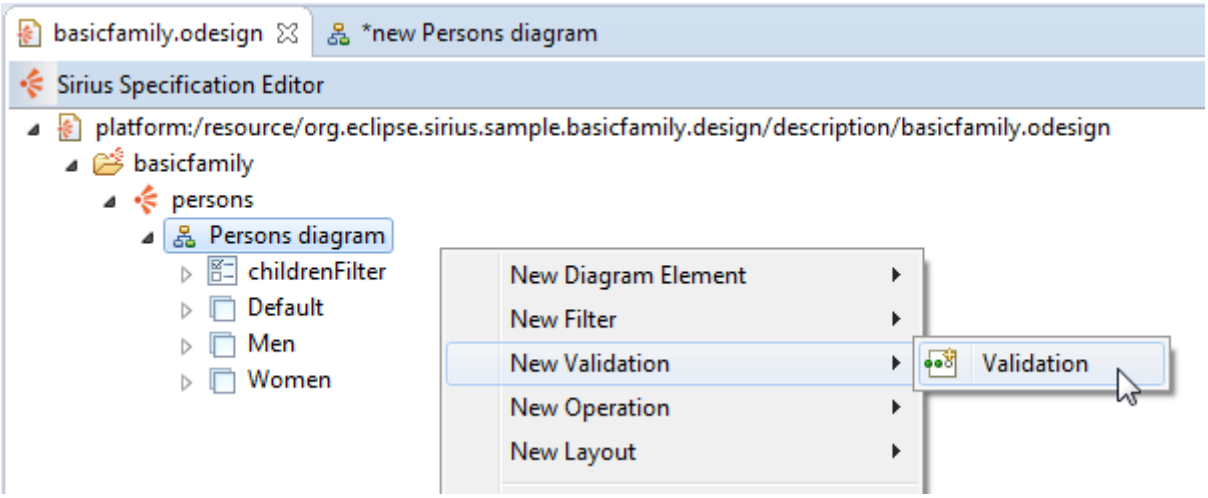
Validation Rules

Validation rules allow the user to evaluate the quality of a model.

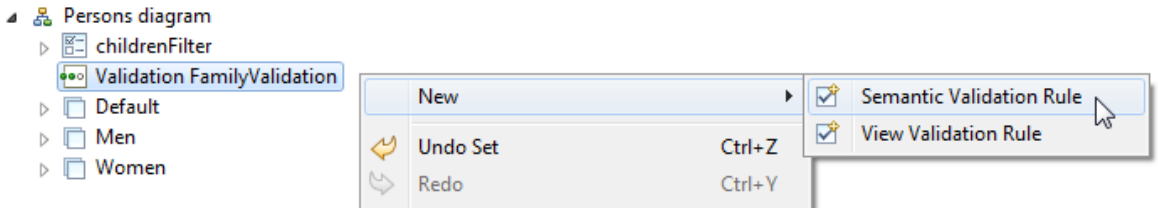
For example, despite the precondition we have created on the *Eddge Creation* tool, users can still create circular parental relations between two persons by using the *Properties View*.

Let's create a *Validation Rule* to detect this problem.

Right click on *Persons diagram* and select the menu **New Validation > Validation**.

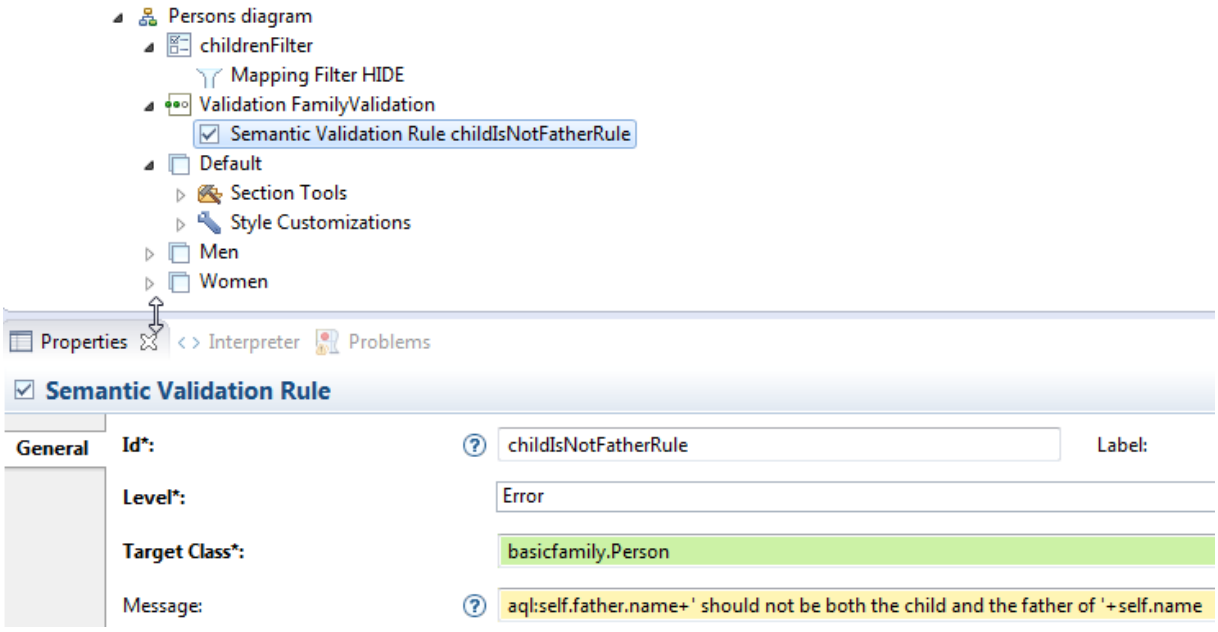


Set the name `FamilyValidation` to this *Validation*. Then create a *Semantic Validation Rule*.

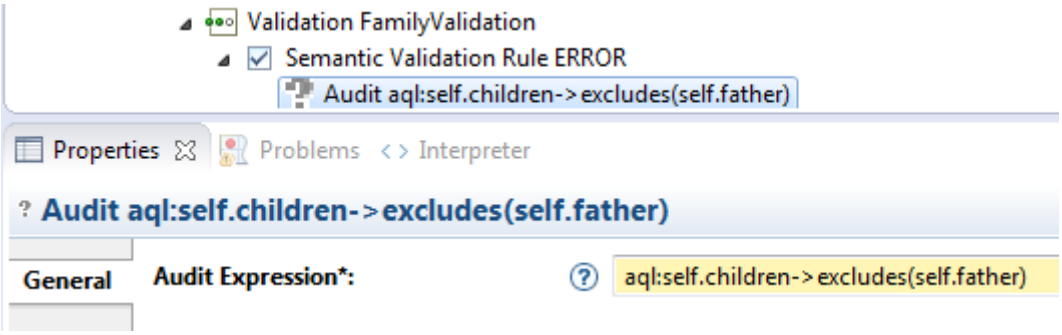


Set the properties of this rule:

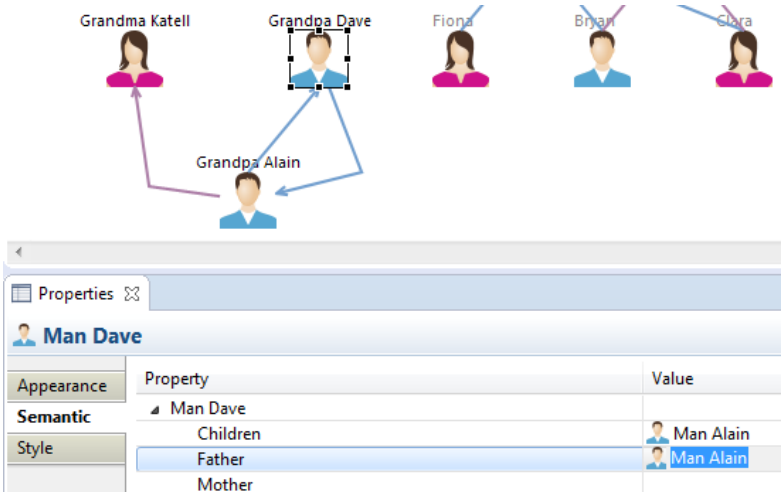
- **Level**: `Error`
- **Target Class**: `basicfamily.Person`
- **Message**: `aql:self.father.name+' should not be both the child and the father of '+self.name`



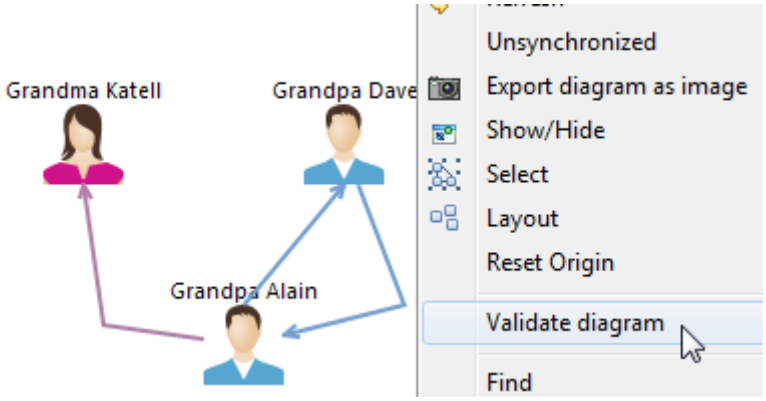
Then create an *Audit* (a condition corresponding to this rule) with the expression `aql:self.children->excludes(self.father)`.



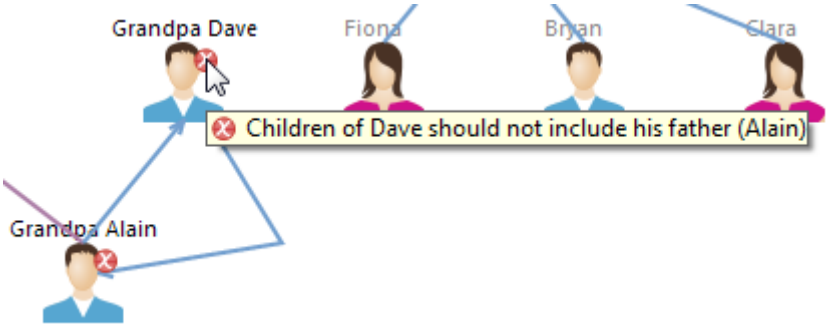
To test this rule, introduce an error in the model with the *Property View*: set the father of a man to its son.



Then right-click on the diagram and select the menu **Validate diagram**.



The circular father relationship is detected and a decorator (corresponding to the rule level) is displayed on the persons with this problem. A tootip contains the error message.

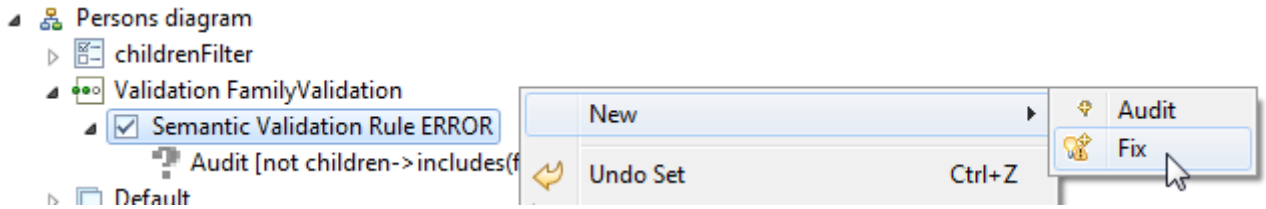


Quick Fixes

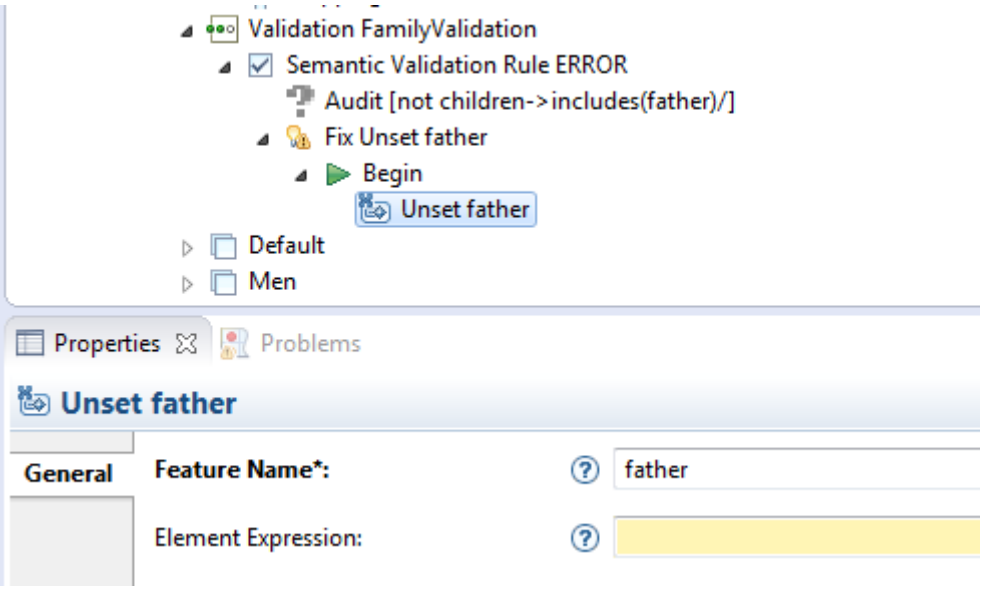
Fixes can be provided to the user for solving the problem automatically.

For example, we can propose to unset one of the two *father* relations.

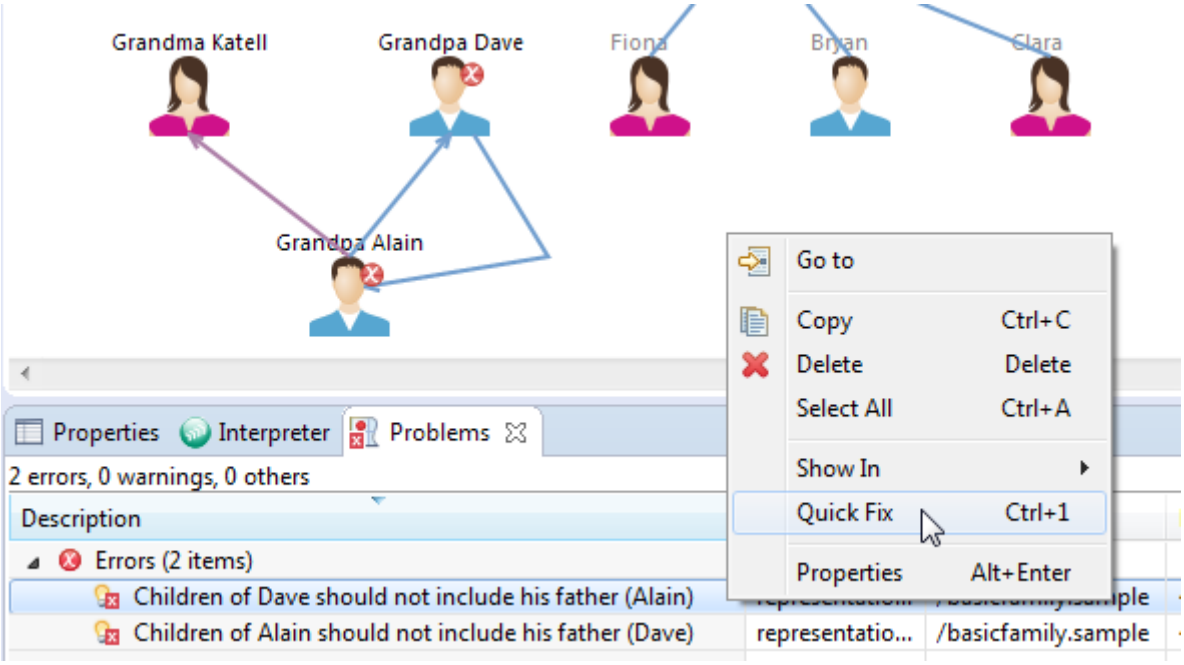
Right-click on the *Rule* and select the menu **New > Fix**.



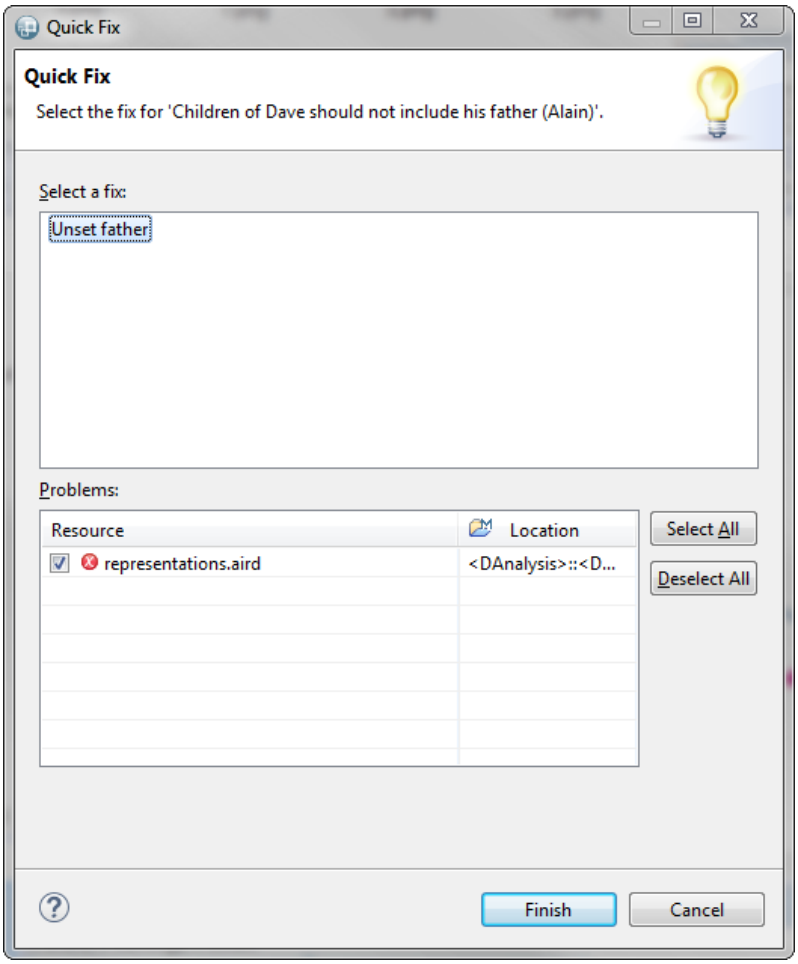
Set the *name* `Unset father` to this *Fix* and create a *Begin*. Then create a *Unset* and enter `father` in the *Feature name*.



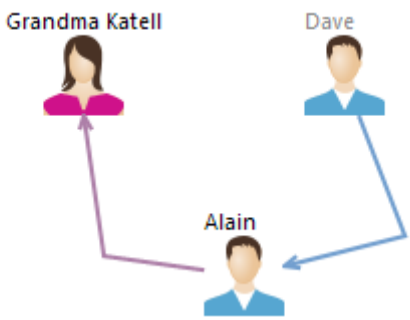
If you open the *Problems View*, you can right-click on one of the errors and select the menu **Quick Fix**.



A dialog opens which proposes the available fixes.



Select *Unset father* and click on *Finish*: the father of the selected person is automatically unset and the problem is solved.

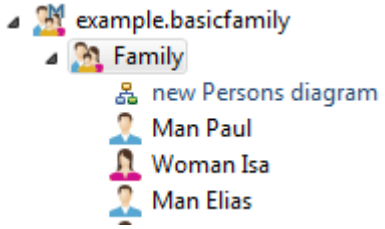


The solution of is this step is available in **sirius-tutorial-step3.zip**

Other representations

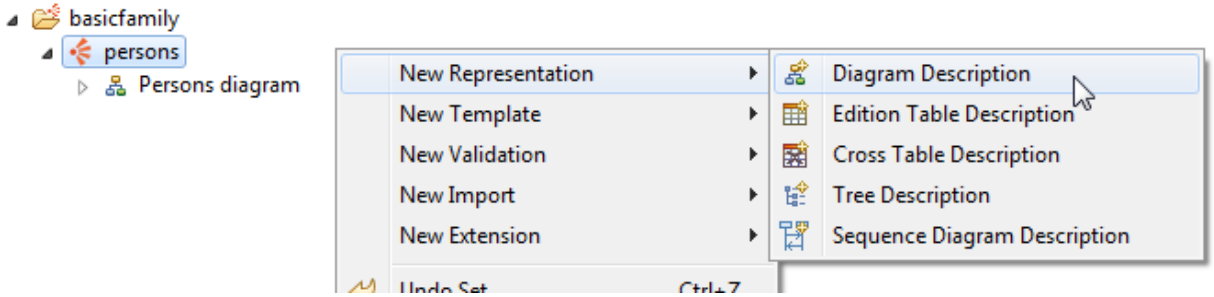
Object-Centered Diagram

The *Persons diagram* was a representation related to the whole model: it displays all the members of the family. This is the reason why it belongs to the *Family* instance, as you can see in the *Model Explorer*.



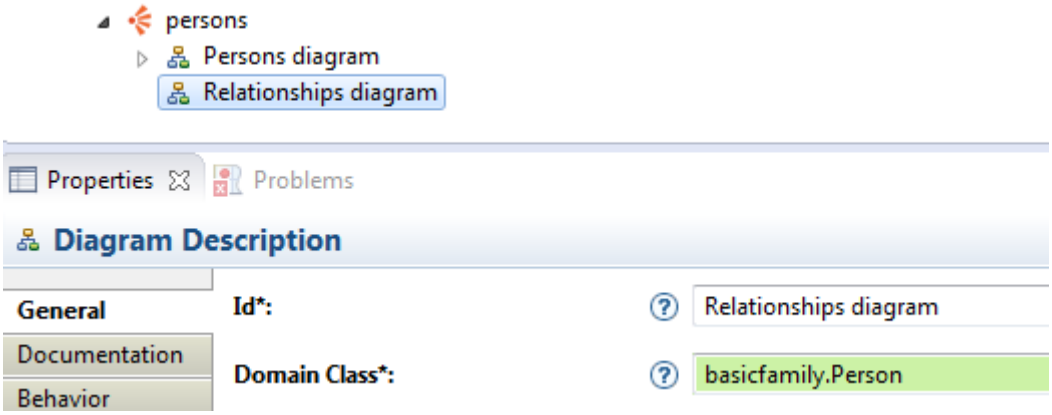
With Sirius, you can create a representation on any instance of the model. For example, let's create a person-centered diagram to show only the information related to a given person.

Right-click on the *persons* viewpoint and select the menu **New Representation > Diagram Description**.

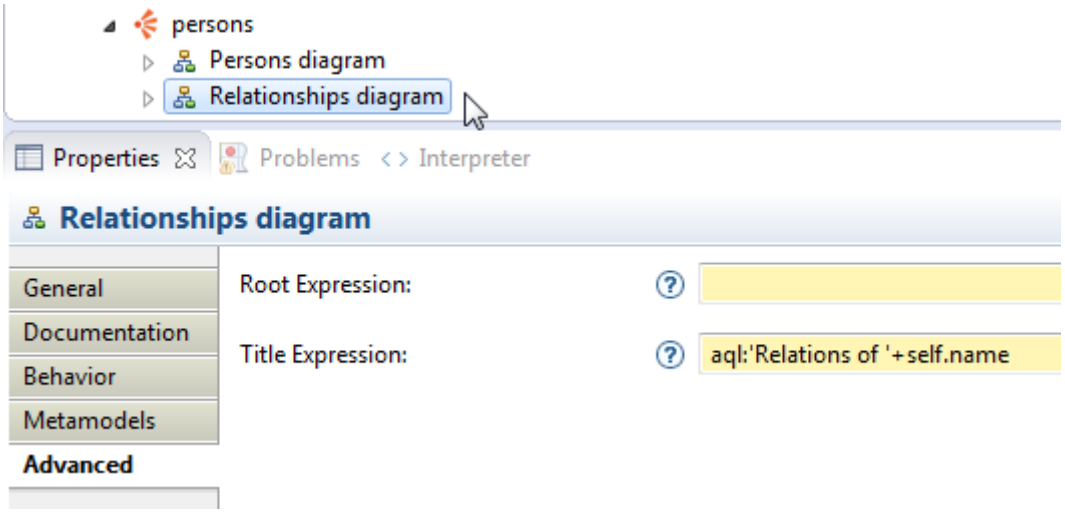


Set these properties to this representation:

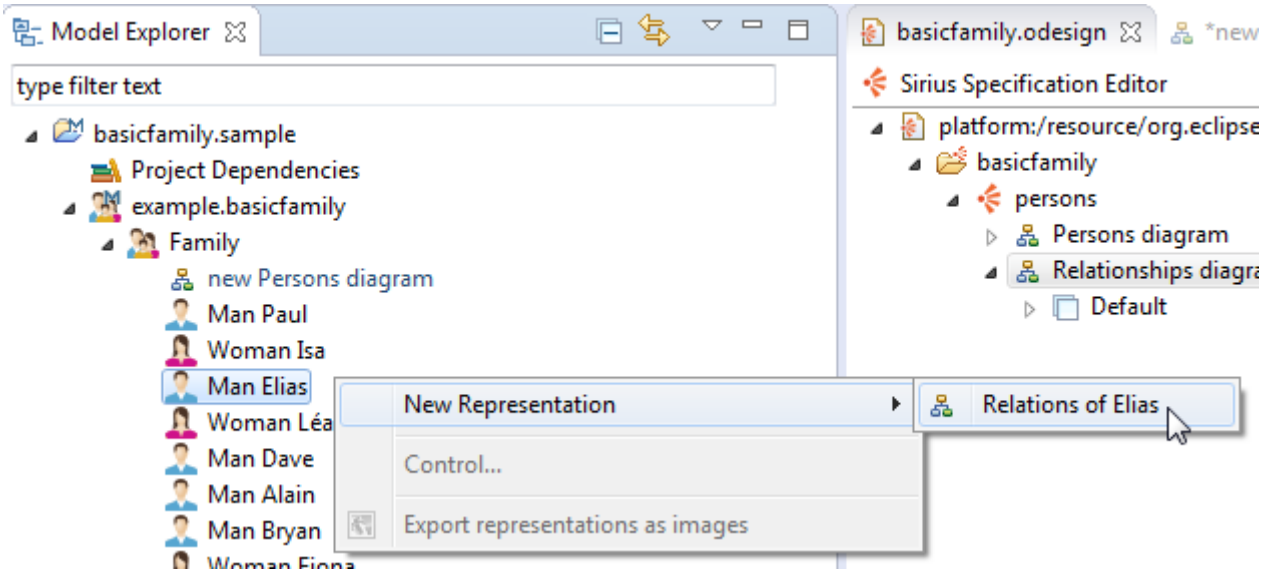
- **Id**: `Relationships diagram`
- **Domain Class**: `basicfamily.Person`



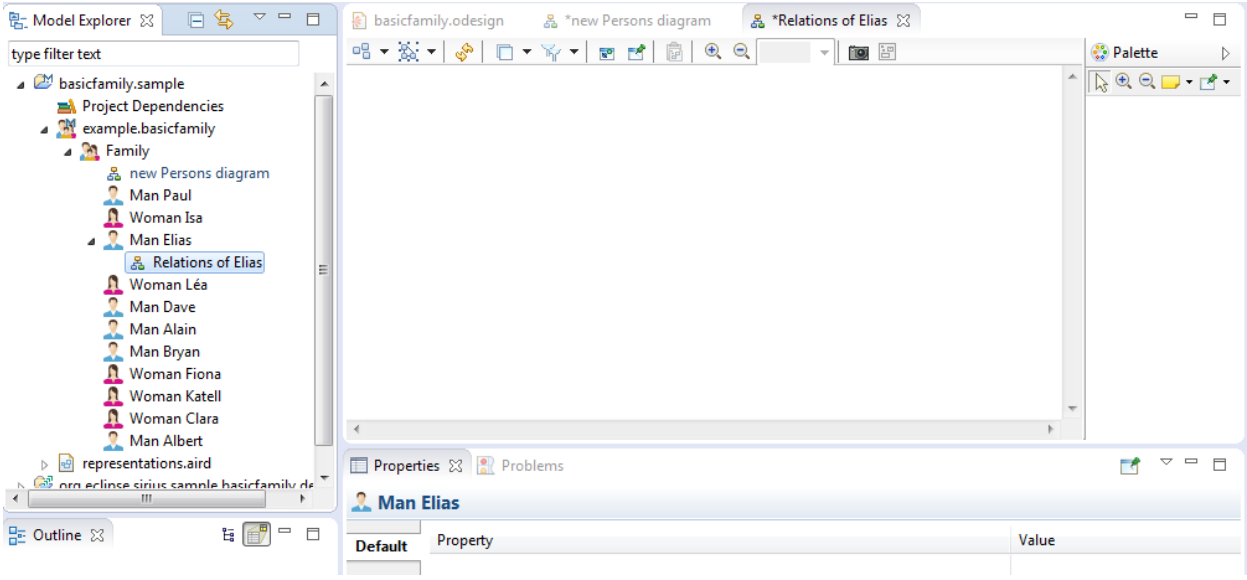
In the *Advanced* tab, set the **Title Expression** to `aql:'Relations of '+self.name` in order to customize the name of the diagram with the person's name.



Save the *odesign* file. Now, you can right click on a person in the *Model Explorer* and create a new representation of type *Relationships diagram*.



The result is a blank diagram, since we have not yet specified its content.



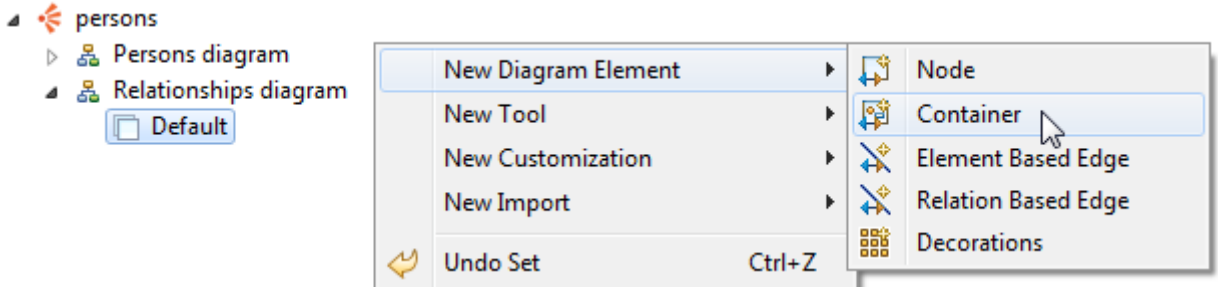
Container

A *Container* is a kind of diagram element that can contain other diagram elements.

On the *Relationships diagram*, let's use a containers to represent the current person (who owns the diagram) with:

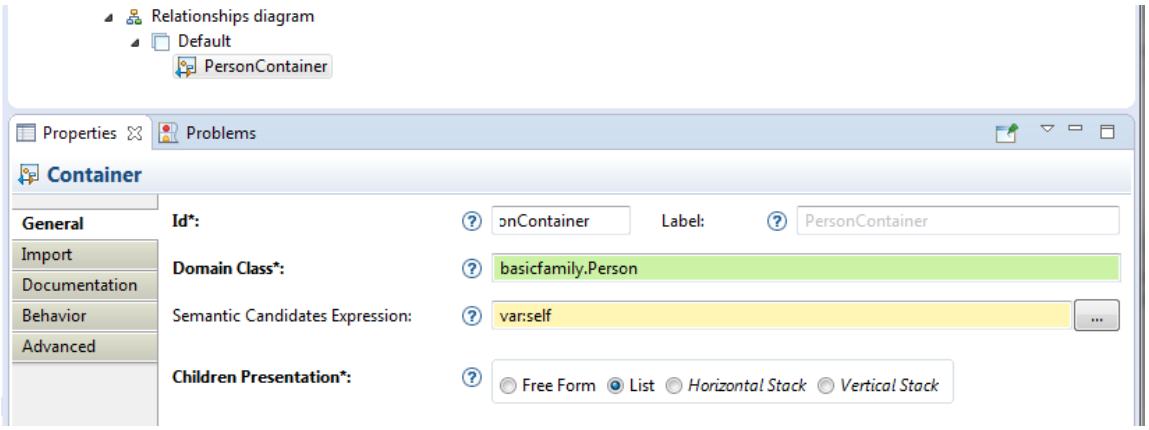
- each children in a list
- its parents as border nodes

Right-click on the *Layer* and select the menu **New Diagram Element > Container**.

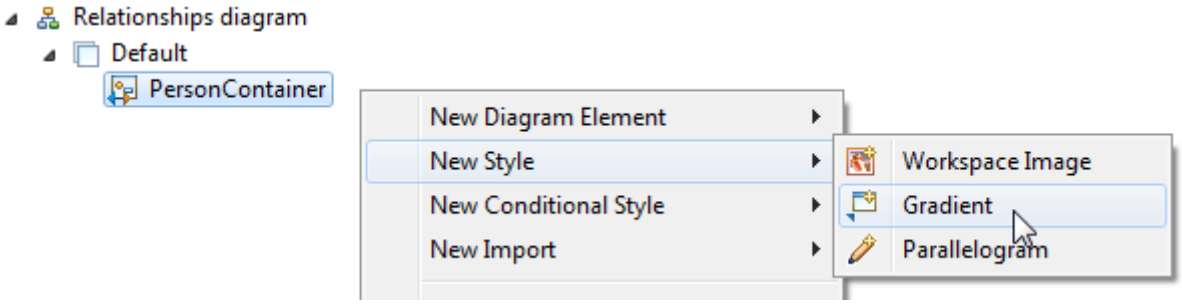


For this container, set these properties:

- **Id**: `PersonContainer`
- **Domain Class**: `basicfamily.Person`
- **Semantic Candidate Expression**: `var:self` (the current person)
- **Children Presentation**: `List`

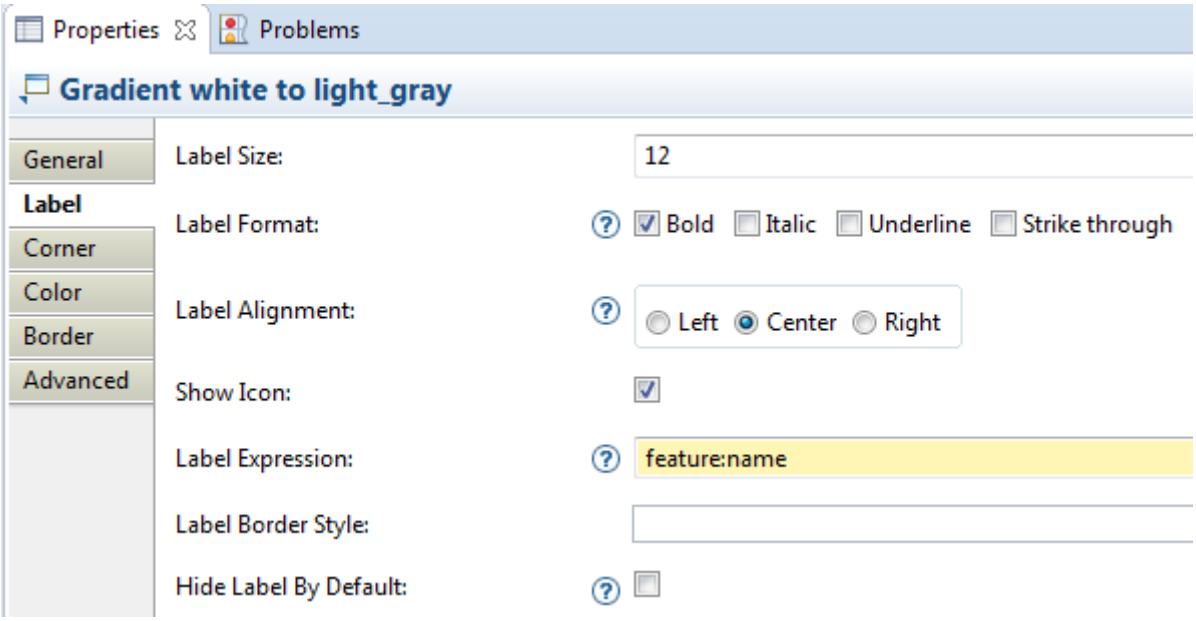


To define the graphical rendering of this container, right-click on it and select the menu **New Style > Gradient**.

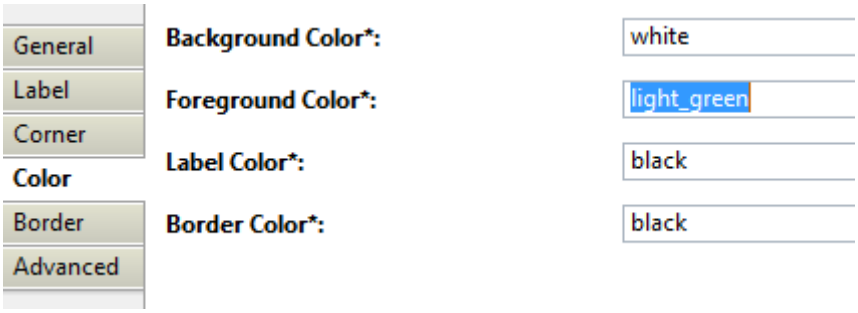


On this style, set these properties:

- *Label* Tab:
  - **Label Size**: `12`
  - **Label Format**: `bold`



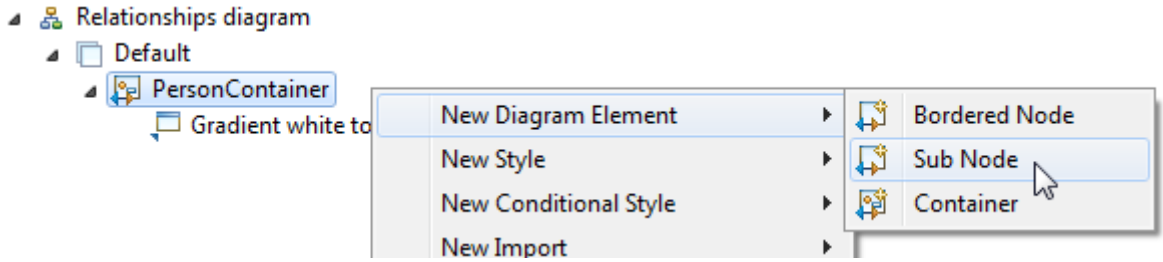
- *Color* Tab:
    - **Foreground Color**: `light_green`



Save the *odesign* file and now you see a green container appear on the diagram that you have created for a person.

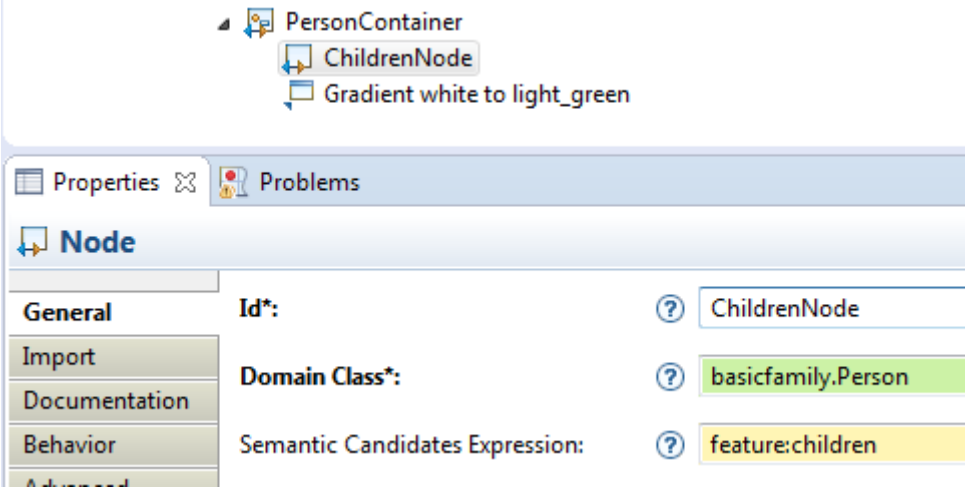
Sub Nodes

Now, to display children inside this container, right-click on the container definition and click on the menu **New Diagram Element > Sub Node**.

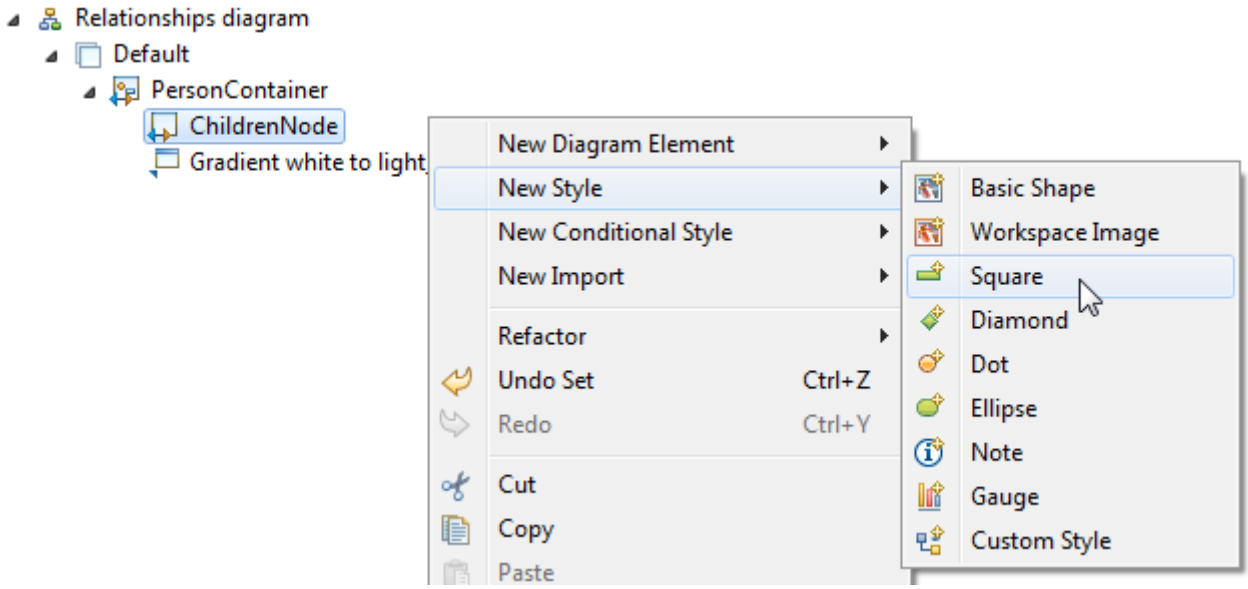


On the *Sub Node* set these properties:

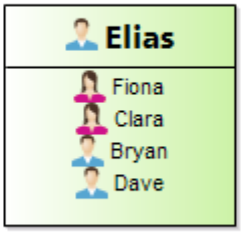
- **Id**: `ChildrenNode`
- **Domain Class**: `basicfamily.Person`
- **Semantic Candidate Expression**: `feature:children`



Then create a style. Note that the kind of style is not important here: for elements in a list, the style is used only to specify the label.



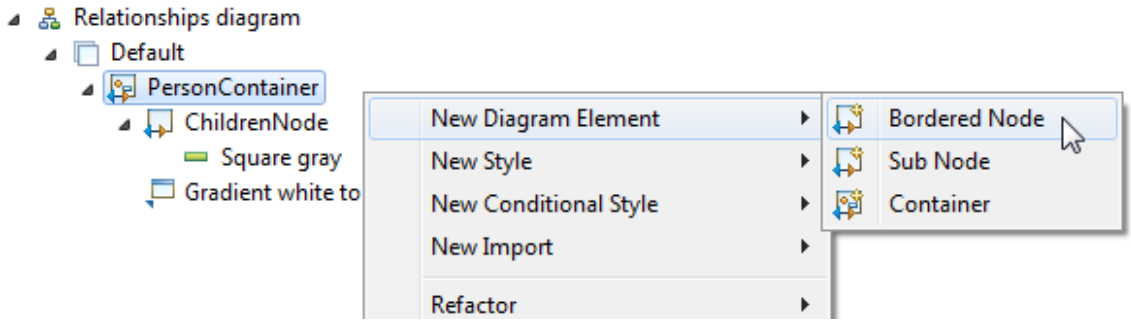
Save the *odesign* file, and now you can see the children listed inside the container.



Border Node

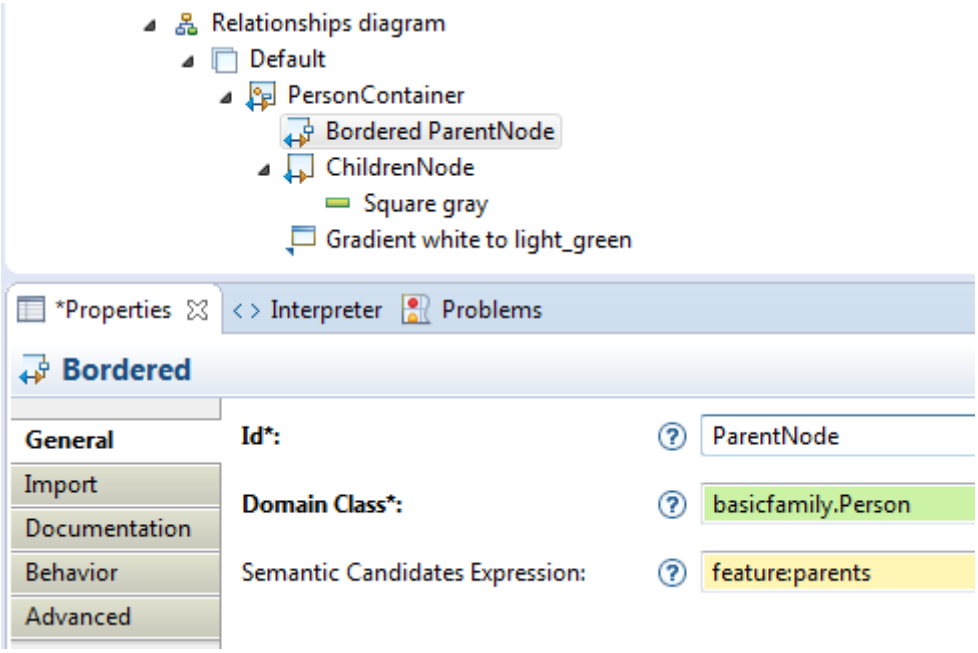
Now let's add *Border Nodes* to this container to display the parents of the current person.

Right-click on *PersonContainer* and select the menu **New Diagram Element > Border Node**:



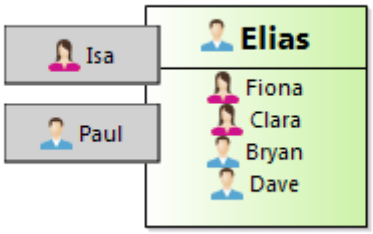
Set these properties to the border node:

- **Id**: `ParentNode`
- **Domain Class**: `basicfamily.Person`
- **Semantic Candidate Expression:** `feature:parents`



Add a *square* as style and set its Color to `light_gray`

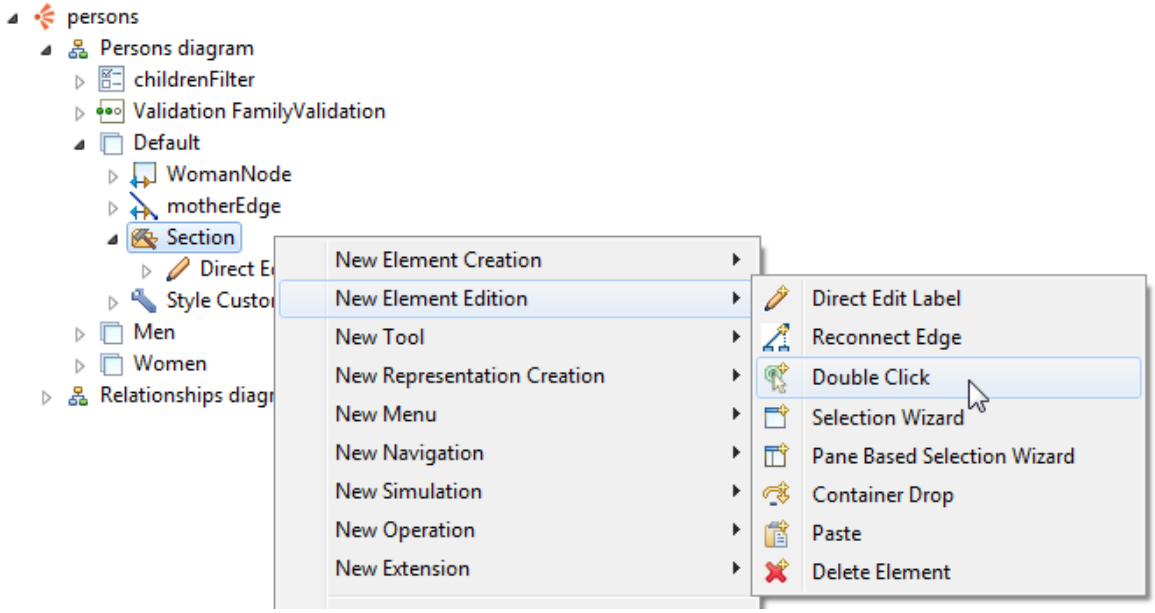
Save the *odesign* file and now you can see the parents attached on the border of the first container:



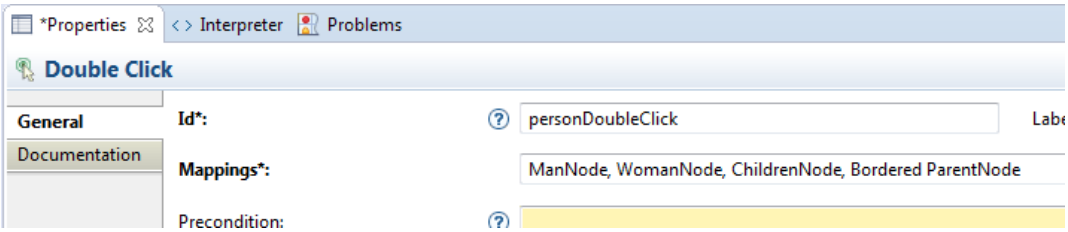
Double-click

Now we can create a specific diagram for each person. To facilitate the navigation between all these diagrams, let's create a *Double Click* tool that allows the user to navigate from any person on a diagram to its dedicated diagram.

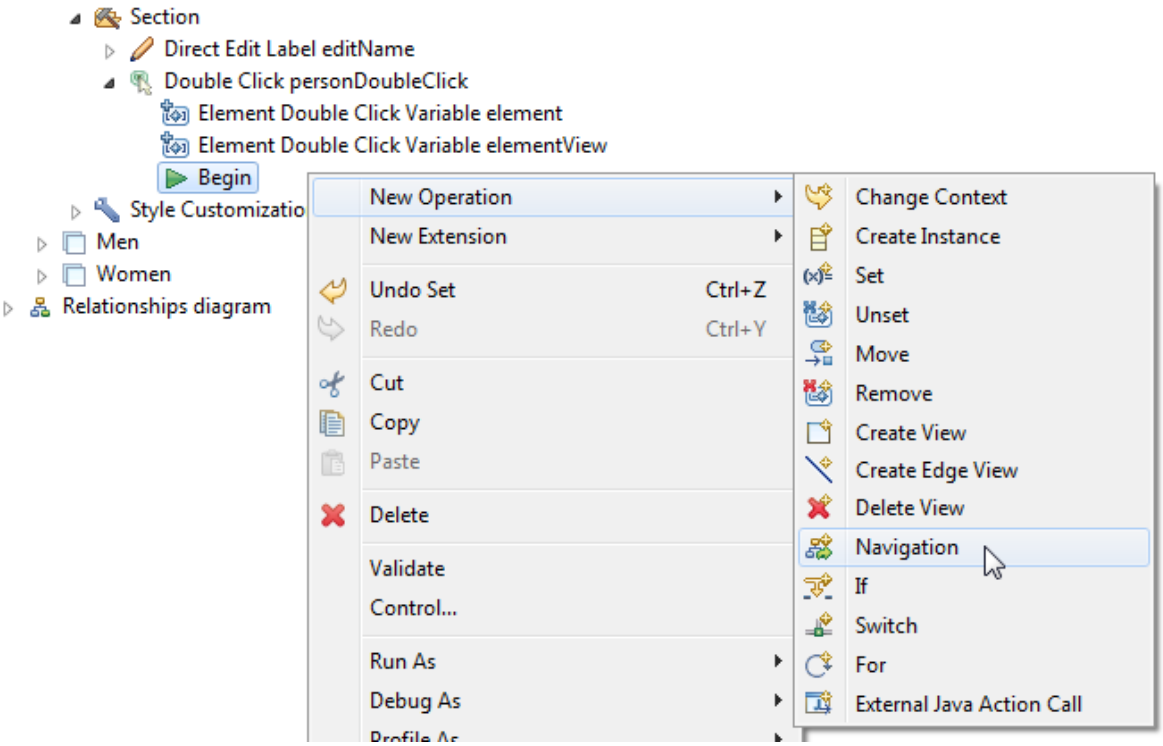
Go back to the first diagram, right click on the *Section* in the first *Layer* and select the menu **New Element Edition > Double Click**.



On the *Mappings* property of the *Double Click* select all the elements of the modeling tool that represent a *Person* (except the two containers, since it doesn't make sense to navigate to the current diagram).

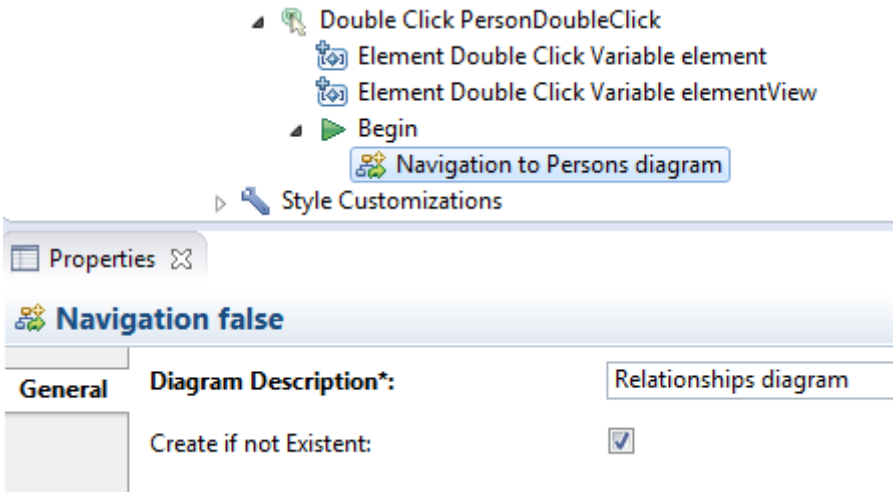


To specify the action of the *Double Click*, right-click on the *Begin* and select the menu **New Operation > Navigation**.

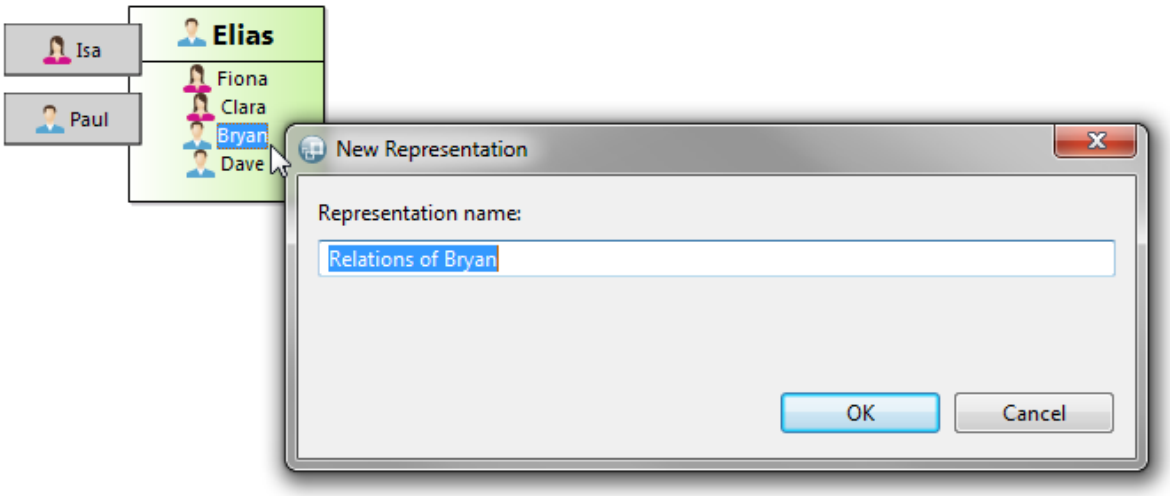


Then set these properties to the *Navigation* :

- **Diagram Description**: `Relationships diagram`
- **Create if not Existent:** `true`



Save the *odesign* file, close and reopen your diagrams. Now you can double-click on persons to open their corresponding *Relationships diagram*. If this diagram has not been created yet, Sirius will propose to create it.



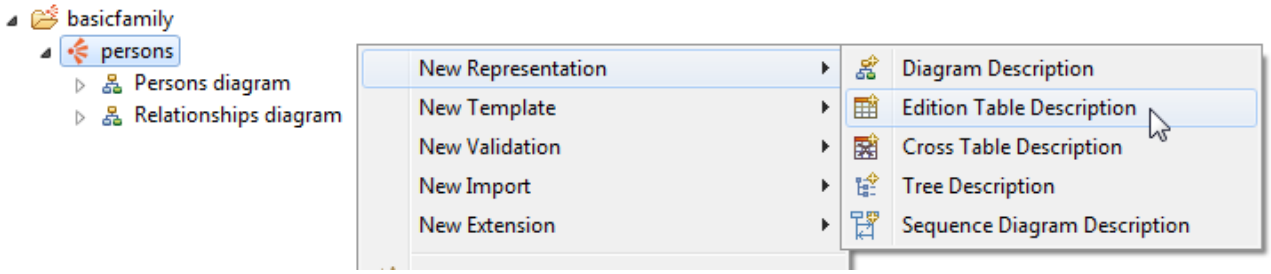
Table

With Sirius it is also possible to represent model elements with a *Table*.

Let's create a table that lists all the persons, with this information for each:

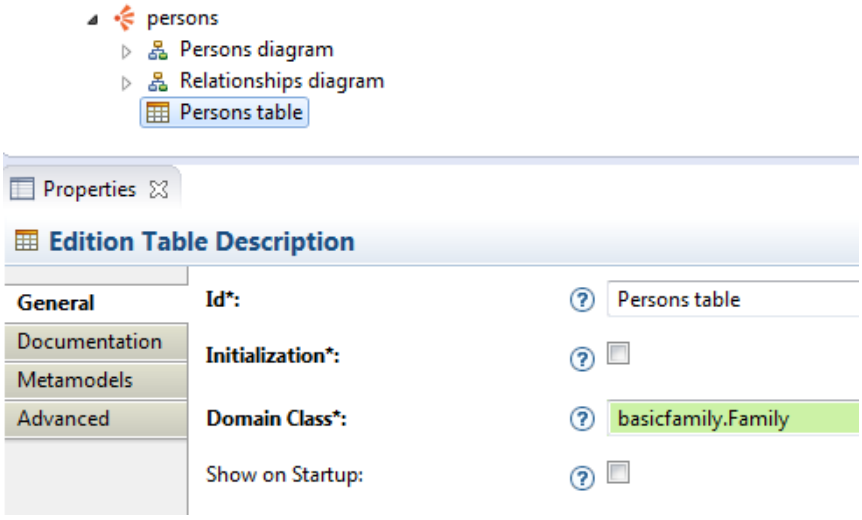
- father's name
- mother's name
- number of children

Right-click on the *Viewpoint* and select the menu **New representation > Edition Table Description**.

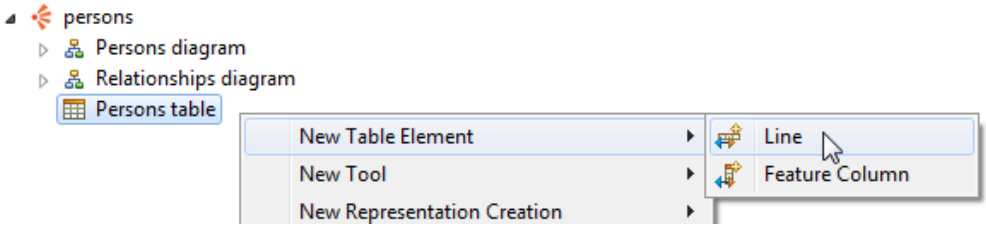


Set these properties to the table:

- **Id**: `Persons table`
- **Domain Class**: `basicfamily.Family`

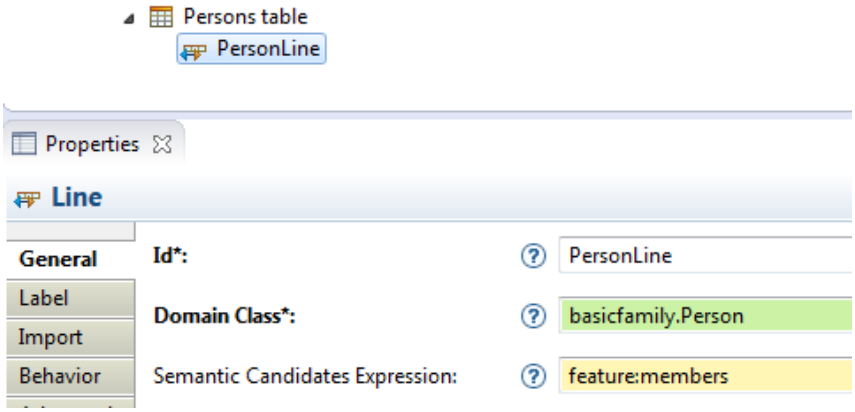


In this table, you need lines to represent the persons. So, right-click on the *Table* and select the menu **New Table Element > Line**.

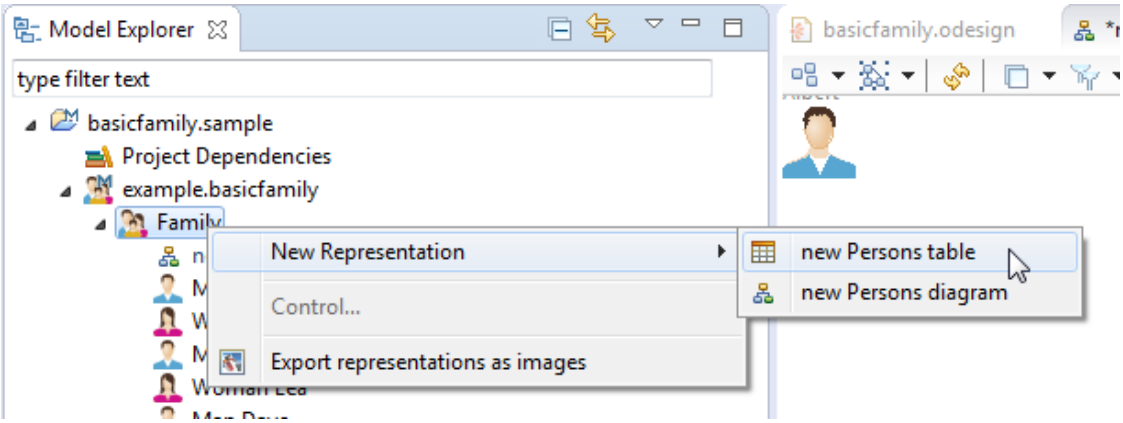


Set these properties to the line:

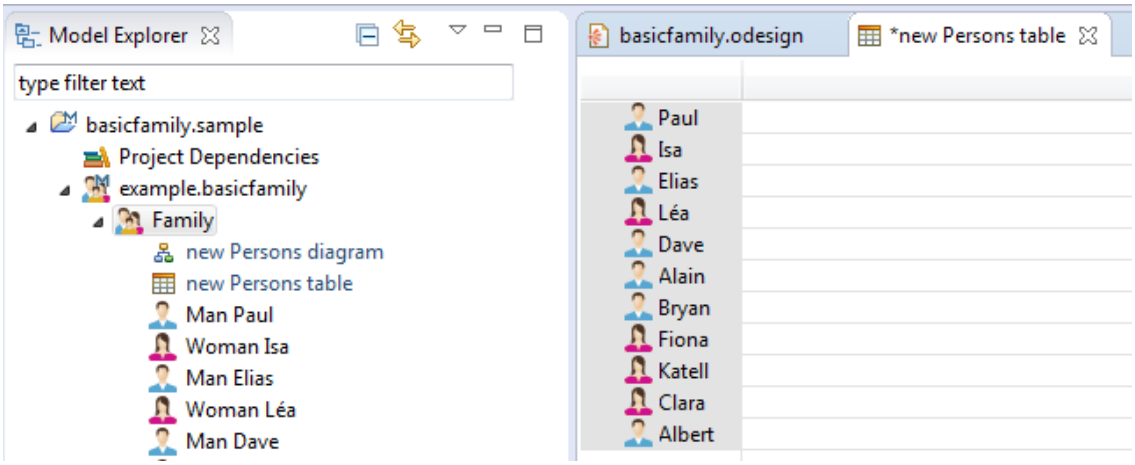
- **Id:** `PersonLine`
- **Domain Class:** `basicfamily.Person`
- **Semantic Candidates Expression:** `feature:members`
- **Header Label Expression:** (*Label* tab): `feature:name`



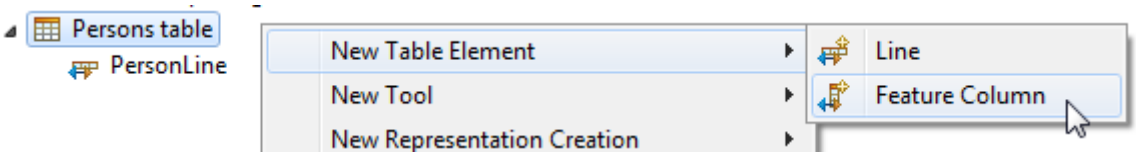
Save the *odesign* file then right-click on the *Family* instance in the *Model Browser* and click on the menu **New Representation**. You can see a new menu **new Persons table**.



If you select this menu, Sirius creates and opens a table with all the family's members.

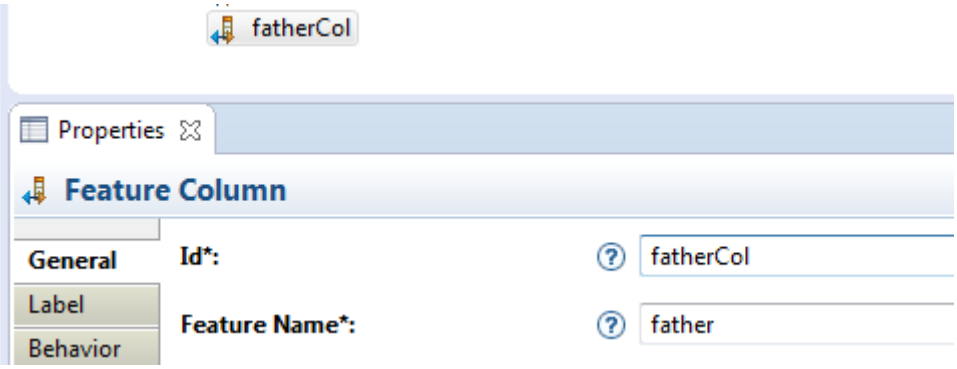


Now, let's add columns to this table to display more information for each person. So, right-click on the *Table* and select the menu **New Table Element > Feature Column**.



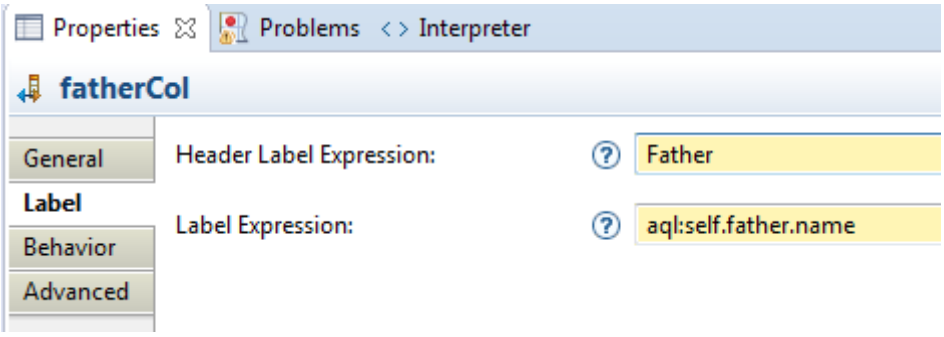
Set these properties to the column:

- **Id**: `fatherCol`
- **Feature Name**: `father`

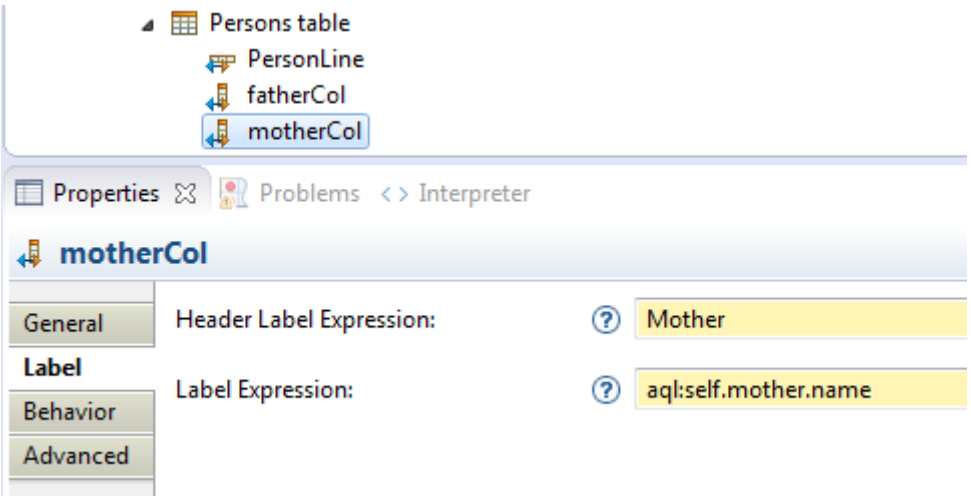


Select the *Label* tab and set these properties:

- Header Label Expression: `Father`
- **Label Expression**: `aql:self.father.name`



Copy and paste *fatherCol* then rename `father` by `mother` in all the properties.



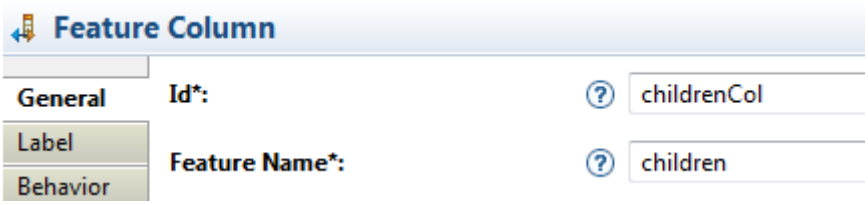
Save the *odesign* file and see the two new columns.

	Father	Mother
Paul		
Isa		
Elias	Paul	Isa
Léa	Paul	Isa
Dave	Elias	
Alain	Dave	Katell
Bryan	Elias	Léa
Fiona	Elias	
Katell		
Clara	Elias	Léa
Albert		

Finally, add a new *Feature Column* to display the number of children.

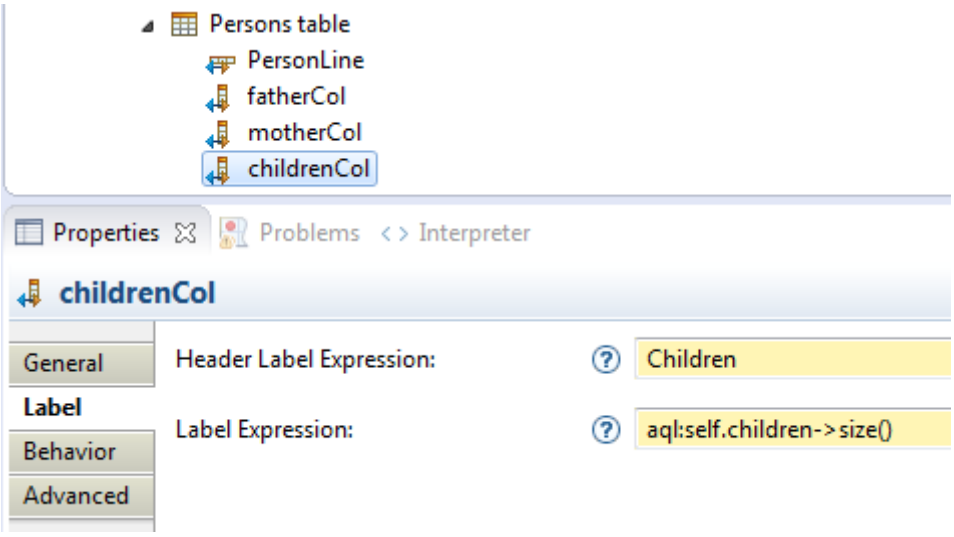
Set these properties to the column:

- **Id**: `childrenCol`
- **Feature Name**: `children`



Select the *Label* tab and set these properties:

- **Header Label Expression**: `Children`
- **Label Expression**: `aql:self.children->size()`



Save the *odesign* file and see the third column.

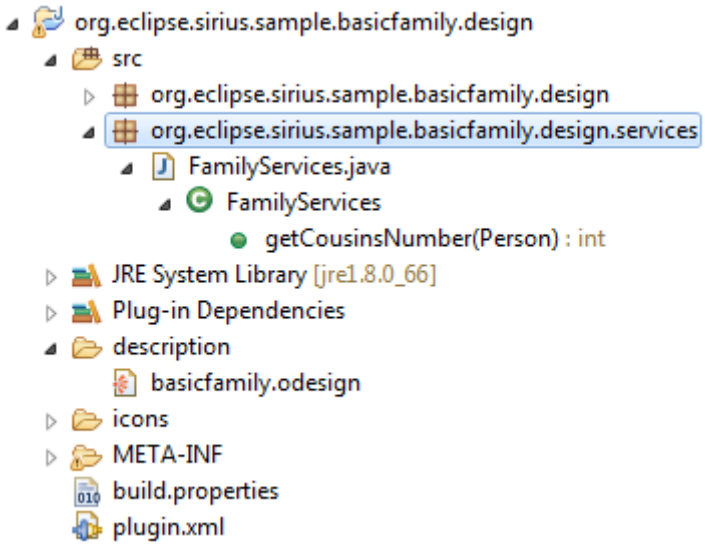
	Father	Mother	Children
Paul			2
Isa			2
Elias	Paul	Isa	4
Léa	Paul	Isa	2
Dave	Elias		1
Alain	Dave	Katell	0
Bryan	Elias	Léa	0
Fiona	Elias		0
Katell			1
Clara	Elias	Léa	0
Albert			0

Java Services

When AQL expressions become complex, you should write *Java Services* instead.

For example, let's create a *Java Service* to compute the number of cousins and display this information in the table.

Create a *Package* named `org.eclipse.sirius.sample.basicfamily.design.services` in your *Viewpoint Specification Project* (which contains the *odesign* file). Then create a *Java Class* named `FamilyServices` with a method named `getCousinsNumber`.

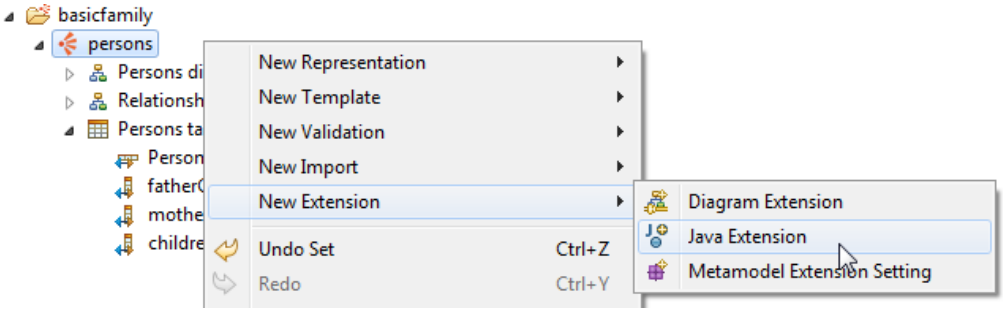


You can use the code below to complete this class:

```
package org.eclipse.sirius.sample.basicfamily.design.services;

import java.util.ArrayList;
import java.util.List;

import org.eclipse.sirius.sample.basicfamily.Person;

public class FamilyServices {

 public int getCousinsNumber(Person person) {
   List<Person> cousins=new ArrayList<Person>();
   List<Person> parents=person.getParents();

   for (Person parent: parents) {
        for (Person grandParent: parent.getParents()) {
                for (Person uncleOrAunt: grandParent.getChildren()) {
                        if (!parents.contains(uncleOrAunt)) {
                                for (Person cousin:uncleOrAunt.getChildren()) {
                                        if (!cousins.contains(cousin))
                                                cousins.add(cousin);
                                }
                        }
                }
        }
   }
   return cousins.size();
 }
}
```

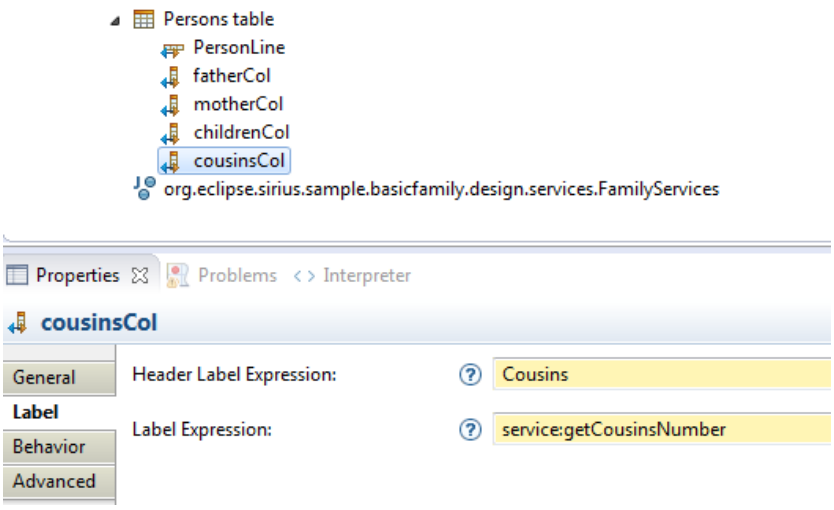
To be able to use the Java methods as *Services*, the class must be declared in the *odesign* file.

Right-click on the *Viewpoint* then select the menu **New Extension > Java Extension**.



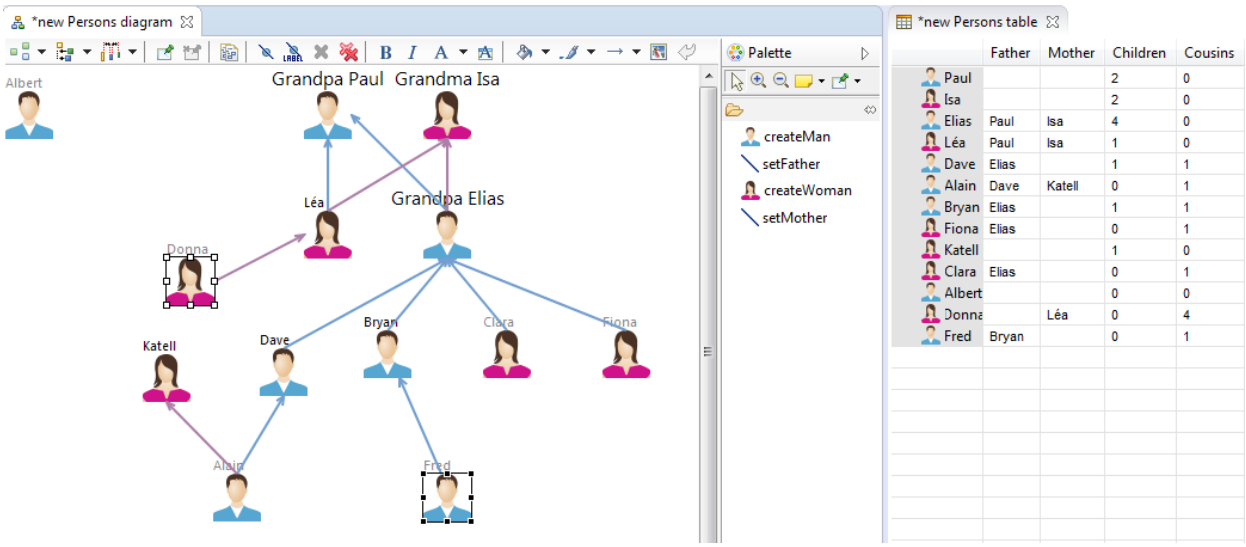
Then, set the *Qualified Class Name* property to `org.eclipse.sirius.sample.basicfamily.design.services.FamilyServices`.

To use the service *getCousinsNumber* in the table, create a new *FeatureColumn* named `cousinsCol` and invoke the service in the expression (`service:getCousinsNumber`).



Note: The *Feature Name* property can be set with any valid feature name (name, parents, ...), since the label will be computed independently (just based on the current person).

In the table, you can see a new column *Cousins* containing the number of cousins for each person (0). Use the diagram to create more persons such as some of them have cousins: Sirius will automatically update the number of cousins in the table.



The solution of is this last step is available in **sirius-tutorial-step4.zip**